Namibia University of Science and Technology · Faculty of Computing and Informatics

ABI820S · Advanced Business Intelligence and Analytics · Group project (30 %)

Green Basket cc – Agriculture Division: a single trusted view of production, yield, cost and revenue

End-to-end business intelligence solution: dimensional data warehouse, reproducible ELT with data-quality controls, Power BI semantic model with row-level security, OLAP and four advanced analytics techniques, interactive dashboards, performance optimisation and data governance.

Client (scenario)	Green Basket cc – Agriculture Division, Windhoek, Namibia
Submission	28 October 2026 (Week 16) · Final report, version 1.0
Data cut-off	30 September 2026 (day-2 master-data delta of 5 October 2026 included)
Platform	PostgreSQL 16 · Python 3.11 · Power BI Desktop (PBIP: TMDL + PBIR)

Team role	Report sections owned	Name	Student number	Signature
Member 1 – Project lead and business analyst	3.1, 3.9, presentation			
Member 2 – Data warehouse architect	3.2, data dictionary			
Member 3 – ETL, data quality and governance	3.3, 3.8			
Member 4 – Power BI developer	3.4, 3.6			
Member 5 – Analytics and performance lead	3.5, 3.7			

Declaration. We declare that this report and the accompanying artefacts are our own group work. Every source is acknowledged in APA style, all data are synthetic or public, and no personal information about real people is used. **Use of generative AI:** an AI assistant (Anthropic Claude) was used to draft code, documentation and analysis. The team executed, verified and edited every artefact and takes full responsibility for the content. Each member can explain the sections they own. This use is disclosed in line with the course's academic-integrity rules, and the team confirms it complies with the NUST policy in force.

Table of contents

Executive summary 3
- Assumptions and scope 4

3.1 Business case and requirements 5
- Organisation profile 5
- Business problem 6
- Business questions and KPIs 6
- Stakeholder analysis 7
- Process model 7

3.2 Data warehouse design 9
- Architecture 9
- Bus matrix 10
- Grain of every fact table 10
- Dimensional model (ERD) 11
- Data dictionary 18
- Comparison with alternative designs 18

3.3 ETL and data integration 20
- Platform choice: PostgreSQL 16 20
- Sources and conformed keys 20
- Pipeline: source → staging → warehouse 21
- Data profiling 22
- Data-quality rules and how each problem was handled 24
- Reconciliation checks 28
- Evidence that the warehouse was loaded 29
- Synthetic and public data 30

3.4 Power BI semantic model 32
- Storage mode: Import, with incremental refresh 32
- Relationships 32
- Model hygiene 33
- DAX measures 34
- Row-level security 36

3.5 Advanced analytics 38
- OLAP operations 38
- Technique 1: revenue forecasting (time series) 40
- Technique 2: customer segmentation (K-means clustering) 42
- Technique 3: what drives yield (regression and key influencers) 44
- Technique 4: anomaly detection in farm logs (Isolation Forest vs rules) 47

3.6 Interactive dashboard design 48
- Page designs 48
- Purposeful chart choice and visual hierarchy 50
- Accessibility 51
- Interactivity, used on purpose 51
- Design rationale: serving technical and non-technical users 52

3.7 Performance evaluation and optimisation 53
- Baseline and measurement method 53
- Optimisation techniques with before-and-after evidence 53
- Evaluation against the KPIs and objectives of §3.1 55

3.8 Data governance and security 57
- Ownership and stewardship 57
- Data-quality rules and monitoring 57
- Metadata and lineage 58
- Security and privacy 58
- Legal and ethical considerations 59
- Governance checklist for ongoing maintenance 59

3.9 Presentation, Q&A and individual reflection 60
- 10-minute presentation outline 60
- Q&A bank (5 minutes) 60
- Individual reflection templates 61

4 Submission package, self-assessment and limitations 63
- Submission package checklist 63
- Marking rubric self-assessment 63
- Limitations and next steps 64

References 65

Appendix A – Data dictionary 66

Appendix B – All DAX measures 80

Appendix C – Warehouse DDL **85**

Appendix D – Diagram sources and package inventory **94**

Mermaid sources of the diagrams 94

Package inventory 96

Executive summary

The problem. Green Basket cc's Agriculture Division runs 14 farms in 11 Namibian regions. It sells maize, wheat, mahangu, vegetables and table grapes to 240 customers across six channels, including export buyers in Europe, the UK and the Gulf. Its data lives in nine disconnected places:

- 14 farm harvest spreadsheets in three different layouts;
- a planting register and a master-data workbook;
- the ERP invoice extract, the stock system and the finance ledger;
- four external sources: weather, world prices, exchange rates and national crop estimates.

Management could not answer basic questions with confidence: which farms and crops make money, why yield missed plan, and what the next season will bring. Profiling found 2,587 duplicate invoice lines, 47 spellings of 10 crop names, three date formats across the harvest logs, and keying errors of ×10 in harvest quantities.

The solution. We built the components below. The whole build is reproducible with one command (`python run_all.py`, about 3 minutes).

- A Kimball dimensional warehouse in PostgreSQL 16:
  - 9 fact tables and 9 conformed dimensions;
  - a complete date dimension with Namibian public holidays;
  - a snowflaked product → crop → crop category hierarchy;
  - SCD Type 2 history on farms and customers.
- An ELT pipeline with 37 data-quality rules and 19 reconciliation checks.
- A Power BI semantic model: 79 DAX measures, three security roles and four report pages.
- Four advanced analytics techniques, each validated against held-out data or known ground truth.

N$ 458.8 m	**51 %**	**57 %**	**18 / 19**
FY2026 net revenue, +9.4 % on FY2025; 6.6 % a year since FY2022	of FY2026 revenue from table grapes; 46 % exported	of FY2026 revenue from the top-10 customers (concentration risk)	reconciliation checks PASS; 1 WARN = N$ 4.9 m awaiting price review

What the data says.

1. **Revenue is a grape business with a maize problem.**
   - Table grapes earned N$ 236.0 m in FY2026, and December alone N$ 182 m. A single month and crop carry the year, so the export desk's December logistics are the division's biggest operational risk.
   - Cereals earn only 34.8 % gross margin, against 60.6 % for fruit.
2. **Rainfall, not management, drives dryland yield.**
   - Holding crop, farm, heat and fertiliser constant, a 10 % rainfall shortfall cuts dryland yield by about 7 % (elasticity 0.73, $p < 0.001$; the generator's true value is about 0.77).
   - In the 2023/24 drought, dryland maize averaged 3.07 t/ha against a plan of 4.21 (−27 %). Irrigated maize held 8.59 t/ha.
   - Nkurenkuru Valley Farm went from 0.56–2.23 t per planted ha (dryland) to 7.44–8.64 t after its centre pivot was installed in August 2023. The SCD Type 2 farm history makes this before/after comparison possible.
3. **The forecast is flat.**
   - The best hold-out model projects FY2027 revenue of N$ 465.6 m (+1.5 %). Hold-out error is 12.7 % WAPE at division level and 15–16 % at category level.
   - Growth must come from mix and margin, not volume.
4. **Customers form five behavioural segments.** Twenty bulk and export key accounts produce 58.6 % of trailing 12-month revenue. Nineteen accounts have lapsed and are retention targets.

Recommendations.

1. Shift maize area from dryland to irrigated blocks where water rights allow. The measured yield gap of about 5 t/ha is worth roughly N$ 16,000 per hectare per season in margin, before capital cost (§3.5).
2. Hedge grape-export concentration with a second export window or market.
3. Make the warehouse the system of record for management reporting. Retire the Excel consolidation and run the 37 DQ rules weekly under named stewards (section 3.8).
4. Publish the Power BI app with the Zone Manager row-level security (section 3.4).

How to read this report.

- Sections 3.1–3.8 follow the brief.
- Section 3.9 holds the presentation outline, the Q&A bank and the reflection templates.
- Section 4 maps every required file to its location and self-assesses against the rubric.
- Every table of results is generated from the warehouse by the build scripts, and every code listing is copied verbatim from the submitted files. The PDF cannot drift from the code.

Assumptions and scope

Table 1. Assumptions (each one is tested or stated in the section shown)

#	Assumption	Consequence / where handled
A1	Green Basket cc is a teaching scenario. All internal data (farm logs, ERP, stock, ledger, master data) is **synthetic**, from a seeded generator (`01_data/generate_synthetic_data.py`, seed 20261028). Its volumes, prices and seasonality are anchored on public Namibian statistics.	No real person or company data is used (§3.8). The generator also records ground truth for every injected defect, so data-quality rules can be scored for precision and recall (§3.3).
A2	The external sources are real public datasets: NASA POWER daily weather, the World Bank Pink Sheet, FRED exchange rates, and NSA/MAFWLR crop estimates. The build environment blocked these hosts, so the generator writes stand-ins in the **exact published file layouts**.	`01_data/fetch_public_data.py` downloads the real files into the same paths with no code change. NSA/MAFWLR estimates are published as PDFs and must be transcribed (§3.3).
A3	The fiscal year runs April–March (FY2026 = 1 Apr 2025 – 31 Mar 2026). The agricultural season runs October–September (2024/25 = 1 Oct 2024 – 30 Sep 2025).	Both calendars are in `dim_date`. DAX year-to-date uses year end `"3/31"` (§3.4).
A4	Reporting currency is the Namibia dollar (N$). The N$ is pegged 1:1 to the South African rand, so FRED's ZAR/USD series converts USD, EUR and GBP.	FX is applied at the monthly average rate of the invoice month (rule FX01).
A5	The data cut-off is 30 Sep 2026. A day-2 master-data delta dated 5 Oct 2026 tests incremental loading and SCD Type 2.	FY2027 is a partial year (6 months), labelled as such everywhere.
A6	Scale: about 303,000 sales lines over 5.5 years, plus 8 further facts. We take this as meeting the brief's "required scale"; the team will confirm with the lecturer.	Performance tests use this volume (§3.7).
A7	Users, user principal names (`@greenbasket.example`, a reserved example domain) and manager codes are fictitious.	RLS tests use these accounts (§3.4).
A8	Cost of goods sold is the standard cost per kg at invoice line. Direct production costs come from the ledger by farm and crop.	Gross margin and cost per tonne are defined in §3.1 and Appendix A.
A9	Deployment target: PostgreSQL 16 on a Windows server, an on-premises data gateway, Power BI Pro workspace and app.	Licensing and governance in §3.8.

3.1 Business case and requirements

Organisation profile

Green Basket cc is a diversified Namibian close corporation. Its **Agriculture Division**, managed from Windhoek, grows and distributes cereals, vegetables and fruit. The division farms 4,330 cultivated hectares on 14 farms in 11 regions, grouped into four management zones. Its irrigation runs from dryland maize on the Oshikoto plains to drip-irrigated table grapes on the Orange River.

Table 2. Farm portfolio (current SCD Type 2 version, 30 Sep 2026)

Zone	Farm (code)	Region	Irrigation	Cultivated ha	Main crops
North-Central	Eenhana Dryland Farm (GB-OHA-01)	Ohangwena	Dryland	380	White maize, mahangu
North-Central	Outapi Canal Farm (GB-OMS-01)	Omusati	Drip (flood until Jun 2024)	140	Tomatoes, onions, cabbage, maize, watermelon
North-Central	Ongwediva Horticulture Hub (GB-OSH-01)	Oshana	Drip + shade-net	45	Tomatoes, onions, cabbage
North-Central	Tsumeb Triangle Farm (GB-OSK-01)	Oshikoto	Dryland (wheat on a borehole pivot block)	520	White maize, wheat
North-East	Mashare Riverside Farm (GB-KAE-01)	Kavango East	Centre pivot	520	Maize, wheat, potatoes, onions
North-East	Ndonga Linena Block (GB-KAE-02)	Kavango East	Drip	180	Maize, watermelon, butternut, cabbage
North-East	Nkurenkuru Valley Farm (GB-KAW-01)	Kavango West	Centre pivot (dryland until Jul 2023)	450	Maize, mahangu (dryland), watermelon
North-East	Liselo Floodplain Farm (GB-ZAM-01)	Zambezi	Centre pivot	560	Maize, wheat, tomatoes, cabbage
Central	Omaruru River Farm (GB-ERO-01)	Erongo	Drip + shade-net (from Oct 2026)	40	Tomatoes, onions, cabbage, butternut
Central	Grootfontein Plains Farm (GB-OTJ-01)	Otjozondjupa	Dryland	520	White maize
Central	Otavi Valley Farm (GB-OTJ-02)	Otjozondjupa	Centre pivot	470	Maize, wheat, potatoes
South	Hardap Dam Irrigation Farm (GB-HAR-01)	Hardap	Flood (canal)	260	Table grapes, maize, wheat, onions, butternut
South	Aussenkehr Orange River Vineyards (GB-KHA-01)	//Kharas	Drip	150	Table grapes (export), onions
South	Noordoewer River Farm (GB-KHA-02)	//Kharas	Drip	95	Table grapes, onions, butternut

Products and markets.

- **Products:** 21 sellable products from 10 crops in three categories (cereals and grains, vegetables, fruit).
- **Customers:** 240 customers in six channels: wholesale, retail, informal market, institutional (schools, hospitals), miller/processor, and export.
- **Export markets:** buyers in the Netherlands, Germany and the United Kingdom take table grapes in the November–January window. Fresh produce also goes to South Africa, Botswana, Angola and the UAE.
- **Regulation:** white maize, wheat and mahangu are controlled crops under the Namibian Agronomic Board (NAB), so domestic millers are a captive but price-regulated channel (Namibian Agronomic Board, 2021). The NAB's Market Share Promotion scheme supports local horticulture by obliging importers to buy a share of their produce locally: currently at least 47 %, as a condition of an import permit (Namibian Agronomic Board, n.d.).

Context. National production is volatile:

- Namibia harvested 94,262 tonnes of grain in 2025 (Windhoek Express, 2025a), a recovery after the severe 2023/24 drought.
- About 456,000 people (15 % of the population analysed) faced high acute food insecurity in July–September 2025, down from 1.15 million a year earlier, after above-average 2024/25 rainfall (Integrated Food Security Phase Classification [IPC], 2025).

Size. FY2026 net revenue was N$ 458.8 m. Table grapes earned 51 % of it, and exports 46 %.

Current strategy. The division's stated strategy is *"grow export horticulture, protect staple supply, and climate-proof production"*. In practice that means:

- expanding drip-irrigated grape and vegetable blocks;
- converting dryland maize to centre pivots where water rights allow (Nkurenkuru in 2023, the Omaruru shade-net in October 2026);
- keeping NAB-controlled grain flowing to millers.

The strategy is implemented through the five SMART objectives below. Each one is tracked by KPIs on the dashboards.

Table 3. Strategic objectives and the KPIs that measure them

#	Objective (SMART)	Baseline (FY2026 / season 2024/25)	Target	KPIs
O1	Grow net revenue at least 8 % a year to N$ 500 m by FY2028 while keeping gross margin at 50 % or more	N$ 458.8 m (+9.4 %), GM 53.8 %	≥ N$ 500 m, GM ≥ 50 %	Net Revenue, FYTD YoY %, Gross Margin %
O2	Close the yield gap: irrigated maize at or above plan and the NSA commercial benchmark; failed area below 3 %	Irrigated maize 8.68 t/ha vs plan 8.63; failed area 2.0 %	Yield vs plan ≥ 0 %, gap vs NSA ≥ 0	Yield (t/ha), Yield vs Plan %, Gap vs Benchmark, Failed Area %
O3	Reduce dependence on a few buyers: top-10 customer share below 50 % by FY2028, export share 40–50 %	Top-10 share 57.3 %, export 46.2 %	< 50 %, 40–50 %	Top 10 Customer Share %, Export Share %, segment revenue
O4	Cut direct production cost per tonne by 5 % in real terms by FY2028	Dryland maize N$ 3,270/t, irrigated N$ 3,850/t (2024/25)	−5 % real	Direct Cost per Tonne, cost by category
O5	One trusted source: 100 % of management KPIs from the warehouse; month-end pack within 3 working days; critical DQ rules ≥ 98 % clean	Excel consolidation; no DQ measurement	100 %; ≤ 3 days; ≥ 98 %	DQ rule results, reconciliation status, data freshness

Business problem

> **Problem statement.** The Agriculture Division cannot produce a timely, trusted and integrated view of production, yield, cost and revenue. Its data sits in nine unconnected sources with different codes, formats and quality levels. As a result, decisions on what to plant where, which blocks to irrigate, how to price and which customers to prioritise are made from manual spreadsheets that arrive late and disagree with each other.

The data profiling in §3.3 turns the problem into numbers:

- **Duplicates:** 2,587 duplicate ERP invoice lines from overlapping extracts. Left in, they overstate revenue.
- **Inconsistent names:** 47 spellings of 10 crop names and 217 non-standard farm codes in the harvest logs.
- **Mixed formats:** three harvest-log layouts with three date formats and four units of measure (kg, t, 10 kg crates, 400 kg bins).
- **Keying errors:** ×10 quantity errors that inflate a block to as much as 47 t/ha of maize.
- **Missing links:** 2,466 invoice lines with no farm of origin, and 694 lines for customers not yet in master data.
- **External data in awkward shapes:**
  - weather fill values of −999;
  - Pink Sheet "..." markers;
  - FX gaps on US holidays;
  - NSA region names that do not match the official list.

Without integration, management cannot link a yield shortfall (farm logs) to its cause (weather), its cost (ledger) and its revenue effect (ERP). That link is exactly what the business questions below need.

Business questions and KPIs

Table 4. Business questions, KPIs, definitions and where they are answered

BQ	Business question	KPIs (DAX measure)	Definition	Facts used	Page
BQ1	Which crops, channels, farms and customers drive revenue and margin, and where is growth slipping?	Net Revenue; Net Revenue FYTD; FYTD YoY %; Gross Margin %; Avg Realised Price (N$/kg); Export Share %	Net revenue = invoice + credit-note revenue after discount in N$; GM % = (net revenue − COGS) / net revenue; FYTD YoY compares like-for-like periods to the last data date	Sales	1 Executive
BQ2	How does yield compare with plan and the national benchmark by crop, farm, irrigation system and season, and what drives the gap?	Yield (t/ha); Yield vs Plan %; Yield Gap vs Benchmark (t/ha); Failed Area %; Rainfall per Season (mm)	Yield = harvested kg / 1,000 / harvested ha of completed cycles; benchmark = NSA commercial-sector yield of the same region, crop and season	Crop Cycle, Harvest, Weather, Crop Estimate	2 Production
BQ3	What does a tonne cost to produce, by farm and crop, and which inputs drive it?	Direct Cost per Tonne; Direct Cost; cost by category	Direct cost of completed cycles (ledger postings by farm, crop and season) / harvested tonnes	Production Cost, Crop Cycle	2 Production
BQ4	Which customer segments matter, how concentrated is revenue, and what should we expect in the next 12 months?	Top 10 Customer Share %; Net Revenue by segment; Forecast Revenue; Backtest WAPE %	Top-10 share = revenue of the 10 largest customers in context / total; WAPE = Σ|actual − forecast| / Σ|actual| over the 12-month hold-out	Sales, Forecast	3 Advanced

BQ	Business question	KPIs (DAX measure)	Definition	Facts used	Page
BQ5	Is stock under control, and can we trust the numbers?	Closing Stock (t); Stock Cover (days); Shrinkage %; Price Anomaly Lines; Unknown Farm Revenue %	Closing stock is semi-additive (last snapshot in period); cover = closing stock / average daily dispatch over 90 days	Inventory, Sales, audit tables	4 Operations

Stakeholder analysis

Table 5. Stakeholder groups, information needs and how the solution serves them

Group	Type	Information needs	Frequency	How served	Power / interest
Executive committee (managing director, CFO)	Non-technical	Revenue, margin, growth vs plan, concentration risk, 12-month outlook	Monthly, board packs	Page 1 KPI cards and trend; Power BI app; *Executive* role	High / high: sponsor
Zone and farm managers (4 zones)	Non-technical, operational	Their farms' yield vs plan, cost per tonne, stock, harvest progress; no access to other zones' commercial data	Weekly in season	Pages 2 and 4; drill-through to farm; *Zone Manager* dynamic RLS	Medium / high
Export desk and sales managers	Non-technical	Export volumes, prices, customer segments, December capacity	Weekly, daily in Nov–Jan	Pages 1 and 3; *Export Desk* static RLS	Medium / high
Agronomist	Semi-technical	Yield drivers, rainfall response, irrigation effect, varietal results	Per season	Page 2; regression results (§3.5)	Medium / high
Finance team	Semi-technical	Reconciled revenue and cost, margin by farm and crop, control totals	Month-end	Pages 1, 2, 4; reconciliation checks R05 and R09	High / medium
Data stewards (farm, sales, finance, master data, external)	Technical / business	DQ rule results, quarantine queues, unmatched codes	Daily or weekly	Page 4 DQ cards; `audit.dq_issue_log`; `mart.dq_summary`	Medium / high
BI developers and data engineers	Technical	Lineage, load logs, model performance, reproducible builds	Continuous	`audit.etl_batch` and `etl_step_log`; PBIP in Git; Performance Analyzer protocol	Medium / high
IT / database administrator	Technical	Security, backup, refresh schedule, gateway	Weekly	Least-privilege database roles; `pg_dump` backup; incremental refresh	High / low
External: NAB, NSA, auditors	Non-technical	Statutory returns, traceability, audit trail	Ad hoc	Exports from `mart`; batch-stamped lineage	Medium / low

Engagement implications.

- The sponsor (executive committee) needs one screen with the five headline KPIs. That screen is page 1.
- Zone managers are the largest user group and the main source of harvest data, so their dashboards double as the data-quality feedback loop: the "Corrected records" column on page 4 shows each farm its own keying errors.
- The technical groups need reproducibility and lineage. Those are covered in §3.3 and §3.8.

Process model

The modelled process is **"plan → plant → harvest → pack and store → sell and invoice"**, the order-to-cash chain for farm produce. Each step creates data in a different system. Those systems (S1–S5) are the internal sources of the warehouse; S6–S9 are the external ones.

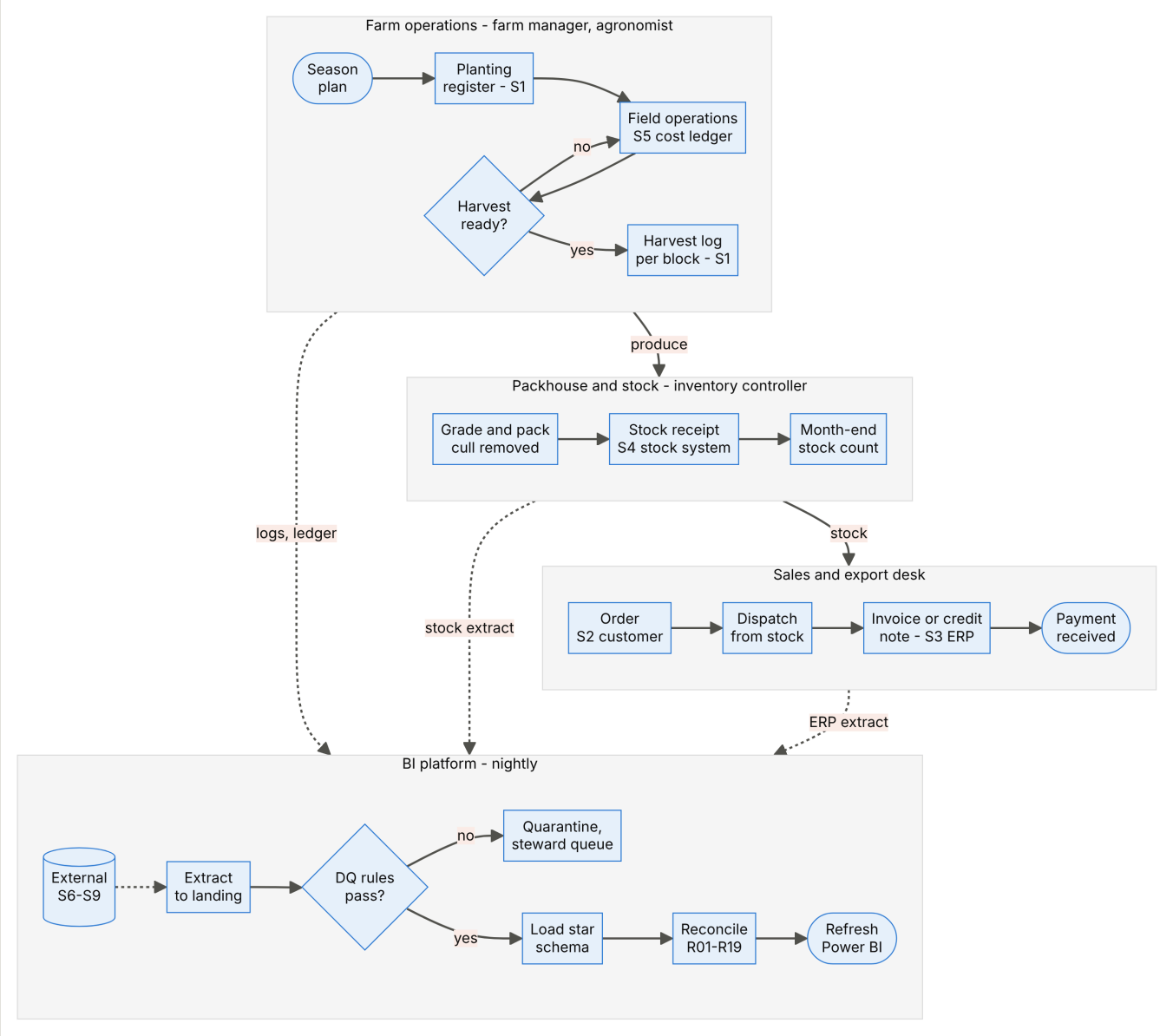


Figure 1. BPMN-style process flow: business lanes with their data capture points (S1-S9) feeding the nightly BI platform lane (Mermaid source: `06_report/diagrams/bpmn.mmd`)

What the model reveals. These are the design drivers for the warehouse:

1. **Harvest, stock and sales describe the same kilograms, but at three different points.** The harvest log records field weight (before cull). The stock system records packed weight. The ERP records invoiced weight. The warehouse keeps all three facts at their own grain and reconciles them across sources (checks R06 and R18 in §3.3).
2. **The farm of origin is lost at dispatch** unless the invoice line carries it. Rule SA05 measures that gap (0.81 % of lines).
3. **External context only matters once it is joined on conformed farm, region, crop and date keys.**
   - Weather joins by farm location and day.
   - Prices join by crop and month.
   - NSA estimates join by region, crop and season.

3.2 Data warehouse design

Architecture

The warehouse follows the Kimball bus architecture (Kimball & Ross, 2013): conformed dimensions are shared by process-oriented fact tables. It is implemented in **PostgreSQL 16** as an **ELT** pipeline. Raw extracts are landed unchanged, then cleansed and conformed in SQL inside the database, where the transformations are versioned, testable and close to the data. The schemas below separate concerns and privileges.

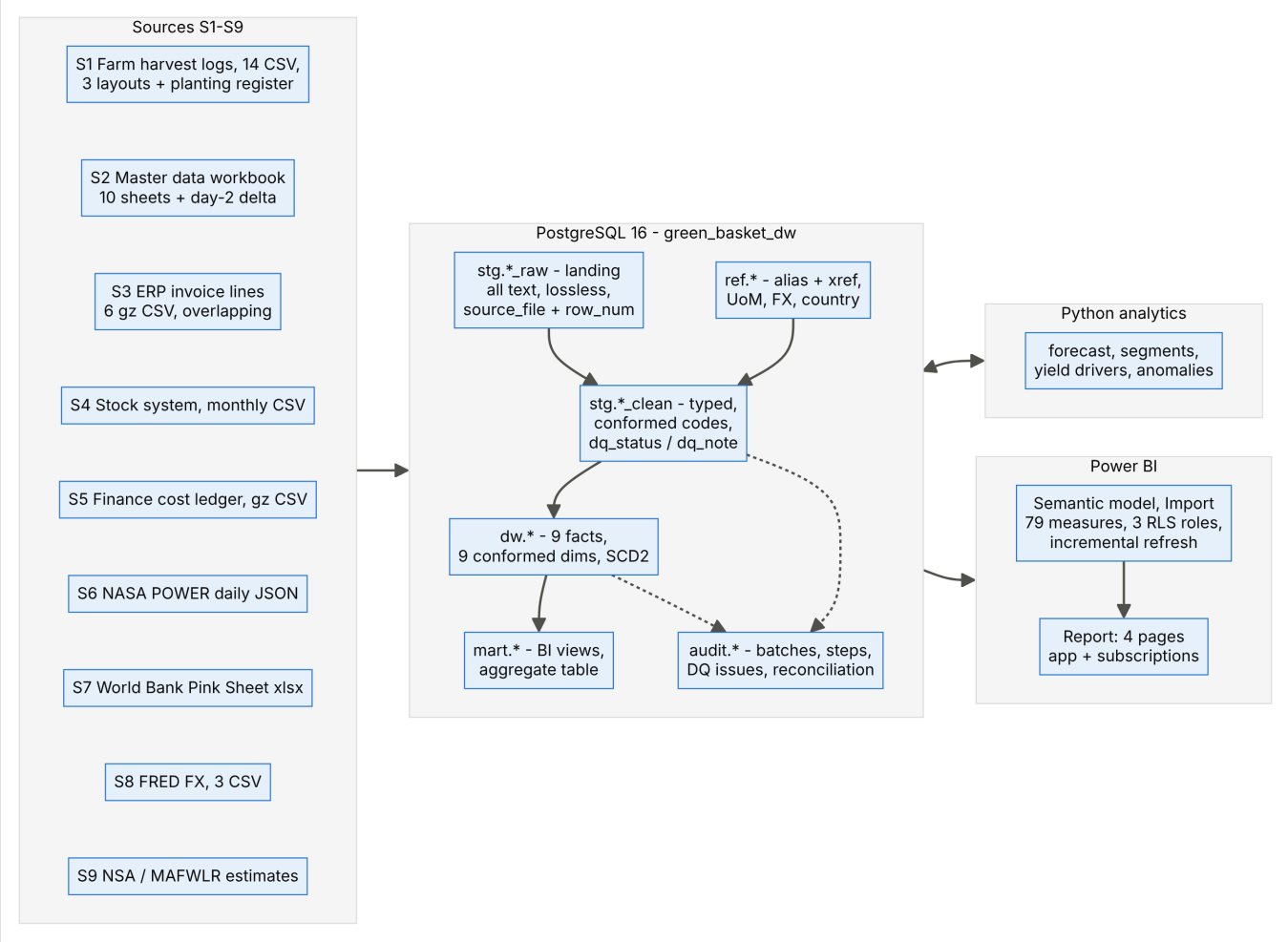


Figure 2. Solution architecture: sources, ELT layers in PostgreSQL, semantic model and consumers (Mermaid source: `06_report/diagrams/architecture.mmd`)

Table 6. Schemas and their role (least privilege per schema; grants in 02_sql/10_security_grants.sql)

Schema	Content	Written by	Read by
`stg`	Landing tables (`*_raw`, everything as text plus source file and row number) and cleansed tables (`*_clean`, typed, with `dq_status` and `dq_note`)	`gb_etl_loader`	ETL only
`ref`	Mapping and reference data: region, crop and cost-category aliases; customer cross-reference; units of measure; monthly FX; countries	ETL and stewards	ETL
`dw`	Dimensional model: 9 dimensions and 9 facts, plus 3 analytics-output tables	`gb_etl_loader`	ETL, analytics
`mart`	Views shaped for Power BI (friendly names, only the columns needed) plus the aggregate `agg_sales_monthly`	ETL	`gb_pbi_reader`
`audit`	`etl_batch`, `etl_step_log`, `dq_rule`, `dq_issue_log`, `recon_result`	ETL	stewards, developers
`etl`	Functions and stored procedures (`sp_*`, `fn_*`)	developers	ETL
`sec`	RLS mapping of user principal name to zone (`user_zone_access`)	data owner	Power BI (RLS)

Bus matrix

Each row is a business process measured by a fact table. Each column is a conformed dimension. A tick means the process carries that dimension's key, directly or through a snowflaked parent ("via").

Table 7. Enterprise bus matrix: 9 business processes x conformed dimensions

Business process (fact table)	Fact type	Date	Farm	Region	Crop	Crop category	Product	Customer	Cost category	Commodity
Sales invoicing (`fact_sales`)	Transaction	✔ invoice + dispatch (role-playing)	✔ origin	via farm	via product	via crop	✔	✔ SCD2		
Harvest recording (`fact_harvest`)	Transaction	✔ harvest	✔	via farm	✔	via crop				
Crop-cycle management (`fact_crop_cycle`)	Accumulating snapshot	✔ planting, first and last harvest, season end	✔ as-was	via farm	✔	via crop				
Production costing (`fact_production_cost`)	Transaction	✔ posting	✔	via farm	✔ (−2 = overhead)	via crop			✔	
Inventory control (`fact_inventory_monthly`)	Periodic snapshot	✔ month-end	✔	via farm	via product	via crop	✔			
Weather monitoring (`fact_weather_daily`)	Periodic snapshot (day)	✔	✔ location	via farm						
World price monitoring (`fact_market_price_monthly`)	Periodic snapshot (month)	✔ month			✔ mapped	via crop				✔
National benchmark (`fact_crop_estimate`)	Periodic snapshot (season)	✔ season end		✔ direct	✔	via crop				
Revenue forecasting (`fact_forecast_monthly`)	Periodic snapshot (model output)	✔ month				✔				

Why it matters. Date, Farm, Region and Crop are **conformed**: they carry identical keys and attributes in every fact. That makes drill-across possible:

- yield (crop cycle) against national yield (NSA estimate) on region × crop × season (§3.5);
- harvest against stock received on farm × month (reconciliation R18);
- revenue against rainfall on farm × season.

Grain of every fact table

Table 8. Declared grain, measures and additivity of each fact (row counts after the 30 Sep 2026 load)

Fact	Grain (one row per …)	Measures and additivity	Degenerate dims	Rows
`fact_sales`	invoice or credit-note **line** (`invoice_no` + `line_no`)	quantity_kg, gross revenue, discount, **net revenue**, COGS, freight: additive. unit_price_txn, fx_rate_nad: non-additive	invoice_no, line_no, doc_type, currency	302,903
`fact_harvest`	harvest **weighing record** of a field block on a day (source file + row)	harvested_kg: additive	block_id, recorder_code	7,368
`fact_crop_cycle`	planted **field block × crop cycle**; milestones updated in place	planted, harvested and planned kg, ha, direct cost, fertiliser: additive. planned_yield_t_ha, days_to_first_harvest: non-additive	cycle_id, block_id, variety, irrigation_method, status	1,327
`fact_production_cost`	**ledger posting line**	amount_nad: additive. input_qty: additive within one cost category	posting_id, cost_centre	15,382
`fact_inventory_monthly`	**farm × product × month-end**	opening and closing stock (kg, N$): **semi-additive** (not over time). received, dispatched, shrinkage: additive	–	6,521
`fact_weather_daily`	**farm location × day**	rainfall: additive over time, *averaged* across locations. temperatures, humidity, radiation: non-additive	–	30,674
`fact_market_price_monthly`	**commodity series × month**	price USD and N$ per tonne: non-additive (average)	–	486

Fact	Grain (one row per …)	Measures and additivity	Degenerate dims	Rows
`fact_crop_estimate`	**region × crop × season × farming sector**	area planted (ha), production (t): additive. yield (t/ha): non-additive	farming_sector	324
`fact_forecast_monthly`	**month × crop category × model × scenario**	actual, forecast and interval bounds (N$): additive across categories within one model	model_name, scenario	216

Three fact types are used on purpose:

- **Transaction facts** (sales, harvest, cost) keep the lowest grain, so every aggregate can be derived from them.
- **The accumulating snapshot** (crop cycle) answers “how is this planting doing?” without re-aggregating thousands of harvest rows.
- **The periodic snapshot** (inventory) stores balances. Stock levels cannot be summed over time, so the DAX measure takes the last snapshot (§3.4) and the SQL roll-up does the same (§3.5, semi-additive example).

Dimensional model (ERD)

The model is a **constellation**: nine star schemas share their dimensions. It is shown in three diagrams for legibility. Primary keys (PK) and foreign keys (FK) are marked. The complete column list is in Appendix A.

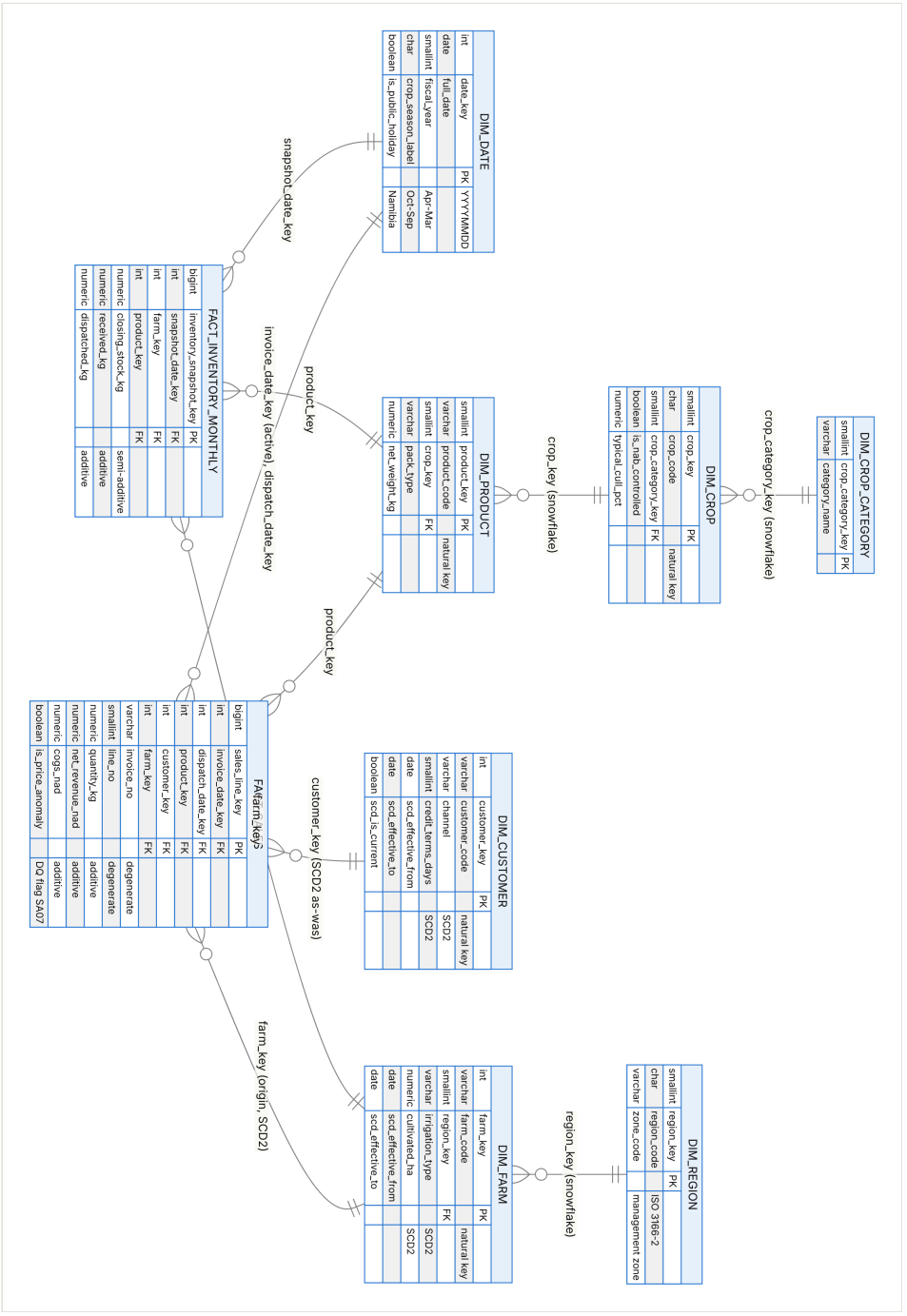


Figure 3. ERD 1 - Sales and inventory stars with the snowflaked product > crop > crop category and farm > region hierarchies (rotated to use the full page) (Mermaid source: `06_report/diagrams/erd_sales.mmd`)

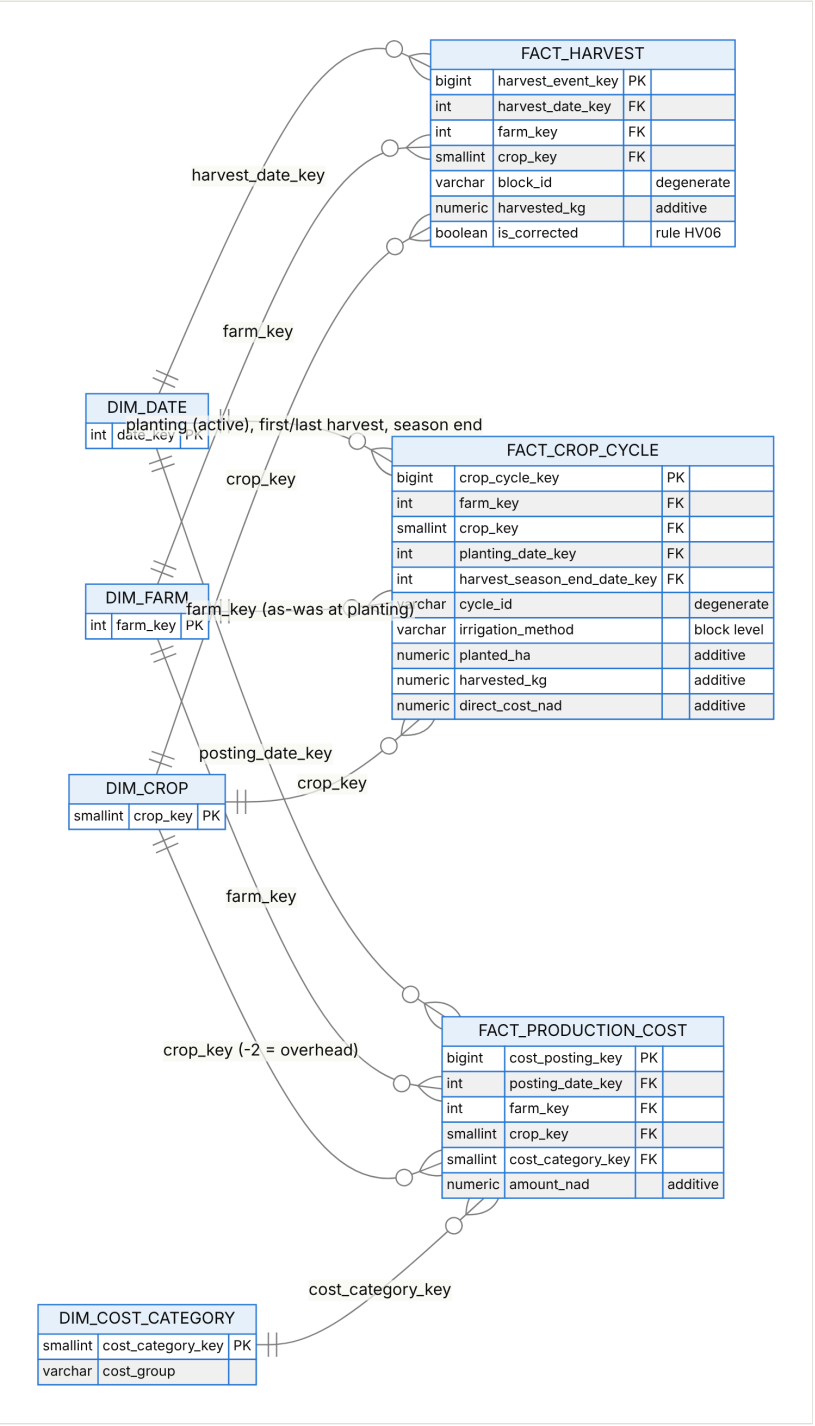


Figure 4. ERD 2 - Production and cost facts sharing the conformed Date, Farm and Crop dimensions (Mermaid source: `06_report/diagrams/erd_production.mmd`)

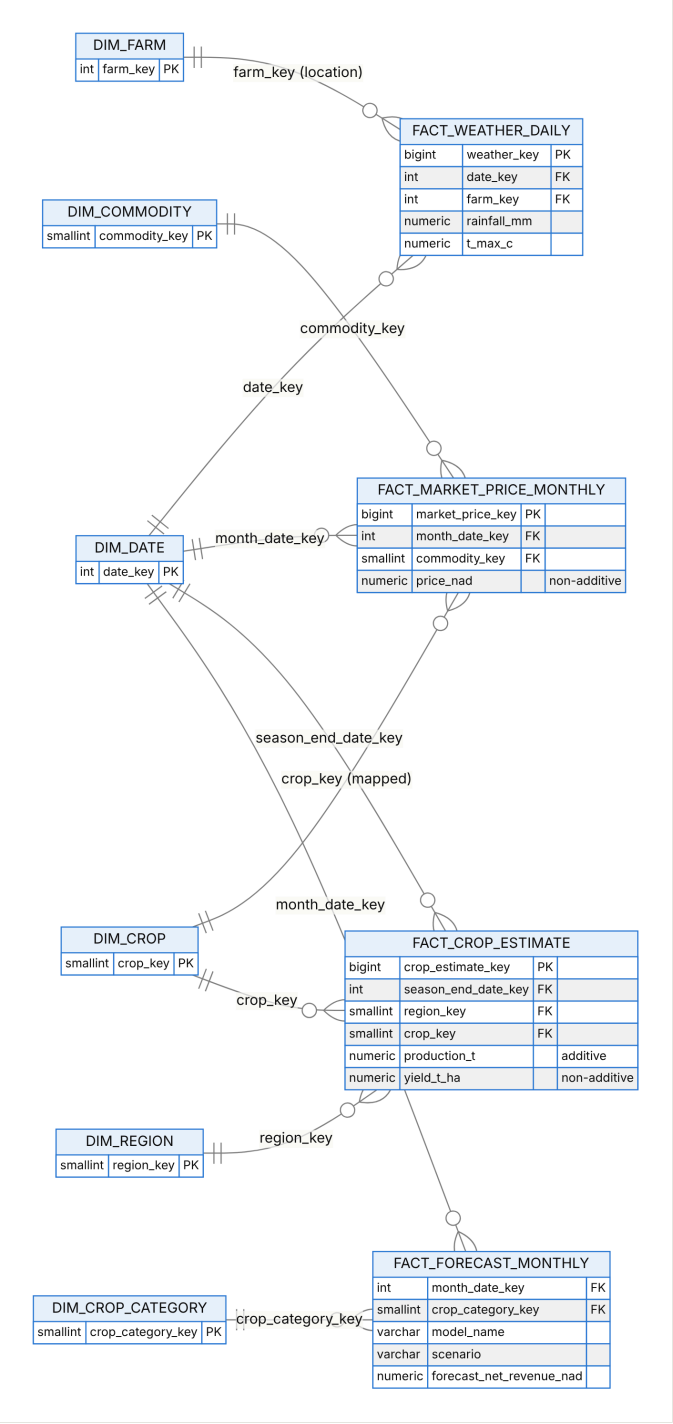


Figure 5. ERD 3 - External benchmark facts (weather, world prices, NSA estimates) and the forecast output (Mermaid source: `06_report/diagrams/erd_external.mmd`)

Dimensions

The model has nine dimensions, well above the required four.

- **`dim_date` is complete.** It has one row per day from 1 Jan 2020 to 31 Dec 2027: 2,922 days plus the −1 "unknown" row. Reconciliation check R14 verifies there are no gaps.
- **Calendar attributes** cover week, month and quarter, with sort keys.
- **The April–March fiscal calendar** gives `fiscal_year`, `fiscal_quarter_label` and `fiscal_month_num`, used for sort-by-column in Power BI.
- **The October–September agricultural season** gives `crop_season_label`, which conforms with the NSA estimates.
- **Working-day logic** includes **Namibian public holidays**, computed by `etl.fn_namibian_holiday`:
  - fixed dates such as Independence Day (21 March) and Heroes' Day (26 August);
  - Easter-dependent holidays: Good Friday, Easter Monday and Ascension Day, from `etl.fn_easter_sunday`;
  - the rule that a holiday falling on a Sunday moves to the Monday.

The function is reproduced below because the brief asks for a *complete* date dimension.

File: `02_sql/04_etl_functions.sql` (lines 66–75)

```
CREATE OR REPLACE FUNCTION etl.fn_easter_sunday(p_year int)
RETURNS date LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE a int; b int; c int; d int; e int; f int; g int; h int; i int; k int; l int; m int; mon int; dy int;
BEGIN
    a := p_year % 19; b := p_year / 100; c := p_year % 100; d := b / 4; e := b % 4;
    f := (b + 8) / 25; g := (b - f + 1) / 3; h := (19 * a + b - d - g + 15) % 30;
    i := c / 4; k := c % 4; l := (32 + 2 * e + 2 * i - h - k) % 7; m := (a + 11 * h + 22 * l) / 451;
    mon := (h + l - 7 * m + 114) / 31; dy := ((h + l - 7 * m + 114) % 31) + 1;
    RETURN make_date(p_year, mon, dy);
END $$;
```

File: `02_sql/04_etl_functions.sql` (lines 79–99)

```
CREATE OR REPLACE FUNCTION etl.fn_namibian_holiday(p_date date)
RETURNS text LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE y int := extract(year FROM p_date); e date; fixed record; obs date;
BEGIN
    e := etl.fn_easter_sunday(y);
    IF p_date = e - 2 THEN RETURN 'Good Friday'; END IF;
    IF p_date = e + 1 THEN RETURN 'Easter Monday'; END IF;
    IF p_date = e + 39 THEN RETURN 'Ascension Day'; END IF;
    FOR fixed IN
        SELECT * FROM (VALUES (1, 1, 'New Year''s Day'), (3, 21, 'Independence Day'), (5, 1, 'Workers'' Day'),
                              (5, 4, 'Cassinga Day'), (5, 25, 'Africa Day'), (5, 28, 'Genocide Remembrance Day'),
                              (8, 26, 'Heroes'' Day'), (12, 10, 'Human Rights Day'), (12, 25, 'Christmas Day'),
                              (12, 26, 'Family Day')) AS t(m, d, n)
    LOOP
        IF fixed.n = 'Genocide Remembrance Day' AND y < 2025 THEN CONTINUE; END IF;
        obs := make_date(y, fixed.m, fixed.d);
        IF p_date = obs THEN RETURN fixed.n; END IF;
        IF extract(isodow FROM obs) = 7 AND p_date = obs + 1 THEN RETURN fixed.n || ' (observed)'; END IF;
    END LOOP;
    RETURN NULL;
END $$;
```

The other dimensions:

- `dim_farm`: SCD Type 2, 14 farms with 20 versions.
- `dim_customer`: SCD Type 2, 240 customers with 277 versions, including late-arriving inferred members.
- `dim_product` → `dim_crop` → `dim_crop_category`: the snowflake.
- `dim_region`: 14 ISO 3166-2 regions plus the management zones.
- `dim_cost_category`: 14 ledger categories in 6 cost groups.
- `dim_commodity`: 6 World Bank series.

Snowflaked dimensions and their trade-offs

There are two deliberate snowflakes: **product → crop → crop category**, and **farm → region**, with management zone as an attribute of region. Customer geography, by contrast, is flattened into `dim_customer`. Each dimension is designed for its use, not for a rule.

Table 9. Star (flattened) vs snowflake for the crop hierarchy

Criterion	Flattened `dim_product` (star)	Snowflake product → crop → category (chosen)
Conformance	Fails. Harvest, crop-cycle, cost, market-price and NSA-estimate facts are recorded **per crop, not per product**, so they would need fake "product = crop" rows or their own crop dimension. Two crop dimensions would drift.	One `dim_crop` is shared by 6 facts directly and reached from sales and inventory through product. Drill-across on crop is exact.
Maintainability	Crop attributes (NAB-controlled, cull %, shelf life) are repeated on 21 product rows. Every correction touches several rows (update anomaly).	Each attribute is stored once. Rule MD03 and steward edits touch one row.
Storage	21 rows × about 8 repeated crop attributes: negligible.	Two small extra tables: negligible. Storage is **not** the deciding factor at this size.
Query performance (SQL)	One join fewer.	One or two extra joins on tables of 12 and 5 rows. The measured cost is below 1 ms. Fact access dominates (§3.7).
Power BI usability	Simplest field list.	Two extra 1:* relationships, and filters flow one way. Drill-down Category › Crop › Product works by stacking the three fields on a visual axis. VertiPaq cost is negligible.
Risk	Grain mismatch, so totals are wrong when crop-level facts are joined to products.	Users must understand that Crop filters Sales through Product. The display folders and the hierarchy hide this.

Region is snowflaked off farm for the same reason: `fact_crop_estimate` is recorded per **region**, and farm location rolls up to region. Customer geography (town, region, zone, country, market group) is **flattened**. No fact is recorded at customer-region grain, so a snowflake there would add joins without adding conformance.

Surrogate keys, special members and natural keys

- **Integer surrogate keys** on every dimension:
  - identity columns for the SCD Type 2 dimensions, because each version needs a new key;
  - managed keys for the small reference dimensions.
- **The smart date key** `YYYYMMDD` (for example 20260930) is a deliberate exception. It is readable, sorts correctly, and lets the Power BI incremental-refresh filter fold into PostgreSQL as an integer range (§3.4, §3.7).

- **Special members** are always present and keep facts referentially complete with `NOT NULL` foreign keys:
  - **−1 Unknown**, for an unmatched or missing code, such as the 2,466 sales lines without a farm of origin (rule SA05);
  - **−2 Not applicable**, for example farm-overhead costs that belong to no crop.
- **Natural keys** (`farm_code`, `customer_code`, `product_code`, `crop_code`, `region_code`) are kept as alternate keys for lookup and lineage, but never used as relationship keys.
- **Degenerate dimensions** (`invoice_no`, `cycle_id`, `block_id`) stay on the facts.

Slowly changing dimension strategy

Table 10. SCD decision per dimension attribute

Dimension	Attributes	SCD type	Reason
`dim_farm`	irrigation_type, cultivated_ha, water_source, farm_manager_code, GlobalGAP certification	**Type 2**	They change the farm's cost and yield structure. History must stay *as-was*. A season planted under dryland must not be re-labelled "centre pivot" after an upgrade.
`dim_farm`	farm_name, town, coordinates, region, total ha	Type 1	Corrections, not business changes (e.g. the GB-KAE-02 name fix of 15 Jul 2022).
`dim_customer`	channel, credit_terms_days, is_key_account	**Type 2**	Revenue by channel must reflect the channel *at the time of sale* (e.g. C00105 moved from retail to wholesale on 10 Jan 2024).
`dim_customer`	name, town, contact formatting	Type 1	Data cleaning (rule MD05).
`dim_customer`	inferred (late-arriving) members	Type 1 completion	A placeholder created by the sales load (rule SA04) is completed in place when master data arrives, so no version is created.
`dim_product`, `dim_crop`, `dim_region`, `dim_cost_category`	all	Type 1	Reference data. Changes are corrections.
`dim_date`	all	Type 0	Fixed.

We also considered **Type 3** (a "previous value" column) and rejected it. Attributes change more than once: customer C00105 has three versions in ten months, and farms change irrigation, area, water source, certification and manager on different dates, so one previous-value column is not enough. For "as-is" reporting, the mart view of `dim_farm` adds **current-value columns**, for example `irrigation_type_current` taken from the current version. This hybrid of Type 2 history plus current attributes (sometimes called Type 6) serves both questions without a second dimension.

How SCD Type 2 is implemented (`etl.sp_load_dim_farm_scd2`):

1. Each farm's first version is reconstructed as at 1 Jan 1900 from the change log (`Farm_History` sheet), using `etl.fn_attr_asof`.
2. Change events are replayed in date order, grouped by farm and effective date.
3. Each candidate version is compared by an **MD5 hash** of its Type 2 attributes, so a re-run creates nothing (idempotent).
4. When the hash differs, the current row is expired (`scd_effective_to = effective_date − 1`) and a new current row is inserted.
5. Type 1 attributes are overwritten on all versions.

Facts pick the version valid on the event date with `event_date BETWEEN scd_effective_from AND scd_effective_to` (see the `fact_sales` loader in §3.3).

File: `02_sql/06_etl_load_dimensions.sql` (lines 185–258)

```
        CREATE OR REPLACE PROCEDURE etl.sp_load_dim_farm_scd2(p_batch integer)
        LANGUAGE plpgsql AS $$
        DECLARE
            t0 timestamptz := clock_timestamp();
            ev record; cur dw.dim_farm%ROWTYPE; src stg.md_farm_clean%ROWTYPE;
            v_cult text; v_irr text; v_water text; v_mgr text; v_gap text; v_hash text;
            n_initial int := 0; n_versions int := 0; n_t1 int := 0;
        BEGIN
            INSERT INTO dw.dim_farm (farm_key, farm_code, farm_name, region_key, irrigation_type, is_irrigated, scd_effective_from,
                                     scd_is_current, scd_version, scd_row_hash, dw_batch_id)
            VALUES (-1, 'UNKNOWN', 'Unknown farm', -1, 'Unknown', false, DATE '1900-01-01', true, 1, md5('unknown'), p_batch)
            ON CONFLICT (farm_key) DO NOTHING;

            -- 1) first version for farms not yet in the dimension = state as at 1900-01-01
            INSERT INTO dw.dim_farm (farm_code, farm_name, region_key, town, latitude, longitude, elevation_m, total_ha, established_year,
                                     harvest_log_template, cultivated_ha, irrigation_type, is_irrigated, water_source, farm_manager_code,
                                     is_globalgap_certified, scd_effective_from, scd_is_current, scd_version, scd_row_hash, dw_batch_id)
            SELECT f.farm_code, f.farm_name, coalesce(r.region_key, -1), f.town, f.latitude, f.longitude, f.elevation_m, f.total_ha,
                   f.established_year, f.harvest_log_template,
                   a.cult::numeric, a.irr, a.irr <> 'Dryland', a.water, a.mgr, a.gap = 'Y',
                   DATE '1900-01-01', true, 1, md5(concat_ws('|', a.cult, a.irr, a.water, a.mgr, a.gap)), p_batch
            FROM stg.md_farm_clean f
            LEFT JOIN dw.dim_region r ON r.region_code = f.region_code
            CROSS JOIN LATERAL (
                SELECT trim_scale(etl.fn_attr_asof('FARM', f.farm_code, 'cultivated_ha', DATE '1900-01-01',
f.cultivated_ha::text)::numeric)::text AS cult,
                       etl.fn_attr_asof('FARM', f.farm_code, 'irrigation_type', DATE '1900-01-01', f.irrigation_type) AS irr,
                       etl.fn_attr_asof('FARM', f.farm_code, 'water_source', DATE '1900-01-01', f.water_source) AS water,
                       etl.fn_attr_asof('FARM', f.farm_code, 'farm_manager_code', DATE '1900-01-01', f.farm_manager_code) AS mgr,
                       etl.fn_attr_asof('FARM', f.farm_code, 'globalgap_certified', DATE '1900-01-01', f.globalgap_certified) AS gap) a
            WHERE NOT EXISTS (SELECT 1 FROM dw.dim_farm d WHERE d.farm_code = f.farm_code);
```

```
            GET DIAGNOSTICS n_initial = ROW_COUNT;

            -- 2) replay Type 2 change events in date order (one new version per farm per effective date)
            FOR ev IN
                SELECT entity_code, effective_date FROM stg.md_history_clean
                WHERE entity = 'FARM' AND attribute IN ('cultivated_ha', 'irrigation_type', 'water_source', 'farm_manager_code',
'globalgap_certified')
                GROUP BY entity_code, effective_date ORDER BY effective_date, entity_code
            LOOP
                SELECT * INTO cur FROM dw.dim_farm WHERE farm_code = ev.entity_code AND scd_is_current;
                CONTINUE WHEN NOT FOUND OR ev.effective_date <= cur.scd_effective_from;
                SELECT * INTO src FROM stg.md_farm_clean WHERE farm_code = ev.entity_code;
                v_cult  := trim_scale(etl.fn_attr_asof('FARM', ev.entity_code, 'cultivated_ha', ev.effective_date,
src.cultivated_ha::text)::numeric)::text;
                v_irr   := etl.fn_attr_asof('FARM', ev.entity_code, 'irrigation_type', ev.effective_date, src.irrigation_type);
                v_water := etl.fn_attr_asof('FARM', ev.entity_code, 'water_source', ev.effective_date, src.water_source);
                v_mgr   := etl.fn_attr_asof('FARM', ev.entity_code, 'farm_manager_code', ev.effective_date, src.farm_manager_code);
                v_gap   := etl.fn_attr_asof('FARM', ev.entity_code, 'globalgap_certified', ev.effective_date, src.globalgap_certified);
                v_hash  := md5(concat_ws('|', v_cult, v_irr, v_water, v_mgr, v_gap));
                CONTINUE WHEN v_hash = cur.scd_row_hash;
                UPDATE dw.dim_farm SET scd_effective_to = ev.effective_date - 1, scd_is_current = false,
                                       dw_batch_id = p_batch, dw_updated_at = now()
                WHERE farm_key = cur.farm_key;
                INSERT INTO dw.dim_farm (farm_code, farm_name, region_key, town, latitude, longitude, elevation_m, total_ha, established_year,
                                         harvest_log_template, cultivated_ha, irrigation_type, is_irrigated, water_source, farm_manager_code,
                                         is_globalgap_certified, scd_effective_from, scd_is_current, scd_version, scd_row_hash, dw_batch_id)
                VALUES (cur.farm_code, cur.farm_name, cur.region_key, cur.town, cur.latitude, cur.longitude, cur.elevation_m, cur.total_ha,
                        cur.established_year, cur.harvest_log_template, v_cult::numeric, v_irr, v_irr <> 'Dryland', v_water, v_mgr,
                        v_gap = 'Y', ev.effective_date, true, cur.scd_version + 1, v_hash, p_batch);
                n_versions := n_versions + 1;
            END LOOP;

            -- 3) Type 1 attributes overwrite every version (corrections, e.g. farm-name spelling)
            UPDATE dw.dim_farm d
            SET farm_name = f.farm_name, region_key = coalesce(r.region_key, -1), town = f.town, latitude = f.latitude,
                longitude = f.longitude, elevation_m = f.elevation_m, total_ha = f.total_ha, established_year = f.established_year,
                harvest_log_template = f.harvest_log_template, dw_updated_at = now()
            FROM stg.md_farm_clean f LEFT JOIN dw.dim_region r ON r.region_code = f.region_code
            WHERE d.farm_code = f.farm_code
              AND (d.farm_name, d.region_key, d.town, d.total_ha) IS DISTINCT FROM (f.farm_name, coalesce(r.region_key, -1), f.town,
f.total_ha);
            GET DIAGNOSTICS n_t1 = ROW_COUNT;

            PERFORM etl.fn_log_step(p_batch, 'load dim_farm (SCD2)', 'dw.dim_farm', (SELECT count(*) FROM stg.md_farm_clean),
                                    (SELECT count(*) FROM dw.dim_farm), 0, t0,
                                    format('%s initial rows, %s new versions, %s Type 1 updates', n_initial, n_versions, n_t1));
        END $$;
```

SCD Type 2 evidence (query `ev_scd2_farm` in `02_sql/12_evidence_queries.sql`). The day-2 delta of 5 Oct 2026 added version 2 of GB-ERO-01, so the incremental path is proven as well.

farm_key	farm_code	farm_name	irrigation_type	cultivated_ha	manager	globalgap	scd_effective_from	scd_effective_to	scd_is_current	scd_version
14	GB-ERO-01	Omaruru River Farm	Drip	40	MGR-113	false	1900-01-01	2026-09-30	false	1
21	GB-ERO-01	Omaruru River Farm	Drip + shade-net	40	MGR-113	false	2026-10-01	9999-12-31	true	2
11	GB-HAR-01	Hardap Dam Irrigation Farm	Flood (canal)	260	MGR-110	false	1900-01-01	2023-09-30	false	1
17	GB-HAR-01	Hardap Dam Irrigation Farm	Flood (canal)	260	MGR-110	true	2023-10-01	9999-12-31	true	2
3	GB-KAE-02	Ndonga Linena Block	Drip	180	MGR-103	false	1900-01-01	9999-12-31	true	1
4	GB-KAW-01	Nkurenkuru Valley Farm	Dryland	600	MGR-105	false	1900-01-01	2023-07-31	false	1
16	GB-KAW-01	Nkurenkuru Valley Farm	Centre pivot	450	MGR-105	false	2023-08-01	9999-12-31	true	2
9	GB-OTJ-01	Grootfontein Plains Farm	Dryland	520	MGR-104	false	1900-01-01	2022-02-28	false	1
15	GB-OTJ-01	Grootfontein Plains Farm	Dryland	520	MGR-131	false	2022-03-01	9999-12-31	true	2

customer_key	customer_code	customer_name	channel	credit_terms_days	is_key_account	is_inferred	scd_effective_from	scd_effective_to	scd_version
273	C00045	Welwitschia Food Store Okahand	Retail	30	false	false	1900-01-01	9999-12-31	1
95	C00096	Sossus Superette Windhoek	Retail	30	false	false	1900-01-01	2026-09-30	1
279	C00096	Sossus Superette Windhoek	Wholesale	30	false	false	2026-10-01	9999-12-31	2
103	C00105	Rundu Family Grocer	Retail	30	false	false	1900-01-01	2023-03-18	1

customer_key	customer_code	customer_name	channel	credit_terms_days	is_key_account	is_inferred	scd_effective_from	scd_effective_to	scd_version
243	C00105	Rundu Family Grocer	Retail	30	true	false	2023-03-19	2024-01-09	2
255	C00105	Rundu Family Grocer	Wholesale	30	true	false	2024-01-10	9999-12-31	3

The business effect is visible. For Nkurenkuru Valley Farm (GB-KAW-01), yield per planted hectare grouped by the farm version valid at planting:

irrigation_at_planting	harvest_season	planted_ha	harvest_t	yield_t_per_planted_ha
Dryland	2020/21	247	550	2.23
Dryland	2021/22	213	119	0.56
Dryland	2022/23	223	284	1.28
Centre pivot	2023/24	123	1,062.00	8.64
Centre pivot	2024/25	146	1,206.00	8.28
Centre pivot	2025/26	141	1,046.00	7.44

With a Type 1 dimension, all six seasons would read “Centre pivot”. The drought-hit dryland seasons would then wrongly drag down the pivot’s apparent performance.

Data dictionary

The complete data dictionary covers every warehouse table and column: data type, nullability, key role, business definition and source or lineage. It is **generated from the database catalogue**: each column carries a `COMMENT ON` in `03_ddl_warehouse.sql`, and `03_analytics/build_data_dictionary.py` exports them, so the dictionary cannot drift from the schema. It is reproduced in full in **Appendix A** and delivered as `06_report/data_dictionary.xlsx`. The table below summarises it.

Table 11. Warehouse tables at a glance (row counts after the 30 Sep 2026 load and 5 Oct 2026 delta)

table_name	row_count	total_size
dw.fact_sales	302,903	89 MB
dw.fact_weather_daily	30,674	4064 kB
dw.fact_production_cost	15,382	3256 kB
dw.fact_harvest	7368	1656 kB
dw.fact_inventory_monthly	6521	1072 kB
dw.fact_crop_cycle	1327	496 kB
dw.fact_market_price_monthly	486	128 kB
dw.fact_crop_estimate	324	128 kB
dw.fact_forecast_monthly	216	112 kB
dw.dim_date	2923	736 kB
dw.dim_customer	282	400 kB
dw.customer_segment	240	72 kB
dw.harvest_anomaly	25	24 kB
dw.dim_farm	22	72 kB
dw.dim_product	22	40 kB
dw.dim_region	16	40 kB
dw.dim_cost_category	15	40 kB
dw.yield_driver_importance	13	24 kB
dw.dim_crop	12	40 kB
dw.dim_commodity	6	40 kB
dw.dim_crop_category	5	40 kB

Comparison with alternative designs

Table 12. Chosen design vs fully snowflaked and flat (denormalised) alternatives

Criterion	Fully snowflaked (normalised dimensions)	Chosen: star core + 2 targeted snowflakes, 9-fact constellation	Flat / denormalised “one big table” per process
Shape	Every hierarchy level is a table: product › crop › category; farm › region › zone; customer › town › region › country; date › month › quarter › year	Star around each fact. Snowflake only where a hierarchy level is itself a conformed grain (crop, region)	One wide table, e.g. sales plus all customer, product, farm and date attributes (about 60 columns)
Query complexity (SQL / DAX)	High: 5–8 joins per query; DAX filters travel long chains	Low: 1–3 joins; short filter paths	Lowest for a single process
Power BI performance	More relationships; long chains slow filter propagation; harder to optimise	Recommended shape for VertiPaq (Microsoft, 2024b): narrow facts, small dimensions	Wide fact with repeated text columns. Compression helps, but every attribute is stored per row and time intelligence needs a separate date table anyway

Criterion	Fully snowflaked (normalised dimensions)	**Chosen: star core + 2 targeted snowflakes, 9-fact constellation**	Flat / denormalised “one big table” per process
Storage	Smallest	Small (fact_sales 14 columns in the mart)	Largest: our model test shows 24 vs 14 fact columns, 16.0 vs 10.8 MB for the more denormalised import (§3.7)
Maintainability and SCD	Good for Type 1; Type 2 at several levels is hard	SCD Type 2 handled in two dimensions	Very poor: a customer channel change means rewriting history rows or losing as-was
Conformance and drill-across	Good	Good: conformed Date, Farm, Region and Crop are shared by 9 facts	None: each process has its own copies of attributes, so harvest vs sales vs NSA drill-across is impossible without re-joining
Fit for this case	Over-engineered for small dimensions (12 crops, 14 regions)	**Best balance**	Acceptable only for a one-off extract or a single dataset

Conclusion. The flat design fails the brief’s core need, which is to integrate production, cost, sales and external benchmarks through shared dimensions. Full normalisation adds joins without a benefit at this dimension size. The chosen design normalises exactly where a level is shared by facts at a coarser grain. We also considered an Inmon-style 3NF enterprise warehouse with dependent marts and Data Vault 2.0. Both were rejected as disproportionate for a single division with nine sources: their auditability benefit is delivered more cheaply by the landing layer, which stores source file and row lineage, and by the audit schema.

3.3 ETL and data integration

Platform choice: PostgreSQL 16

Table 13. PostgreSQL 16 vs SQL Server Express for this warehouse

Criterion	PostgreSQL 16 (chosen)	SQL Server 2022 Express
Licence and cost	Open source (PostgreSQL Licence), no edition limits	Free, but capped
Hard limits	None relevant (single-table and database limits are in terabytes)	**10 GB per database**, about 1.4 GB buffer-pool memory, lesser of 1 socket or 4 cores (Microsoft, 2024c)
Scheduling	`pg_cron` or the OS scheduler running `run_all.py`	**No SQL Server Agent** in Express; needs Windows Task Scheduler
Semi-structured sources	Native JSON/JSONB: NASA POWER JSON is expanded in SQL (`jsonb_each_text`)	`OPENJSON` (adequate)
Analytical features used	BRIN indexes, covering `INCLUDE` indexes, materialized views, `GROUPING SETS`/`CUBE`/`ROLLUP`, `percentile_cont`, procedures with `CALL`	Mostly equivalent: columnstore and indexed views, `GROUPING SETS`
Power BI connectivity	Built-in PostgreSQL connector (Npgsql), supported by the on-premises data gateway; query folding verified for the incremental-refresh filter	Native connector; best tooling integration
Team skills and portability	Runs identically on Windows, Linux and cloud (Azure Database for PostgreSQL)	Windows/Linux, Microsoft stack

Decision. PostgreSQL removes Express's 10 GB ceiling, which five years of invoice lines plus daily weather for more farms would reach as the division grows. It also parses the external JSON natively and has no edition-based feature gaps. The trade-off is less integrated Microsoft tooling (no SSMS or SSIS). That is mitigated by using plain SQL scripts and Python, which are version-controlled and run from one command.

Sources and conformed keys

Table 14. The nine sources integrated (S1-S5 internal synthetic; S6-S9 public layouts)

ID	Source	Format (as landed)	Volume	Refresh	Owner	Conformed keys produced
S1	Farm harvest logs (14 files, layouts A/B/C) + planting register	CSV	7,476 + 1,327 rows	Daily in season	Farm managers	farm_code, crop_code, block_id, cycle_id, date
S2	Master data workbook + day-2 delta	XLSX, 10 sheets	238 rows + change logs	On change	Master data steward	farm, customer, product, crop, region, cost category
S3	ERP invoice and credit-note lines (6 overlapping annual files)	gzip CSV	305,490 rows	Daily	Sales administration	invoice_no + line_no, customer_code, product_code, farm_code
S4	Stock system month-end movements	CSV	6,519 rows	Monthly	Inventory controller	farm_code, product_code, month
S5	Finance cost ledger	gzip CSV	15,428 rows	Daily	Finance	posting_id, farm_code, crop_code, cost category, cost_centre = cycle_id
S6	NASA POWER daily point data (Agroclimatology)	JSON, 1 per farm	14 × 2,191 days	Daily	External data steward	farm location (lat/long from S2), date
S7	World Bank Commodity Price Data (Pink Sheet), monthly	XLSX (wide)	6 series × 81 months	Monthly	External data steward	commodity → crop, month
S8	FRED exchange rates DEXSFUS, DEXUSEU, DEXUSUK	CSV	5,283 daily rows	Daily	External data steward	currency, month (monthly mean)
S9	NSA / MAFWLR crop estimates by region	CSV (transcribed)	348 rows	Per season	External data steward	region, crop, season, farming sector

Integration depends on six conformance mechanisms, all held in `ref.*` and steward-maintained:

1. **Farm codes:** 217 non-standard variants in the logs (`' gb-ero-01'`, `GB_OHA_01`) are standardised by rule HV08. ERP lines without a farm go to member −1 (SA05).
2. **Crops:** 47 spellings across logs, ledger and NSA map to 10 codes through `ref.crop_alias`. Pink Sheet series map to crops through `dim_commodity.crop_key`.
3. **Regions:** NSA and master-data spellings ("Zambezi (Caprivi)", "Kavango") map to ISO 3166-2 codes through `ref.region_alias` (MD01).
4. **Customers:** 3 duplicate accounts are merged to a survivor through `ref.customer_xref` (MD04/SA03). 5 customers that appear in invoices before master data become **inferred members** (SA04).
5. **Units and currency:**

- kg, t, ton, 10 kg crates and 400 kg bins convert to kg via `ref.uom_conversion`.
- USD, EUR and GBP convert to N$ via the FRED monthly means, using the N$/ZAR parity (`ref.fx_rate_monthly`).

6. **Dates:** three layouts (DD/MM/YYYY, ISO, DD-Mon-YY) are parsed by `etl.fn_try_date`. It accepts a value only if formatting the parsed date reproduces the input exactly, so 13/02 cannot silently become 2 December.

Pipeline: source → staging → warehouse

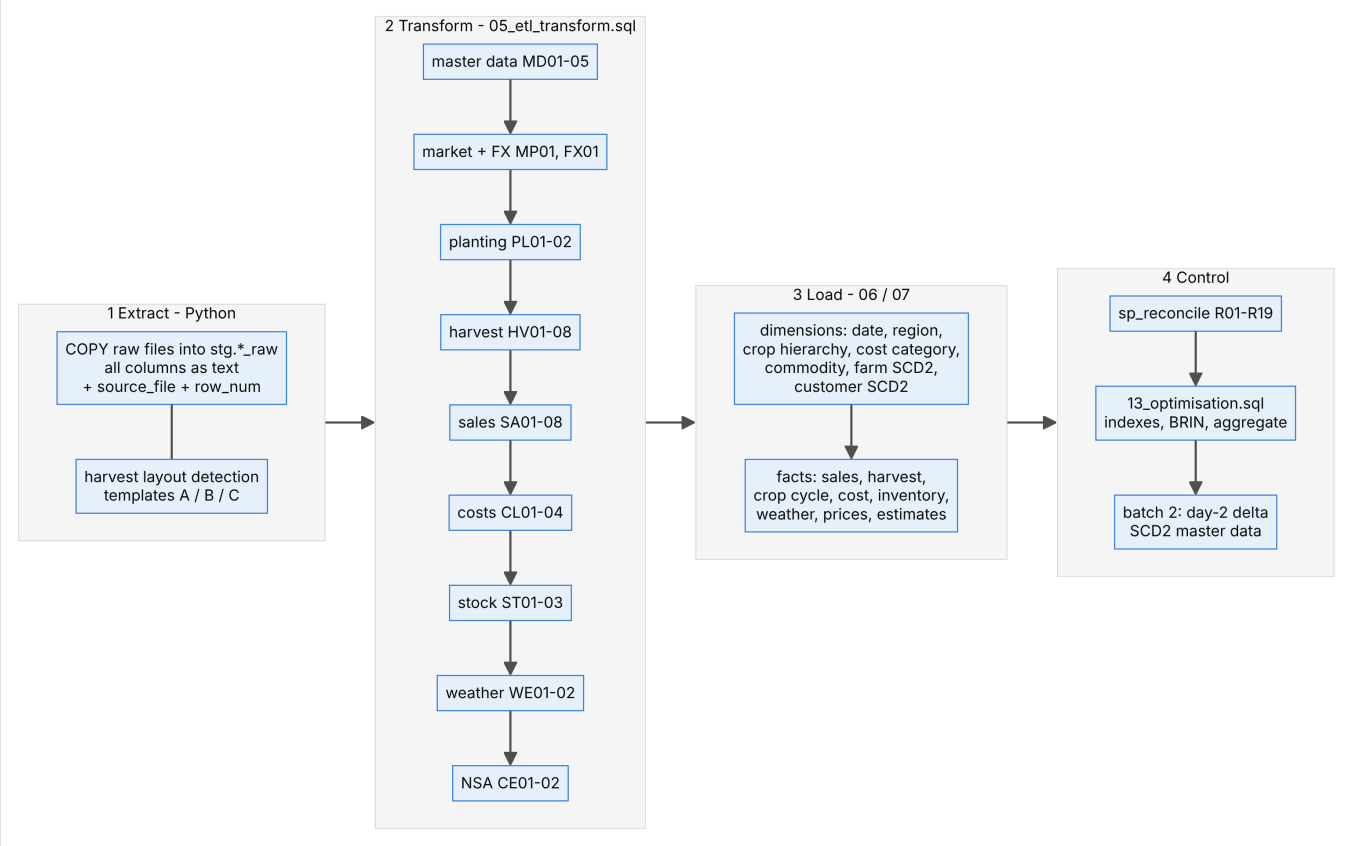


Figure 6. Data lineage and run order (run_pipeline.py); every arrow is logged in audit.etl_step_log (Mermaid source: `06_report/diagrams/lineage.mmd`)

The scripts run in the order shown. One command builds a fresh warehouse from the raw files: `python 02_sql/run_pipeline.py --create-db --with-delta` (or `python run_all.py` for everything).

Table 15. SQL and Python deliverables of the pipeline (all in the submission package)

File	Purpose
`02_sql/00_create_database.sql`	Database and least-privilege login roles (`gb_etl_loader`, `gb_pbi_reader`)
`02_sql/01_ddl_schemas_audit.sql`	Schemas, audit tables (`etl_batch`, `etl_step_log`, `dq_rule`, `dq_issue_log`, `recon_result`), DQ rule catalogue
`02_sql/02_ddl_staging_ref.sql`	Landing (`*_raw`) and cleansed (`*_clean`) staging tables; reference and mapping tables with steward seed data
`02_sql/03_ddl_warehouse.sql`	**Warehouse DDL**: 9 dimensions, 9 facts, analytics-output tables, `COMMENT ON` for every column (Appendix C)
`02_sql/04_etl_functions.sql`	Parsing, normalisation, holiday, SCD-history and batch-logging functions
`02_sql/05_etl_transform.sql`	Nine transform procedures implementing the 37 DQ rules
`02_sql/06_etl_load_dimensions.sql`	Dimension loads including SCD Type 2
`02_sql/07_etl_load_facts.sql`	Fact loads (surrogate-key lookups, upserts), `sp_run_full_load`, `sp_run_master_delta`
`02_sql/08_reconciliation.sql`	`sp_reconcile`: 19 checks
`02_sql/09_mart_views.sql`	BI-facing views, `mart.dq_summary`, `mart.recon_latest`
`02_sql/10_security_grants.sql`	Grants
`02_sql/11_olap_queries.sql`, `12_evidence_queries.sql`, `13_optimisation.sql`	OLAP demonstrations (§3.5), evidence queries, performance tuning (§3.7)
`02_sql/run_pipeline.py`	Orchestration: build, extract, `CALL`s, reconcile, optimise, delta

Design principles.

1. **Lossless landing.** Every raw value is stored as text with its source file and row number, so any warehouse figure can be traced back to a source row.

2. **Never silently drop.** Each rule logs the raw value and the action taken (`audit.dq_issue_log`). Rejected rows stay in staging with `dq_status = 'REJECTED'` as a quarantine for the steward to resubmit.
3. **Idempotent and re-runnable.**
   - Facts are upserted with `ON CONFLICT ... DO UPDATE` on their natural keys.
   - SCD Type 2 uses hash comparison.
   - Re-running a batch changes nothing.
4. **Incremental.** Batch 2 (`sp_run_master_delta`) applies the 5 Oct 2026 master-data delta. It creates new SCD versions only for changed rows.
5. **Auditable.** Every step writes rows in, rows out, rows rejected and duration to `audit.etl_step_log` under a batch id stamped on every warehouse row (`dw_batch_id`).

The fact loader shows the **as-was surrogate-key lookup** (an SCD date-range join), inferred members and the idempotent upsert:

File: `02_sql/07_etl_load_facts.sql` (lines 9–54)

```
CREATE OR REPLACE PROCEDURE etl.sp_load_fact_sales(p_batch integer)
LANGUAGE plpgsql AS $$
DECLARE t0 timestamptz := clock_timestamp(); n_inferred int; n bigint;
BEGIN
    -- SA04: late-arriving customers become inferred members (completed later by sp_load_dim_customer_scd2)
    INSERT INTO dw.dim_customer (customer_code, customer_name, customer_type, region_key, channel, is_inferred,
                                 scd_effective_from, scd_is_current, scd_version, scd_row_hash, dw_batch_id)
    SELECT DISTINCT s.customer_code, 'Inferred customer ' || s.customer_code || ' (awaiting master data)', 'Unknown', -1,
           'Unassigned', true, DATE '1900-01-01', true, 1, md5('inferred'), p_batch
    FROM stg.sales_clean s
    WHERE s.dq_status IN ('OK', 'CORRECTED')
      AND NOT EXISTS (SELECT 1 FROM dw.dim_customer d WHERE d.customer_code = s.customer_code);
    GET DIAGNOSTICS n_inferred = ROW_COUNT;

    INSERT INTO dw.fact_sales (invoice_date_key, dispatch_date_key, product_key, customer_key, farm_key, invoice_no, line_no,
                               doc_type, ref_invoice_no, sales_rep_code, currency_code, quantity_sales_uom, quantity_kg,
                               unit_price_txn, fx_rate_nad, gross_revenue_nad, discount_nad, net_revenue_nad, cogs_nad,
                               freight_cost_nad, is_price_anomaly, dw_batch_id)
    SELECT to_char(s.invoice_date, 'YYYYMMDD')::int,
           to_char(coalesce(s.dispatch_date, s.invoice_date), 'YYYYMMDD')::int,
           p.product_key, coalesce(c.customer_key, -1), coalesce(f.farm_key, -1),
           s.invoice_no, s.line_no, s.doc_type, s.ref_invoice_no, s.sales_rep_code, s.currency_code,
           s.quantity, s.quantity_kg, s.unit_price, s.fx_rate_nad, s.gross_revenue_nad, s.discount_nad, s.net_revenue_nad,
           s.cogs_nad, s.freight_nad, s.is_price_anomaly, p_batch
    FROM stg.sales_clean s
    JOIN dw.dim_product p ON p.product_code = s.product_code
    LEFT JOIN dw.dim_customer c ON c.customer_code = s.customer_code
                               AND s.invoice_date BETWEEN c.scd_effective_from AND c.scd_effective_to
    LEFT JOIN dw.dim_farm f ON f.farm_code = s.farm_code
                           AND s.invoice_date BETWEEN f.scd_effective_from AND f.scd_effective_to
    WHERE s.dq_status IN ('OK', 'CORRECTED')
    ON CONFLICT (invoice_no, line_no) DO UPDATE
    SET invoice_date_key = EXCLUDED.invoice_date_key, dispatch_date_key = EXCLUDED.dispatch_date_key,
        product_key = EXCLUDED.product_key, customer_key = EXCLUDED.customer_key, farm_key = EXCLUDED.farm_key,
        quantity_sales_uom = EXCLUDED.quantity_sales_uom, quantity_kg = EXCLUDED.quantity_kg,
        unit_price_txn = EXCLUDED.unit_price_txn, fx_rate_nad = EXCLUDED.fx_rate_nad,
        gross_revenue_nad = EXCLUDED.gross_revenue_nad, discount_nad = EXCLUDED.discount_nad,
        net_revenue_nad = EXCLUDED.net_revenue_nad, cogs_nad = EXCLUDED.cogs_nad, freight_cost_nad = EXCLUDED.freight_cost_nad,
        is_price_anomaly = EXCLUDED.is_price_anomaly, dw_batch_id = EXCLUDED.dw_batch_id
    WHERE (dw.fact_sales.customer_key, dw.fact_sales.farm_key, dw.fact_sales.net_revenue_nad, dw.fact_sales.quantity_kg)
          IS DISTINCT FROM (EXCLUDED.customer_key, EXCLUDED.farm_key, EXCLUDED.net_revenue_nad, EXCLUDED.quantity_kg);
    GET DIAGNOSTICS n = ROW_COUNT;
    PERFORM etl.fn_log_step(p_batch, 'load fact_sales', 'dw.fact_sales',
                            (SELECT count(*) FROM stg.sales_clean WHERE dq_status IN ('OK', 'CORRECTED')), n, 0, t0,
                            n_inferred || ' inferred customer members created');
END $$;
```

Data profiling

Before any rule was written, every source was profiled with `03_analytics/data_profiling.py` for:

- row and column counts;
- duplicates;
- missing keys;
- unparseable values;
- spelling variants;
- outliers, flagged when beyond Q3 + 3 × IQR.

Table 16. Profiling summary by source (03_analytics/data_profiling.py)

Source	Rows	Cols	Duplicates	Missing key values	Invalid dates	Other issues found
S1 Farm harvest logs (14 CSV, 3 layouts)	7476	8	89	0	11	217 non-standard farm codes; 47 crop spellings for 10 crops; 2 non-positive quantities; 3 date formats; units kg/t/crates/bins

Source	Rows	Cols	Duplicates	Missing key values	Invalid dates	Other issues found
S1 Planting register (block level)	1327	11	0	13	0	29 dates in DD/MM/YYYY among ISO dates; harvested_ha blank for 79 open/growing blocks (expected)
S3 ERP sales invoice lines (6 gz CSV)	305490	17	2587	0	0	4591 product-code variants; 2488 blank farm codes; 39 blank currencies; 2095 lines on duplicate customer codes; 696 lines for customers missing from master data
S4 Stock system (monthly)	6519	10	0	0	0	3 negative closing balances; gaps in monthly series detected by continuity check
S5 Cost ledger (gz CSV)	15428	12	46	0	0	47 category spellings for 14 categories; 454 amounts with N$ / thousands separators; 50 crop-specific postings without crop
S2 Master data workbook (10 sheets)	238	12	3	2	0	18 non-standard region spellings; 8 ALL-CAPS and 4 padded names; 3 probable duplicate customers (name match); 1 product without net weight
S6 NASA POWER weather (14 JSON)	153370	5	0	697	0	-999 fill values (incl. last 3 days = data latency)
S7 World Bank Pink Sheet (xlsx)	486	6	0	2	0	wide sheet with 3 title rows; '...' missing markers; header text with trailing blanks ('Urea')
S8 FRED FX (3 CSV)	5283	2	0	72	0	blank values on US holidays; daily → monthly
S9 NSA / MAFWLR crop estimates (CSV)	348	9	0	0	0	104 rows with non-standard region names; 24 rows for a crop outside scope; numbers with thousands separators

Table 17. Harvest-log layouts, date formats and units of measure found by profiling

template_code	date_fmt	rows
A	DD/MM/YYYY	4069
B	YYYY-MM-DD	1208
C	DD-Mon-YY	2199

unit text (template C)	rows
crates(10kg)	968
t	444
ton	421
KG	113
kg	95
Kg	85

Table 18. Numeric distributions and outliers (beyond Q3 + 3 IQR)

measure	n	minimum	p25	median	p75	maximum	beyond_3iqr
Harvest qty, template A (kg)	4066	-4,822.40	4,623.60	10,918.80	28,366.10	340,017.20	273
Harvest qty, template B (t)	1207	0.03	7.43	22.86	78.80	502.90	3
Sales quantity (sales UoM)	305490	-4,595.00	10.00	24.00	54.00	18,378.00	19477
Sales unit price (txn ccy)	305490	4.17	43.97	89.23	117.32	56,577.30	21147

measure	n	minimum	p25	median	p75	maximum	beyond_3iqr
Cost amount (N$, parsed)	15428	-6,794,519.28	13,960.83	28,096.28	64,644.00	8,152,036.39	867

Unit-price outliers (> Q3 + 3 IQR of log price within product): 160

Reading the outliers. Not every statistical outlier is an error. Bulk maize sales and export grape containers are legitimately 100× larger than informal-market sales. A global screen on unit price (log-price beyond Q3 + 3 IQR within product) gives 160 candidates, many of them legitimate premium or clearance sales.

The production rule SA07 is therefore narrower. It compares each line's realised N$/kg with the **median of the same product in the same month** and flags lines above 4× or below 0.25× of it. It flags 119 lines, and all 119 are injected ×10 price errors (precision and recall 1.0, below).

Similarly, the harvest rule HV06 compares a record with its *own block* and sister blocks, not with all harvests.

Data-quality rules and how each problem was handled

The rule catalogue (`audit.dq_rule`, seeded in `01_ddl_schemas_audit.sql`) holds 37 rules. Each rule has:

- a DQ dimension (validity, completeness, consistency, uniqueness, accuracy, integrity, timeliness);
- an action: **REJECT** (quarantine), **CORRECT**, **MAP**, **MERGE**, **IMPUTE**, **INFER** or **FLAG**;
- an accountable steward role.

The results of the full load are below, with an example per rule taken from the issue log.

Table 19. DQ rule results for the full load (batch 1)

rule_code	rule_name	dq_dimension	action	rows_affected	owner_role
CE01	Crop outside Green Basket scope	Integrity	REJECT	24	External Data Steward
CE02	Yield not published	Completeness	IMPUTE	0	External Data Steward
CL01	Duplicate ledger posting	Uniqueness	REJECT	46	Finance Data Steward
CL02	Cost category alias	Consistency	MAP	4656	Finance Data Steward
CL03	Crop missing on crop-specific cost	Completeness	FLAG	50	Finance Data Steward
CL04	Amount text formatting	Validity	CORRECT	452	Finance Data Steward
FX01	FX observation blank	Completeness	FLAG	72	External Data Steward
HV01	Harvest date unparseable	Validity	REJECT	11	Farm Data Steward
HV02	Harvest quantity missing	Completeness	REJECT	6	Farm Data Steward
HV03	Harvest quantity not positive	Validity	REJECT	2	Farm Data Steward
HV04	Unknown farm or crop code	Integrity	REJECT	0	Farm Data Steward
HV05	Duplicate harvest record	Uniqueness	REJECT	89	Farm Data Steward
HV06	Harvest quantity unit/decimal error	Accuracy	CORRECT	7	Farm Data Steward
HV07	Harvest record without planting block	Integrity	FLAG	0	Farm Data Steward
HV08	Farm code formatting	Consistency	CORRECT	217	Farm Data Steward
MD01	Region name variant	Consistency	MAP	127	Master Data Steward
MD02	Customer region missing	Completeness	FLAG	2	Master Data Steward
MD03	Product net weight missing	Completeness	IMPUTE	1	Master Data Steward
MD04	Duplicate customer account	Uniqueness	MERGE	3	Master Data Steward
MD05	Customer name formatting	Consistency	CORRECT	9	Master Data Steward
MP01	Commodity price missing	Completeness	IMPUTE	2	External Data Steward
PL01	Planting date alternative format	Consistency	CORRECT	29	Agronomist
PL02	Planted area missing	Completeness	IMPUTE	13	Agronomist
PL03	Status inconsistent with harvest log	Consistency	FLAG	1	Agronomist
SA01	Duplicate invoice line	Uniqueness	REJECT	2587	Sales Data Steward
SA02	Product code variant	Consistency	CORRECT	4556	Sales Data Steward
SA03	Customer code merged	Uniqueness	MERGE	2088	Sales Data Steward
SA04	Unknown customer (late-arriving)	Timeliness	INFER	694	Sales Data Steward
SA05	Farm of origin missing	Completeness	FLAG	2466	Sales Data Steward
SA06	Currency missing	Completeness	CORRECT	39	Sales Data Steward
SA07	Unit price anomaly	Accuracy	FLAG	119	Sales Data Steward

rule_code	rule_name	dq_dimension	action	rows_affected	owner_role
SA08	Quantity sign inconsistent with document type	Validity	REJECT	0	Sales Data Steward
ST01	Closing stock sign error	Accuracy	CORRECT	3	Inventory Controller
ST02	Stock-flow identity broken	Consistency	FLAG	0	Inventory Controller
ST03	Missing monthly stock snapshot	Completeness	IMPUTE	2	Inventory Controller
WE01	Weather fill value (temperature)	Completeness	IMPUTE	228	External Data Steward
WE02	Weather fill value (rainfall)	Completeness	IMPUTE	136	External Data Steward

Table 20. One logged example per rule (raw value and action)

rule_code	source_object	source_key	column_name	raw_value	action_taken
CE01	NSA crop estimates	2020/2021/Ohangwena/Sorghum	crop	Sorghum	Excluded (out of scope)
CL01	Cost ledger	JNL-2021-0000436	posting_id	JNL-2021-0000436	Removed duplicate
CL02	Cost ledger	JNL-2021-0001791	cost_category	FERTILISER	Mapped to CC02
CL03	Cost ledger	JNL-2021-0008100	crop_code		Loaded to Unknown crop (-1)
CL04	Cost ledger	JNL-2021-0010824	amount_nad	N$25,149.46	Parsed to 25149.46
FX01	FRED DEXSFUS	2020-02-05	value		Excluded from monthly average
HV01	Harvest log	harvest_log_GB-ERO-01.csv:643	date	2025-13-02	Rejected to quarantine
HV02	Harvest log	harvest_log_GB-KHA-02.csv:150	quantity		Rejected to quarantine
HV03	Harvest log	harvest_log_GB-HAR-01.csv:34	quantity	-20.819	Rejected to quarantine
HV05	Harvest log	harvest_log_GB-ERO-01.csv:238	quantity	7,029	Removed duplicate
HV06	Harvest log	harvest_log_GB-ERO-01.csv:193	quantity	502896.0	Divided by 10 (block yield 3.8x)
HV08	Harvest log	harvest_log_GB-ERO-01.csv:43	farm_code	gb-ero-01	Standardised to GB-ERO-01
MD01	Master data: Farms	GB-ZAM-01	region	Zambezi (Caprivi)	Mapped to NA-CA
MD02	Master data: Customers	C00175	region		Loaded with Unknown region (-1)
MD03	Master data: Products	BUT-C2-BIN	net_weight_kg		Imputed 400 kg from pack type “400kg bin”
MD04	Master data: Customers	C00241	customer_code	C00241	Merged into survivor C00063
MD05	Master data: Customers	C00018	customer_name	MOPANE WHOLESALE INVESTMENTS CC	Cleaned to: Mopane Wholesale Investments CC
MP01	World Bank Pink Sheet	SORGHUM 2021-02	price	missing	Linearly interpolated
PL01	Planting register	GB-KAE-01/MZW/2023S/P01	planting_date	22/11/2023	Parsed as DD/MM/YYYY
PL02	Planting register	GB-ZAM-01/WHT/2023W/P01	planted_ha		Imputed with mean block area of the cycle
PL03	Planting register	GB-OTJ-02/MZW/2023S/P01	status	Completed	Kept (no harvest records yet)
SA01	ERP sales	INV2022-000478/3 (sales_invoice_lines_FY2022.csv.gz:708)	invoice_no+line_no		Removed duplicate
SA02	ERP sales	CRN2022-000010/1 (sales_invoice_lines_FY2022.csv.gz:2469)	product_code	CAB C1 HD	Mapped to CAB-C1-HD
SA03	ERP sales	CRN2023-000071/1 (sales_invoice_lines_FY2023.csv.gz:11643)	customer_code	C00242	Merged to survivor C00049
SA04	ERP sales	CRN2026-000137/1 (sales_invoice_lines_FY2026.csv.gz:34761)	customer_code	C00150	Inferred member created
SA05	ERP sales	CRN2022-000047/1 (sales_invoice_lines_FY2022.csv.gz:9233)	farm_code		Loaded to Unknown farm (-1)
SA06	ERP sales	CRN2024-000226/1 (sales_invoice_lines_FY2024.csv.gz:52061)	currency_code		Defaulted to EUR
SA07	ERP sales	INV2022-001385/2 (sales_invoice_lines_FY2022.csv.gz:2091)	unit_price	64.3000 NAD	Flagged for review (price 25.72 N$/kg)
ST01	Stock system	GB-OTJ-01/MZW-WM2-BLK/2021-05-31	closing_kg	-56554.2	Sign corrected
ST03	Stock system	GB-OSK-01/MZW-WM1-BLK/2022-12-31	row	missing	Imputed from adjacent snapshots and ERP dispatches
WE01	NASA POWER	GB-ERO-01/2021-01-12	T2M_MAX/T2M_MIN	-999	Interpolated
WE02	NASA POWER	GB-ERO-01/2021-01-07	PRECTOTCORR	-999	Left NULL, flagged

Table 21. How each class of problem was handled

Problem class	Examples	Treatment	Rationale
Missing values	Blank harvest quantity (HV02); blank planted area (PL02); missing product weight (MD03); weather −999 (WE01/02); Pink Sheet "…" (MP01)	Reject if a measure is unusable. Impute when a defensible estimate exists: the cycle's mean block area, the pack type's weight, linear interpolation for temperature and prices. Leave rainfall NULL rather than invent rain.	Imputation only where the error is bounded. Rainfall drives yield analysis, so fabricated rain would bias §3.5.
Duplicates	Overlapping ERP extracts (SA01, 2,587 lines); re-keyed harvest rows (HV05, 89); duplicate postings (CL01, 46); duplicate customers (MD04, 3)	Deduplicate on the natural key, keeping the first occurrence. Merge customers to a survivor with a cross-reference so history moves with them.	Prevents double-counted revenue (R04/R05 prove totals match).
Outliers / keying errors	×10 harvest quantities (HV06, 7); ×10 unit prices (SA07, 119); negative stock (ST01, 3)	**Correct** only when a mechanism explains the value (÷10 restores it to its reference; a sign flip restores the stock identity); otherwise **flag** for review	Precision first: genuine extremes such as grape containers are never "fixed".
Inconsistent formats	3 date layouts; 47 crop spellings; 4,556 product-code variants; amount text "N$25,149.46"; region aliases	Strict parsing (`fn_try_date`, `fn_try_numeric`), alias and xref tables, normalisation (`fn_norm`)	Repeatable and steward-maintainable, instead of one-off Excel fixes.
Referential gaps	Lines without farm (SA05); customers missing from master data (SA04); harvest rows without a planting block (HV07)	Unknown member −1, or inferred member completed later	Facts are never lost and gaps stay measurable (R11, R12).

Validating the rules against ground truth

Because the internal data is synthetic, the generator records **every defect it injects**. That allows something rarely possible with real data: scoring the DQ rules themselves (`03_analytics/validate_dq_rules.py`).

Table 22. DQ rule validation against the generator's injected defects

27 injected defect types checked against the rules that should catch them: 23 match exactly. Record-level (7 rules with a truth list): 236 of 237 injected records found, 0 false positives.

rule	defect	level	injected	flagged	precision	recall	match
HV05	Exact duplicate harvest records	record	89	89	1.000	1.000	exact
HV01	Unparseable harvest dates	record	11	11	1.000	1.000	exact
HV02	Blank harvest quantities	record	6	6	1.000	1.000	exact
HV03	Negative harvest quantities	record	2	2	1.000	1.000	exact
HV06	x10 / x1000 quantity keying errors	record	8	7	1.000	0.875	TP 7 / FP 0 / FN 1
SA07	Unit price keyed x10 (invoice lines)	record	119	119	1.000	1.000	exact
ST03	Missing monthly stock snapshots	record	2	2	1.000	1.000	exact
HV08	Farm code case / whitespace / underscore variants	count	215	217			see note
PL01	Planting dates keyed as DD/MM/YYYY	count	29	29			exact
PL02	Planted area left blank	count	13	13			exact
MD03	Product without net weight	count	1	1			exact
MD04	Duplicate customer accounts	count	3	3			exact
MD05	Customer names in capitals / padded	count	9	9			exact
MD02	Domestic customer without region	count	2	2			exact
SA01	Duplicate invoice lines (overlapping extracts)	count	2587	2587			exact
SA02	Product-code spelling variants	count	4556	4556			exact
SA03	Lines on a duplicate customer code	count	2088	2088			exact
SA04	Lines for customers not yet in master data	count	694	694			exact

rule	defect	level	injected	flagged	precision	recall	match
SA05	Lines without farm of origin	count	2466	2466			exact
SA06	Export lines without currency	count	39	39			exact
ST01	Closing stock keyed with the wrong sign	count	3	3			exact
CL01	Duplicate ledger postings	count	46	46			exact
CL04	Amounts with N$ / thousands separators	count	451	452			see note
CL03	Crop-specific cost without crop	count	48	50			see note
MP01	Pink Sheet '...' missing markers	count	2	2			exact
FX01	Blank FX observations	count	72	72			exact
CE01	NSA rows for a crop outside scope	count	24	24			exact

The three count-level differences ("see note") were investigated record by record:

- **HV08 (+2):** the two extra fixes are the duplicate copies of rows with malformed farm codes. Both copies are standardised before deduplication.
- **CL04 (+1):** includes 2 duplicate copies of formatted amounts, so at most one formatted amount parsed correctly without being logged. The ledger control total (R09) matches to the cent.
- **CL03 (+2):** a reversal and its correction re-posting copied a posting whose crop was already missing. They are correctly flagged too.

Two genuine bugs were found by this validation and fixed before submission. We report them because they show why row counts alone do not prove data quality.

1. **HV06 masked itself.**
   - The first version compared a block's yield with the median of *all* blocks in the cycle, including itself.
   - At Hardap Dam (2025/26 maize), the cycle has only two blocks. A ×10 record (1,571 t instead of 157 t) pushed the median up so far that the ratio fell to 1.7 and escaped the 2.5 threshold. The block showed 47 t/ha of maize on the dashboard.
   - The rule now uses a **leave-one-out** median of the *sister* blocks. When only one sister exists, the planned yield must agree too.
   - Recall rose from 6/8 to 7/8, with still no false positives. The remaining miss (a 1,090 kg tomato pick keyed as ten times 109 kg) lies within the normal range of picks and cannot be distinguished by any rule.
2. **"NA" is a country.**
   - pandas' default missing-value list includes the string "NA", which is Namibia's ISO 3166 code. Reading the master-data workbook therefore turned the country of 226 domestic customers into NULL ("Unknown" on the dashboard).
   - The extract now reads Excel with `keep_default_na=False`, so landing is lossless.
   - The defect would have passed every row-count check. It was caught by a plausibility review of dimension values.

The corrected HV06 rule:

File: `02_sql/05_etl_transform.sql` (lines 289–346)

```
            -- HV06 unit / decimal-shift errors (precision first: a value is only changed when a keying mechanism explains it)
            --   method 1 "neighbour spike": > 5x the median of the 2 previous and 2 next records of the same field block;
            --   method 2 "block yield": the block's implied t/ha is > 2.5x the median of its sister blocks in the cycle
            --   (leave-one-out: the block under test is excluded, otherwise in a 2-block cycle the outlier masks itself;
            --   with a single sister the block must also exceed 2.5x its planned yield).
            -- Correction factor 10 or 1000 is applied only if it brings the value back within 3x of its reference;
            -- unexplained outliers are flagged for the Farm Data Steward and left unchanged.
            WITH seq AS (
                SELECT source_file, row_num, harvested_kg, cycle_id, block_id,
                       lag(harvested_kg, 1) OVER w AS p1, lag(harvested_kg, 2) OVER w AS p2,
                       lead(harvested_kg, 1) OVER w AS n1, lead(harvested_kg, 2) OVER w AS n2
                FROM stg.harvest_clean
                WHERE dq_status IN ('OK', 'CORRECTED') AND cycle_id IS NOT NULL
                WINDOW w AS (PARTITION BY cycle_id, block_id ORDER BY harvest_date, row_num)
            ), nb AS (
                SELECT s.source_file, s.row_num, s.harvested_kg, x.med, x.cnt
                FROM seq s CROSS JOIN LATERAL (
                    SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY v) AS med, count(v) AS cnt
                    FROM unnest(ARRAY[s.p1, s.p2, s.n1, s.n2]) AS v) x
            ), spike AS (
                SELECT source_file, row_num, harvested_kg, harvested_kg / med AS ratio, 'neighbour spike' AS method,
                       CASE WHEN abs(ln(harvested_kg / 1000 / med)) < ln(3) THEN 1000
                            WHEN abs(ln(harvested_kg / 10 / med)) < ln(3) THEN 10 END AS factor
                FROM nb WHERE cnt >= 2 AND harvested_kg > 5 * med
            ), blk AS (
                SELECT h.cycle_id, h.block_id, sum(h.harvested_kg) AS kg, max(h.harvested_kg) AS mx,
                       nullif(max(p.planted_ha), 0) AS ha, max(p.planned_yield_t_ha) * 1000 AS plan_kg_ha
                FROM stg.harvest_clean h JOIN stg.planting_clean p USING (cycle_id, block_id)
                WHERE h.dq_status IN ('OK', 'CORRECTED') GROUP BY h.cycle_id, h.block_id
            ), ref AS (
                SELECT b.*, nullif(s.med_y, 0) AS med_y, s.n
```

```
                FROM blk b CROSS JOIN LATERAL (
                    SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY o.kg / o.ha) AS med_y, count(o.ha) AS n
                    FROM blk o WHERE o.cycle_id = b.cycle_id AND o.block_id <> b.block_id) s
            ), yld AS (
                SELECT DISTINCT ON (h.source_file, h.row_num) h.source_file, h.row_num, h.harvested_kg,
                       (r.kg / r.ha) / r.med_y AS ratio, 'block yield' AS method, 10 AS factor
                FROM ref r
                JOIN stg.harvest_clean h ON h.cycle_id = r.cycle_id AND h.block_id = r.block_id AND h.harvested_kg = r.mx
                                        AND h.dq_status IN ('OK', 'CORRECTED')
                WHERE r.n >= 1 AND r.kg / r.ha > 2.5 * r.med_y
                  AND (r.n >= 2 OR r.kg / r.ha > 2.5 * r.plan_kg_ha)
                  AND ((r.kg - r.mx + r.mx / 10) / r.ha) / r.med_y BETWEEN 0.5 AND 1.6
            ), fix AS (
                SELECT DISTINCT ON (source_file, row_num) * FROM (SELECT * FROM spike UNION ALL SELECT * FROM yld) u
                ORDER BY source_file, row_num, (factor IS NULL), method DESC
            ), logged AS (
                INSERT INTO audit.dq_issue_log (batch_id, rule_code, source_object, source_key, column_name, raw_value, action_taken)
                SELECT p_batch, 'HV06', 'Harvest log', source_file || ':' || row_num, 'quantity', harvested_kg::text,
                       CASE WHEN factor IS NULL THEN 'Flagged for review: ' || method || ' ' || round(ratio::numeric, 1) || 'x, no unit
explanation'
                            ELSE 'Divided by ' || factor || ' (' || method || ' ' || round(ratio::numeric, 1) || 'x)' END
                FROM fix RETURNING source_key
            )
            UPDATE stg.harvest_clean c
            SET harvested_kg = CASE WHEN f.factor IS NULL THEN c.harvested_kg ELSE round(c.harvested_kg / f.factor, 1) END,
                dq_status    = CASE WHEN f.factor IS NULL THEN c.dq_status ELSE 'CORRECTED' END,
                dq_note      = concat_ws(',', c.dq_note, 'HV06')
            FROM fix f WHERE f.source_file = c.source_file AND f.row_num = c.row_num;
```

Reconciliation checks

`etl.sp_reconcile` runs 19 checks after every load and stores them in `audit.recon_result`, with tolerance, status and a generated explanation. They are of six types:

- **row counts:** raw = loaded + rejected + duplicates;
- **control totals:** kg, N$;
- **cross-source:** ERP kg vs stock dispatches; harvest vs stock receipts;
- **integrity:** stock-flow identity, SCD Type 2 overlaps and gaps, date continuity;
- **referential and completeness:** unknown members, weather coverage, price coverage;
- **plausibility:** Green Basket harvest vs NSA regional totals; price anomalies.

Table 23. Reconciliation results for the latest full batch (mart.recon_latest)

check_code	check_name	check_type	source_value	target_value	status	explanation
R01	Harvest log rows: raw = loaded + rejected + duplicates	ROW_COUNT	7,476	7,476	PASS	7368 loaded, 19 rejected to quarantine, 89 duplicates removed
R02	Harvest rows: cleansed staging = fact_harvest	ROW_COUNT	7,368	7,368	PASS	Every accepted harvest record loaded once
R03	Harvest control total (kg): staging = warehouse	CONTROL_TOTAL	214,476,707	214,476,707	PASS	Sum of standardised kilograms
R04	ERP sales rows: raw = loaded + rejected + duplicates	ROW_COUNT	305,490	305,490	PASS	302903 lines in fact_sales, 0 rejected, 2587 duplicate lines from overlapping extracts removed
R05	Net revenue control total (N$): staging = warehouse	CONTROL_TOTAL	2,110,889,915	2,110,889,915	PASS	Includes credit notes (negative)
R06	Cross-source: ERP invoiced kg vs stock-system dispatched kg	CROSS_SOURCE	196,854,183	196,994,249	PASS	Difference = goods dispatched on 29-30 Sep 2026 and invoiced after the extract (unbilled at cut-off). 4589630 kg were invoiced in a later month than dispatched (timing only).
R07	Stock-flow identity exceptions (opening + in - out - loss = closing)	INTEGRITY	0	0	PASS	After sign-error correction (ST01) and gap imputation (ST03)
R08	Stock continuity breaks (closing of month m-1 <> opening of month m)	INTEGRITY	0	0	PASS	Gaps imputed by rule ST03
R09	Cost ledger control total (N$): raw (excl. duplicates) = warehouse	CONTROL_TOTAL	1,267,300,948	1,267,300,948	PASS	Reversal/correction pairs retained (they are legitimate postings)

check_code	check_name	check_type	source_value	target_value	status	explanation
R10	Cost ledger rows: raw = loaded + duplicates/rejected	ROW_COUNT	15,428	15,428	PASS	46 duplicate/rejected postings
R11	Sales lines on Unknown farm (-1), % of lines	REFERENTIAL	2.0	0.814	PASS	2466 of 302903 lines lack farm of origin (rule SA05); threshold 2 %
R12	Sales lines with Unknown customer (-1) / open inferred members	REFERENTIAL	0	0	PASS	5 inferred customer member(s) currently awaiting master data (rule SA04)
R13	SCD2 integrity violations (current-row count, overlaps, gaps)	INTEGRITY	0	0	PASS	dim_farm 20 versions / 14 farms; dim_customer 277 versions / 240 customers
R14	Date dimension: contiguous days 2020-01-01..2027-12-31	COMPLETENESS	2,922	2,922	PASS	Required for Power BI time intelligence (mark as date table)
R15	Weather completeness: farm-days with rainfall (%)	COMPLETENESS	100	99.830	PASS	30674 farm-days; temperature complete for 30674 after interpolation (WE01)
R16	Commodity prices: months x 6 series loaded	COMPLETENESS	486	486	PASS	Missing Pink Sheet markers interpolated (MP01)
R17	Plausibility: Green Basket harvest exceeds the NSA regional estimate	PLAUSIBILITY	0	0	PASS	240 region-crop-season combinations compared
R18	Cross-source: harvest net of cull (kg) vs stock received (kg)	CROSS_SOURCE	205,316,227	205,881,267	PASS	Net residual -565040 kg (-0.27 %): harvest rows quarantined for invalid date/quantity (235129 kg as keyed; blank and negative entries carry no usable value until resubmitted) plus the gap between crop-level typical cull and actual grading losses; within the 0.5 % tolerance.
R19	Invoice lines flagged as price anomalies (SA07), count	PLAUSIBILITY	0	119	WARN	N$ 4,899,760 of revenue on flagged lines awaiting Sales Data Steward review

The single **WARN** (R19) is a business action, not a load failure: 119 invoice lines worth N$ 4.9 m carry a ×10 unit price and wait for the Sales Data Steward to issue credit notes. The revenue stays in the warehouse (it was invoiced) but is flagged and visible on page 4.

The two cross-source checks show how integration exposes business timing:

- **R06** (ERP vs stock dispatches) differs by goods dispatched on 29–30 September and invoiced after the cut-off.
- **R18** (harvest vs stock receipts) differs by 0.27 %. That is the quarantined harvest rows plus the gap between standard cull rates and actual grading losses.

Evidence that the warehouse was loaded

Table 24. Load batches (audit.etl_batch)

batch_id	batch_type	description	started	seconds	status
1	FULL	Full rebuild from raw extracts (cut-off 2026-09-30)	2026-10-27 14:02:38	31.6	SUCCEEDED
2	INCREMENTAL	Day-2 master-data delta 2026-10-05	2026-10-27 14:03:13	0.8	SUCCEEDED

Table 25. Step log of the full load (audit.etl_step_log): rows in, out, rejected and duration

step_name	rows_in	rows_out	rows_rejected	ms	note
transform master data	238	235	3	12	customers after merge of duplicates
load dim_date		2923	0	119	
load dim_region	14	16	0	1	
load dim_crop_category / dim_crop / dim_product	21	22	0	5	

step_name	rows_in	rows_out	rows_rejected	ms	note
load dim_cost_category	14	15	0	1	
load dim_farm (SCD2)	14	21	0	8	14 initial rows, 6 new versions, 0 Type 1 updates
load dim_customer (SCD2)	235	273	0	33	235 initial rows, 37 new versions, 0 inferred members completed
transform market prices & FX	5769	891	0	47	
load dim_commodity	6	6	0	2	
transform planting register	1327	1327	0	37	
transform harvest logs	7476	7368	108	1,005.00	
transform ERP sales	305,490	302,903	2587	15,439.00	
transform cost ledger	15,428	15,382	46	458	
transform stock movements	6519	6521	0	754	2 rows imputed
transform weather	14	30,674	0	383	JSON documents expanded to farm-days
transform NSA crop estimates	348	324	24	12	
load fact_sales	302,903	302,903	0	9,506.00	5 inferred customer members created
load fact_harvest	7368	7368	0	128	
load fact_crop_cycle (accumulating snapshot)	1327	1327	0	87	
load fact_production_cost	15,382	15,382	0	346	
load fact_inventory_monthly (periodic snapshot)	6521	6521	0	85	
load fact_weather_daily	30,674	30,674	0	404	
load fact_market_price_monthly	486	486	0	15	
load fact_crop_estimate	324	324	0	10	

Table 26. Sample query output: fact_sales joined to its dimensions (query ev_sample_fact_sales)

invoice_no	line_no	invoice_date	product_code	customer	channel	farm_code	ccy	quantity_kg	net_revenue_nad	cogs_nad	gross_margin_nad
INV2025-020001	1	2024-10-26	WHT-B3-BLK	Kalahari Wholesale Trading C	Wholesale	GB-OTJ-02	NAD	304	1,488.78	1,556.48	-67.7
INV2026-000500	1	2025-04-08	MAH-GR-B20	Informal Trader OD-001 (Okah	Informal market	GB-OHA-01	NAD	240	4,152.12	1,509.60	2,642.52
INV2026-024956	1	2025-12-01	GRP-EXP-C45	Maasvlakte Fruit Import BV	Export	GB-HAR-01	EUR	25,078.50	1,077,829.47	418,309.38	659,520.09
INV2026-024957	1	2025-12-01	GRP-EXP-C45	Maasvlakte Fruit Import BV	Export	GB-KHA-01	EUR	46,782.00	2,261,156.86	780,323.76	1,480,833.10
INV2026-024958	1	2025-12-01	GRP-EXP-C45	Westland Versimport BV	Export	GB-KHA-01	EUR	22,941.00	1,074,304.22	382,655.88	691,648.34
INV2026-024959	1	2025-12-01	GRP-EXP-C45	Westland Versimport BV	Export	GB-KHA-02	EUR	27,616.50	1,239,469.54	460,643.22	778,826.32

The extract log (`audit.etl_step_log`, query `ev_extract_files`) lists 44 landed files and their row counts. It is part of the evidence pack in `06_report/evidence/ev_extract_files.md`.

Screenshots to include in the final PDF (captured by the team on the demo machine; see `04_powerbi/BUILD_GUIDE.md` §9):

1. pgAdmin object browser showing the schemas `stg`, `ref`, `dw`, `mart`, `audit`, `etl`, `sec`.
2. The console output of `run_pipeline.py`, ending in the reconciliation table.
3. `SELECT * FROM mart.recon_latest;` in pgAdmin.
4. The `dw.dim_farm` rows for GB-KAW-01 (SCD Type 2).
5. Power BI *Table view* of `Sales` showing 302,903 rows.

Synthetic and public data

Generator design (`01_data/generate_synthetic_data.py`, about 1,900 lines). It is seeded (20261028) and fully reproducible: two runs produce identical files, and its manifest records counts and ground truth. It simulates the business rather than random rows:

- **Weather first.** A daily climate is generated per farm location. Season rainfall factors are anchored on the observed pattern: the 2023/24 El Niño drought at 0.50 of normal, and the good 2024/25 rains at 1.22.

- **Crop cycles.** Each farm's crop plan generates field blocks. Yield follows an agronomic response model. Plans are set at the budgeted yield, so yield vs plan is meaningful.
- **Downstream flows.** Harvests flow into stock after cull. Stock is sold through the six channels at channel-specific prices linked to World Bank and NAB-type price levels, FX and seasonality (grapes November–January). Costs are posted by cost centre (cycle).
- **Defects at realistic rates.** Duplicates from overlapping extracts, three harvest-log layouts, keying errors, missing values, late-arriving customers, master-data changes with effective dates, and a day-2 delta file.

File: `01_data/generate_synthetic_data.py` (lines 876–893)

```
                                # yield drivers
                                sy = season_start_year(first_harvest) if crop in ("WHT", "ONI", "POT", "TOM", "CAB", "BUT", "WML", "GRP") else
season_start_year(planting)
                                ratio = self.season_rain_ratio(farm_code, season_start_year(planting) if crop in ("MZW", "MAH", "WML") else sy)
                                flower_a = planting + timedelta(days=int(delay * 0.45))
                                flower_b = planting + timedelta(days=int(delay * 0.75))
                                hd = self.heat_days(farm_code, flower_a, min(flower_b, CUTOFF)) if flower_a <= CUTOFF else 0
                                fert_index = float(np.clip(rng.normal(1.0, 0.12), 0.7, 1.3))
                                base = self.base_yield(farm_code, crop, irrigated, state)
                                mg = self.mgmt_factor(farm_code, planting)
                                if not irrigated:
                                    f_rain = float(np.clip(0.15 + 0.85 * ratio ** 0.9, 0.1, 1.25))
                                else:
                                    f_rain = float(np.clip(1 + 0.04 * (ratio - 1), 0.95, 1.05))
                                heat_sens = 0.007 if crop in ("MZW", "WHT", "MAH") else (0.002 if crop == "GRP" else 0.004)
                                f_heat = max(0.6, 1 - heat_sens * hd)
                                f_fert = 1 + (0.30 if crop in ("MZW", "WHT", "MAH") else 0.20) * (fert_index - 1)
                                cyc_yield = base * mg * f_rain * f_heat * f_fert * float(rng.lognormal(0, 0.06))
                                plan_yield = round(base * mg * float(rng.uniform(0.95, 1.05)), 1)
```

The response model is known exactly, so §3.5 can test whether the analytics **recover the truth**: rainfall elasticity, heat and fertiliser effects. That is a stronger validation than fit statistics alone.

Public data and reproducible links. The external sources are public, free and need no API key. `01_data/fetch_public_data.py` downloads them into the same paths the pipeline reads:

- NASA POWER daily point API, Agroclimatology community: https://power.larc.nasa.gov/api/temporal/daily/point (parameters PRECTOTCORR, T2M_MAX, T2M_MIN, RH2M, ALLSKY_SFC_SW_DWN), one request per farm coordinate (NASA Langley Research Center, 2024).
- World Bank Commodity Price Data ("Pink Sheet"), monthly historical workbook: https://www.worldbank.org/en/research/commodity-markets (World Bank, 2026).
- FRED series DEXSFUS, DEXUSEU and DEXUSUK: https://fred.stlouisfed.org/graph/fredgraph.csv?id=DEXSFUS (Federal Reserve Bank of St. Louis, 2026).
- NSA / MAFWLR crop estimates: published as PDF reports by the Namibia Statistics Agency and the Ministry of Agriculture, Fisheries, Water and Land Reform. They are transcribed into `raw/external/nsa/crop_estimates_by_region.csv` in the layout of the stand-in file.

During the build, outbound access to these hosts was blocked. The generator therefore writes stand-ins that copy the published layouts exactly: the POWER JSON structure with −999 fill values, the Pink Sheet's three title rows and "…" markers, and FRED's blank holiday observations. The ETL already handles the real files. This is stated as assumption A2, and the stand-ins are never presented as real observations.

3.4 Power BI semantic model

The semantic model `GreenBasket_Agri.SemanticModel` is delivered as a **Power BI Project (PBIP)**: the model is stored in TMDL and the report in PBIR (Microsoft, 2024a, 2024d). The definition files use format version 4.0 (Microsoft Fabric Community, 2025). It is defined in one place, `04_powerbi/model_spec.py`, and generated by `04_powerbi/build_pbip.py`. Before release it was validated three ways:

- **Microsoft's Tabular Object Model** deserialised and re-serialised it in Power BI compatibility mode (round trip OK).
- **All 58 report files** passed the official PBIR JSON schemas.
- **A static DAX check** confirmed that every measure, column and table reference resolves.

22 tables (9 dimensions, 9 facts, Measures, field parameter, forecast drivers, RLS map)	**79** DAX measures in 8 display folders	**34** relationships, all many-to-one, single direction	**3** security roles (static and dynamic RLS)

Storage mode: Import, with incremental refresh

Table 27. Storage-mode decision

Option	Assessment for Green Basket	Decision
Import	303 k sales lines plus 8 smaller facts compress to about 11 MB in VertiPaq (DuckDB columnar proxy, §3.7), far below the 1 GB Pro limit. Every visual is answered from memory in milliseconds. Full DAX and time intelligence. Data freshness is daily, matching the sources (farm logs and ERP extracts are daily).	**Chosen**
DirectQuery	Every visual becomes SQL through the on-premises gateway to PostgreSQL. Latency adds up across 6–10 visuals per page. Load lands on the operational database. Some DAX patterns are slower. There is no real-time need.	Rejected
Composite (Dual + DirectQuery detail)	Useful once invoice detail reaches tens of millions of rows. Import the aggregate `mart.agg_sales_monthly` and query the detail by DirectQuery through user-defined aggregations. The aggregate already exists (§3.7), so this is a ready growth path.	Future option
Direct Lake	Needs Microsoft Fabric (OneLake Delta tables). Not in the division's licensing.	Not applicable

Incremental refresh. The `Sales` query filters on the integer invoice date key between the `RangeStart` and `RangeEnd` parameters. The filter is written so that it folds to PostgreSQL:

```
Int64.From(DateTime.ToText(RangeStart, "yyyyMMdd"))
```

The policy archives 5 years and refreshes the last 10 days. A daily refresh then re-reads about 2,200 rows instead of 302,903 (§3.7).

Relationships

All relationships are **many-to-one** from fact to dimension, with **single-direction** cross-filtering. There are no bidirectional or many-to-many relationships, which avoids ambiguous filter paths and keeps RLS propagation predictable. Role-playing dates are inactive relationships, activated inside measures with `USERELATIONSHIP`:

- Sales: invoice date (active) and dispatch date.
- Crop Cycle: planting date (active), first harvest, last harvest, and season end. Season end is used by every yield measure, so yield conforms to the NSA season.

`Security User Zone` is deliberately **disconnected**. It is read only by the RLS expression.

Table 28. The 34 relationships of the semantic model (generated from model_spec.py)

#	From (many side)	To (one side)	Cardinality	Cross-filter	Active
1	Sales[Invoice Date Key]	Date[Date Key]	many-to-one	single	yes
2	Sales[Dispatch Date Key]	Date[Date Key]	many-to-one	single	no (USERELATIONSHIP)
3	Sales[Product Key]	Product[Product Key]	many-to-one	single	yes
4	Sales[Customer Key]	Customer[Customer Key]	many-to-one	single	yes
5	Sales[Farm Key]	Farm[Farm Key]	many-to-one	single	yes
6	Product[Crop Key]	Crop[Crop Key]	many-to-one	single	yes
7	Crop[Crop Category Key]	Crop Category[Crop Category Key]	many-to-one	single	yes
8	Farm[Region Key]	Region[Region Key]	many-to-one	single	yes
9	Harvest[Harvest Date Key]	Date[Date Key]	many-to-one	single	yes
10	Harvest[Farm Key]	Farm[Farm Key]	many-to-one	single	yes

#	From (many side)	To (one side)	Cardinality	Cross-filter	Active
11	Harvest[Crop Key]	Crop[Crop Key]	many-to-one	single	yes
12	Crop Cycle[Planting Date Key]	Date[Date Key]	many-to-one	single	yes
13	Crop Cycle[First Harvest Date Key]	Date[Date Key]	many-to-one	single	no (USERELATIONSHIP)
14	Crop Cycle[Last Harvest Date Key]	Date[Date Key]	many-to-one	single	no (USERELATIONSHIP)
15	Crop Cycle[Harvest Season End Date Key]	Date[Date Key]	many-to-one	single	no (USERELATIONSHIP)
16	Crop Cycle[Farm Key]	Farm[Farm Key]	many-to-one	single	yes
17	Crop Cycle[Crop Key]	Crop[Crop Key]	many-to-one	single	yes
18	Production Cost[Posting Date Key]	Date[Date Key]	many-to-one	single	yes
19	Production Cost[Farm Key]	Farm[Farm Key]	many-to-one	single	yes
20	Production Cost[Crop Key]	Crop[Crop Key]	many-to-one	single	yes
21	Production Cost[Cost Category Key]	Cost Category[Cost Category Key]	many-to-one	single	yes
22	Inventory[Snapshot Date Key]	Date[Date Key]	many-to-one	single	yes
23	Inventory[Farm Key]	Farm[Farm Key]	many-to-one	single	yes
24	Inventory[Product Key]	Product[Product Key]	many-to-one	single	yes
25	Weather[Date Key]	Date[Date Key]	many-to-one	single	yes
26	Weather[Farm Key]	Farm[Farm Key]	many-to-one	single	yes
27	Market Price[Month Date Key]	Date[Date Key]	many-to-one	single	yes
28	Market Price[Commodity Key]	Commodity[Commodity Key]	many-to-one	single	yes
29	Market Price[Crop Key]	Crop[Crop Key]	many-to-one	single	yes
30	Crop Estimate[Season End Date Key]	Date[Date Key]	many-to-one	single	yes
31	Crop Estimate[Region Key]	Region[Region Key]	many-to-one	single	yes
32	Crop Estimate[Crop Key]	Crop[Crop Key]	many-to-one	single	yes
33	Forecast[Month Date Key]	Date[Date Key]	many-to-one	single	yes
34	Forecast[Crop Category Key]	Crop Category[Crop Category Key]	many-to-one	single	yes

Model hygiene

- **Hidden technical columns.** 93 of 198 columns are hidden:
  - all surrogate and foreign keys;
  - sort keys;
  - the raw numeric columns behind measures.

  Users therefore see only business attributes and measures. **Implicit measures are disabled** (`discourageImplicitMeasures`), so every number on a page comes from a governed DAX measure.
- **Display folders.**
  - Date columns sit in *Calendar*, *Fiscal (Apr–Mar)* and *Agricultural season (Oct–Sep)*.
  - Farm and Customer separate *As-is (current)* attributes from *SCD history*.
  - Measures sit in a dedicated `_Measures` table in folders *0 Labels* to *7 Data Quality & Security*.
- **Sort-by-column.** Six settings, for example:
  - Month Name by Month Number;
  - Fiscal Month (Apr = 1) by Fiscal Month Number;
  - Year Month by Year Month Key;
  - Day Name by ISO day of week.

  These make April the first month of the fiscal year in every visual.
- **Hierarchies.** Fiscal Calendar (FY › FQ › Month › Date), Calendar, Agricultural Season (Season › Month › Date) and Geography (Zone › Region).
- **Data categories.** Region is *State or Province*; Town is *City*; farm coordinates are *Latitude/Longitude* for map visuals.
- **Date table.** `Date` is marked as the date table, and Auto date/time is switched off (`__PBI_TimeIntelligenceEnabled = 0`), which avoids dozens of hidden date tables.
- **Format strings.** `"N$" #,0` for currency and `0.0%` for ratios, set on the measures themselves.

DAX measures

There are 79 measures in total; Appendix B lists all of them, generated from `model_spec.py`. The fourteen below cover every required pattern:

- time intelligence: YTD, YoY and rolling averages;
- `CALCULATE` with filter modification: `KEEPFILTERS`, `REMOVEFILTERS`, `USERELATIONSHIP`, and a `TOPN` table filter;
- a semi-additive balance;
- a drill-across benchmark.

Fiscal year to date uses year end `"3/31"`. The comparison with last year is **like-for-like**: it stops at the same day last year as the latest data date. Without that cut-off, a partial FY2027 would be compared with a complete FY2026 and show a false −73 %.

File: `04_powerbi/measures.dax` (generated from `model_spec.py`)

```
-- Display folder: 1 Sales | Format: "N$" #,0
-- Invoice + credit-note revenue after discount, N$ (FX-converted).
Net Revenue =
    SUM ( 'Sales'[Net Revenue] )

-- Display folder: 1 Sales | Format: 0.0%
-- Gross margin as a share of net revenue.
Gross Margin % =
    DIVIDE ( [Gross Margin], [Net Revenue] )

-- Display folder: 2 Time Intelligence | Format: dd mmm yyyy
-- Latest invoice date in the model (ignores date filters).
Last Sales Date =
    VAR _k =
        CALCULATE ( MAX ( 'Sales'[Invoice Date Key] ), REMOVEFILTERS ( 'Date' ) )
    RETURN
        IF ( NOT ISBLANK ( _k ), DATE ( INT ( _k / 10000 ), MOD ( INT ( _k / 100 ), 100 ), MOD ( _k, 100 ) ) )

-- Display folder: 2 Time Intelligence | Format: "N$" #,0
-- Fiscal year-to-date (year ends 31 March), no future guard.
Net Revenue FYTD Raw =
    TOTALYTD ( [Net Revenue], 'Date'[Date], "3/31" )

-- Display folder: 2 Time Intelligence | Format: "N$" #,0
-- Fiscal year-to-date net revenue; blank for periods after the last sale.
Net Revenue FYTD =
    IF ( MIN ( 'Date'[Date] ) <= [Last Sales Date], [Net Revenue FYTD Raw] )

-- Display folder: 2 Time Intelligence | Format: "N$" #,0
-- Same fiscal-YTD window one year earlier, cut at the same day (like-for-like).
Net Revenue FYTD PY =
    VAR _cutoffPY =
        EDATE ( [Last Sales Date], -12 )
    RETURN
        CALCULATE (
            [Net Revenue FYTD Raw],
            SAMEPERIODLASTYEAR ( 'Date'[Date] ),
            KEEPFILTERS ( 'Date'[Date] <= _cutoffPY )
        )

-- Display folder: 2 Time Intelligence | Format: 0.0%
-- Fiscal-YTD growth versus the like-for-like prior-year window.
Net Revenue FYTD YoY % =
    DIVIDE ( [Net Revenue FYTD] - [Net Revenue FYTD PY], [Net Revenue FYTD PY] )
```

Prior year, year-on-year and rolling measures. The rolling measures stop at the last data date, so the trailing months are not half-empty.

File: `04_powerbi/measures.dax` (generated from `model_spec.py`)

```
-- Display folder: 2 Time Intelligence | Format: "N$" #,0
-- Net revenue in the same period last year.
Net Revenue PY =
    CALCULATE ( [Net Revenue], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

-- Display folder: 2 Time Intelligence | Format: 0.0%
-- Growth versus the same period last year.
Net Revenue YoY % =
    VAR _cur = [Net Revenue]
    VAR _py = [Net Revenue PY]
    RETURN
        IF ( NOT ISBLANK ( _cur ) && NOT ISBLANK ( _py ), DIVIDE ( _cur - _py, _py ) )

-- Display folder: 2 Time Intelligence | Format: "N$" #,0
-- Moving annual total: trailing 12 months ending at the context end (capped at the last sale).
Net Revenue MAT =
    VAR _end =
        MIN ( MAX ( 'Date'[Date] ), [Last Sales Date] )
    RETURN
        IF (
            MIN ( 'Date'[Date] ) <= [Last Sales Date],
            CALCULATE ( [Net Revenue], DATESINPERIOD ( 'Date'[Date], _end, -12, MONTH ) )
        )

-- Display folder: 2 Time Intelligence | Format: "N$" #,0
-- Average monthly net revenue over the trailing three months.
Net Revenue 3M Rolling Avg =
    VAR _end =
        MIN ( MAX ( 'Date'[Date] ), [Last Sales Date] )
    RETURN
        IF (
            MIN ( 'Date'[Date] ) <= [Last Sales Date],
```

```
            DIVIDE ( CALCULATE ( [Net Revenue], DATESINPERIOD ( 'Date'[Date], _end, -3, MONTH ) ), 3 )
        )
```

`CALCULATE` with filter-context modification:

- `KEEPFILTERS` intersects with the user's filters instead of replacing them, so Export Share stays correct when the user also slices by channel.
- `REMOVEFILTERS` creates a share of the grand total.
- `TOPN` builds a table filter.
- `USERELATIONSHIP` switches the date role.

File: `04_powerbi/measures.dax` (generated from `model_spec.py`)

```
-- Display folder: 1 Sales | Format: "N$" #,0
-- Net revenue from customers in the Export channel (CALCULATE with filter modification).
Export Revenue =
    CALCULATE ( [Net Revenue], KEEPFILTERS ( 'Customer'[Channel] = "Export" ) )

-- Display folder: 1 Sales | Format: 0.0%
-- Export revenue as a share of net revenue.
Export Share % =
    DIVIDE ( [Export Revenue], [Net Revenue] )

-- Display folder: 1 Sales | Format: 0.0%
-- Share of the selection in total revenue across all crops (filters on the product hierarchy removed).
Revenue Share of All Crops % =
    DIVIDE (
        [Net Revenue],
        CALCULATE ( [Net Revenue], REMOVEFILTERS ( 'Product' ), REMOVEFILTERS ( 'Crop' ), REMOVEFILTERS ( 'Crop Category' ) )
    )

-- Display folder: 1 Sales | Format: 0.0%
-- Revenue concentration: share of the ten largest customers in the context.
Top 10 Customer Share % =
    VAR _top10 =
        TOPN ( 10, VALUES ( 'Customer'[Customer Code] ), [Net Revenue] )
    RETURN
        DIVIDE ( CALCULATE ( [Net Revenue], _top10 ), [Net Revenue] )

-- Display folder: 2 Time Intelligence | Format: #,0
-- Tonnes by DISPATCH date - activates the inactive role-playing relationship.
Dispatched Volume (t) =
    CALCULATE (
        DIVIDE ( SUM ( 'Sales'[Quantity (kg)] ), 1000 ),
        USERELATIONSHIP ( 'Sales'[Dispatch Date Key], 'Date'[Date Key] ),
        KEEPFILTERS ( 'Sales'[Document Type] = "INV" )
    )
```

Yield, the benchmark and the semi-additive stock balance.

- **Yield** uses the season-end date role, so it conforms to the NSA estimates.
- **The benchmark** drills across to `Crop Estimate` through the shared Region, Crop and Date dimensions.
- **Closing stock** takes the last snapshot in the period. It is never summed over months; the SQL proof that the sum would be wrong is in §3.5.

File: `04_powerbi/measures.dax` (generated from `model_spec.py`)

```
-- Display folder: 3 Production & Yield | Format: #,0
-- Harvest of completed cycles, by agricultural season of harvest.
Completed Harvest (t) =
    CALCULATE (
        DIVIDE ( SUM ( 'Crop Cycle'[Harvested (kg)] ), 1000 ),
        KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Completed" ),
        USERELATIONSHIP ( 'Crop Cycle'[Harvest Season End Date Key], 'Date'[Date Key] )
    )

-- Display folder: 3 Production & Yield | Format: #,0.00
-- Tonnes per harvested hectare (completed cycles).
Yield (t/ha) =
    DIVIDE ( [Completed Harvest (t)], [Harvested Area (ha)] )

-- Display folder: 3 Production & Yield | Format: #,0.00
-- NSA commercial-sector yield for the same region/crop/season (drill-across).
Benchmark Yield (t/ha) =
    VAR _prod =
        CALCULATE ( SUM ( 'Crop Estimate'[Production (t)] ), KEEPFILTERS ( 'Crop Estimate'[Farming Sector] = "Commercial" ) )
    VAR _area =
        CALCULATE ( SUM ( 'Crop Estimate'[Area Planted (ha)] ), KEEPFILTERS ( 'Crop Estimate'[Farming Sector] = "Commercial" ) )
    RETURN
        DIVIDE ( _prod, _area )

-- Display folder: 4 Inventory | Format: #,0
-- Stock on hand at the LAST month-end snapshot in the period across all farms/products (semi-additive).
Closing Stock (t) =
    VAR _last =
        CALCULATE (
            MAX ( 'Inventory'[Snapshot Date Key] ),
            REMOVEFILTERS ( 'Farm' ), REMOVEFILTERS ( 'Region' ), REMOVEFILTERS ( 'Product' ), REMOVEFILTERS ( 'Crop' ), REMOVEFILTERS ( 'Crop
Category' )
        )
    RETURN
        DIVIDE ( CALCULATE ( SUM ( 'Inventory'[Closing Stock (kg)] ), 'Inventory'[Snapshot Date Key] = _last ), 1000 )

-- Display folder: 4 Inventory | Format: #,0
-- Days of sales covered by closing stock.
```

```
Stock Cover (days) =
    DIVIDE ( [Closing Stock (t)], [Avg Daily Dispatch L90D (t)] )
```

Field parameter. `Metric Selector` is a field parameter over six measures:

- Net Revenue
- Gross Margin
- Gross Margin %
- Sales Volume (t)
- Avg Realised Price (N$/kg)
- Export Share %

One trend visual can then answer six questions. A slicer on `'Metric Selector'[Metric]` switches the measure without duplicating visuals. This is a non-technical user's self-service entry point (§3.6).

Row-level security

Table 29. Security roles (generated from model_spec.py)

Role	Table filtered	DAX filter expression	Purpose
Executive (all data)	-	(none - unrestricted)	Unrestricted read for the Executive Committee and CFO.
Zone Manager (dynamic)	Farm	`'Farm'[Zone Code] IN CALCULATETABLE ( VALUES ( 'Security User Zone'[Zone Code] ), 'Security User Zone'[User UPN] = USERPRINCIPALNAME () )`	Dynamic RLS: users see farms (and all facts from those farms) in the zones mapped to their UPN in sec.user_zone_access.
Export Desk (static)	Customer	`'Customer'[Channel] = "Export"`	Static RLS: the export desk sees export-channel customers only.

Dynamic RLS design.

- The mapping of user to zone lives in the warehouse (`sec.user_zone_access`). The data owner maintains it there, so access changes need no model republish.
- The model imports the mapping as the hidden, disconnected table `Security User Zone`.
- The *Zone Manager* filter on `Farm` keeps only farms whose management zone is mapped to `USERPRINCIPALNAME()`.
- Filters flow from Farm to all six farm-linked facts: sales, harvest, crop cycle, cost, inventory and weather.
- One user can hold several zones. `ops.north` holds North-Central and North-East.
- An unmapped user sees **nothing**, so the role fails closed.

Test plan and results. The expected values were computed independently in SQL (query `ev_rls_expected` in `12_evidence_queries.sql`). Each role is then checked in Power BI Desktop with **Modeling › View as › Other user + role**, and after publishing with **Security › Test as role**.

Table 30. RLS test matrix: expected FY2026 net revenue per test identity (SQL)

Test identity (UPN)	Role	Zones	Farms visible	Expected FY2026 net revenue (N$)
exec (no restriction)	Executive (all data)	all incl. Unknown	14	458,773,435
zm.northcentral@greenbasket.example	Zone Manager (dynamic)	NC	4	62,708,333
zm.northeast@greenbasket.example	Zone Manager (dynamic)	NE	4	76,932,250
zm.central@greenbasket.example	Zone Manager (dynamic)	CE	3	55,619,506
zm.south@greenbasket.example	Zone Manager (dynamic)	SO	3	256,687,639
ops.north@greenbasket.example	Zone Manager (dynamic)	NC + NE	8	139,640,583
intern@greenbasket.example	Zone Manager (dynamic)	none	(blank)	(blank)
export desk	Export Desk (static)	Channel = Export	14	212,133,855

zone_code	zone_name	farms	net_revenue_fy2026_nad
CE	Central	3	55,619,506.00
NC	North-Central	4	62,708,333.00
NE	North-East	4	76,932,250.00
SO	South	3	256,687,639.00
XX	Unknown	1	6,825,707.00
		15	458,773,435.00

Reading the test.

- The four zone totals add up to N$ 451,947,728.
- The N$ 6,825,707 difference is revenue on the *Unknown* farm (rule SA05). Only executives see it, by design.
- The page-4 card *Farms visible (RLS)* (`[Visible Farms]`) and *Signed-in user* (`[Current User]`) make every test self-documenting in a screenshot.

Security limits that the team must know.

- **RLS restricts *Viewers* only.** Workspace Admins, Members and Contributors bypass it, so zone managers consume the content through the **app** with Viewer rights (§3.8).
- **Dimension members are not filtered.** Zone managers can still see the *names* of customers outside their zone in a slicer, although with no revenue. This is accepted for a B2B customer list. If it became sensitive, the remedy is "apply security filter in both directions" on Sales→Customer, at a measured performance cost, or object-level security on `Customer`.
- **RLS is not a substitute for a data-sharing decision.** The Executive role holds all data, so its membership is limited to the executive committee and the CFO, through an Entra ID group.

Screenshots to include:

1. Manage roles showing the DAX filter.
2. View as zm.northcentral with the page-4 card showing 4 farms.
3. View as the unmapped intern with blank visuals.
4. The service *Test as role* page after publishing.

3.5 Advanced analytics

OLAP operations

Each operation was run as SQL against the warehouse (`02_sql/11_olap_queries.sql`; the outputs below were captured by `capture_sql_evidence.py`). Each is mirrored by an interaction in the Power BI report, which is what the screenshots should show.

Roll-up: month → fiscal quarter → fiscal year → total

`GROUP BY ROLLUP` produces the subtotal levels in one pass. `GROUPING()` labels them.

File: `02_sql/11_olap_queries.sql` (lines 8–18)

```sql
-- name: olap_rollup
-- ROLL-UP: month -> fiscal quarter -> fiscal year -> grand total (net revenue, N$ million)
SELECT coalesce(d.fiscal_year_label, 'ALL YEARS')        AS fiscal_year,
       coalesce(d.fiscal_quarter_label, '(year total)')   AS fiscal_quarter,
       round(sum(f.net_revenue_nad) / 1e6, 2)             AS net_revenue_nad_m,
       GROUPING(d.fiscal_year_label, d.fiscal_quarter_label) AS grouping_level
FROM dw.fact_sales f
JOIN dw.dim_date d ON d.date_key = f.invoice_date_key
WHERE d.fiscal_year BETWEEN 2025 AND 2026
GROUP BY ROLLUP (d.fiscal_year_label, d.fiscal_quarter_label)
ORDER BY d.fiscal_year_label NULLS LAST, d.fiscal_quarter_label NULLS LAST;
```

fiscal_year	fiscal_quarter	net_revenue_nad_m	grouping_level
FY2025	FY2025-FQ1	60.8	0
FY2025	FY2025-FQ2	55.76	0
FY2025	FY2025-FQ3	248.97	0
FY2025	FY2025-FQ4	53.89	0
FY2025	(year total)	419.41	1
FY2026	FY2026-FQ1	63.15	0
FY2026	FY2026-FQ2	53.49	0
FY2026	FY2026-FQ3	276.35	0
FY2026	FY2026-FQ4	65.79	0
FY2026	(year total)	458.77	1
ALL YEARS	(year total)	878.18	3

What it reveals.

- FQ3 (October–December) carries **60 %** of each year's revenue: N$ 276.4 m of N$ 458.8 m in FY2026. That is the grape export window.
- The other three quarters are nearly flat at N$ 53–66 m.

Power BI: the Fiscal Calendar hierarchy on the page-1 trend. *Drill up* goes from months to quarters to years.

Drill-down along the snowflaked product hierarchy

category	crop	product	net_revenue_nad_m	volume_t
Cereals & grains	Mahangu (pearl millet)	Mahangu Grain 20kg Bag	1.83	132
Cereals & grains	Mahangu (pearl millet)	Mahangu Grain 50kg Bag	2.75	198
Cereals & grains	Mahangu (pearl millet)	(crop total)	4.59	329
Cereals & grains	Wheat	Wheat B1 Bulk	13.22	2,189.00
Cereals & grains	Wheat	Wheat B3 Bulk	3.33	687
Cereals & grains	Wheat	(crop total)	16.55	2,876.00
Cereals & grains	White maize	White Maize WM1 50kg Bag	8.76	1,098.00
Cereals & grains	White maize	White Maize WM1 Bulk	42.91	6,822.00
Cereals & grains	White maize	White Maize WM2 Bulk	17.63	3,093.00
Cereals & grains	White maize	(crop total)	69.3	11,013.00
Cereals & grains	(category total)	(crop total)	90.44	14,218.00
Fruit	Table grapes	Table Grapes Export 4.5kg Carton	209.81	4,533.00
Fruit	Table grapes	Table Grapes Local 5kg Box	26.19	1,000.00
Fruit	Table grapes	(crop total)	236	5,532.00
Fruit	Watermelon	Watermelon Class 1 per Unit	3.43	672
Fruit	Watermelon	Watermelon Class 2 Bulk Bin	0.8	247
Fruit	Watermelon	(crop total)	4.23	919
Fruit	(category total)	(crop total)	240.23	6,452.00
ALL CATEGORIES	(category total)	(crop total)	330.67	20,670.00

What it reveals.

- Drilling from category to crop to product shows that **export cartons** (N$ 209.8 m) outweigh local grape boxes (N$ 26.2 m) eight to one.
- Within maize, bulk WM1 (N$ 42.9 m) dominates bagged product.

Power BI: the crop bar chart on page 1, with Category › Crop › Product on the axis. *Expand to next level* is enabled.

Slice: FY2026 revenue by management zone

zone_name	farm_versions	net_revenue_nad_m	share_pct
South	3	256.69	56
North-East	4	76.93	16.8
North-Central	4	62.71	13.7
Central	3	55.62	12.1
Unknown	1	6.83	1.5

What it reveals. Fixing one member (FY2026) shows the **South** zone (Aussenkehr, Noordoewer, Hardap) earning 56 % of revenue on 3 farms. Concentration risk is geographic as well as by customer.

Power BI: the Fiscal Year slicer set to FY2026, with the Zone slicer.

Dice: Vegetables × North-Central × FY2025–26 × Retail and Informal

fiscal_year_label	crop_name	channel	net_revenue_nad_m	avg_price_nad_per_kg
FY2025	Butternut	Informal market	0.68	6.59
FY2026	Butternut	Informal market	0.74	6.87
FY2025	Butternut	Retail	0.89	8.3
FY2026	Butternut	Retail	1.06	8.62
FY2025	Cabbage	Informal market	1.27	2.77
FY2026	Cabbage	Informal market	1.2	2.83
FY2025	Cabbage	Retail	2.75	3.12
FY2026	Cabbage	Retail	2.52	3.18
FY2025	Onions	Informal market	1.69	7.96
FY2026	Onions	Informal market	2.04	7.81
FY2025	Onions	Retail	3.36	9.21
FY2026	Onions	Retail	3.88	8.97
FY2025	Tomatoes	Informal market	5.4	11.51
FY2026	Tomatoes	Informal market	4.91	11.35
FY2025	Tomatoes	Retail	9.8	13.55
FY2026	Tomatoes	Retail	8.9	13.42

What it reveals.

- The sub-cube isolates the northern fresh-vegetable trade.
- Informal-market prices run 11–21 % below retail for the same crop: tomatoes at N$ 11.35 vs 13.42 per kg in FY2026.
- Tomato revenue fell in both channels in FY2026. That is an early warning, not visible at division level.

Power BI: four slicers on page 2 or 4. The question is answered without writing SQL.

Pivot: crops as columns, fiscal years as rows

fiscal_year_label	maize	wheat	mahangu	vegetables	table_grapes	watermelon	total
FY2022	57.9	21.2	5.4	111.7	156.3	3	355.5
FY2023	61.4	22.4	4.1	110.6	170.3	3.4	372.2
FY2024	48.1	17.8	3.6	114.3	194.2	3.6	381.5
FY2025	63.6	15.8	2.5	124.3	208.9	4.3	419.4
FY2026	69.3	16.6	4.6	128.1	236	4.2	458.8
FY2027	42.7	2.4	2.9	75.3	0.2		123.5

What it reveals.

- Rotating crops into columns makes the growth path legible: table grapes grew from N$ 156 m to N$ 236 m (+51 % in four years).
- Maize dipped in FY2024, the sales year of the 2022/23 crop (rain factor 0.86). It then grew through the 2023/24 drought. That is consistent with new irrigated area coming online: the Nkurenkuru pivot in 2023 and the Liselo expansion in 2024. Irrigation appears to have insulated revenue; prices also moved, so this is not a clean attribution.
- FY2027 is a 6-month partial year.

Power BI: a matrix with Crop on columns. Switching rows and columns is a pivot.

Cube and drill-across (beyond the required five)

`GROUP BY CUBE(channel, category)` gives every subtotal combination. It shows that **export fruit** earns a 63.7 % gross margin, against 32.1 % for millers:

channel	category	net_revenue_nad_m	gross_margin_pct
Export	Fruit	209.81	63.7
Export	Vegetables	2.33	60.2
Export	ALL CATEGORIES	212.13	63.7
Informal market	Cereals & grains	2.63	56.6
Informal market	Fruit	1.75	58.9
Informal market	Vegetables	27.94	51.9
Informal market	ALL CATEGORIES	32.32	52.7
Institutional	Cereals & grains	1.63	54.2
Institutional	Vegetables	11.07	52.8
Institutional	ALL CATEGORIES	12.7	52.9
Miller/Processor	Cereals & grains	66.23	32.1
Miller/Processor	ALL CATEGORIES	66.23	32.1

Drill-across joins two fact tables through the conformed Region, Crop and Date dimensions. Green Basket maize yield (`fact_crop_cycle`) is set against the NSA commercial benchmark (`fact_crop_estimate`):

region_name	season	gb_harvested_ha	gb_yield_t_ha	nsa_commercial_t_ha	nsa_communal_t_ha	gap_vs_commercial
Hardap	2022/23	40	7.47	7.07		0.39
Hardap	2023/24	57	6.63	6.13		0.5
Hardap	2024/25	40	6.73	6.71		0.02
Kavango East	2022/23	247	8.2	8.5	0.47	-0.3
Kavango East	2023/24	247	8.06	8.45	0.32	-0.39
Kavango East	2024/25	218	8.78	8.9	0.66	-0.13
Kavango West	2022/23	219	1.3	1.36	0.51	-0.06
Kavango West	2023/24	121	8.79	7.96	0.31	0.83
Kavango West	2024/25	145	8.34	7.78	0.62	0.56
Ohangwena	2022/23	110	1.36	1.27	0.47	0.09
Ohangwena	2023/24	57	1.11	0.56	0.32	0.55
Ohangwena	2024/25	82	2.44	2.38	0.7	0.07
Omusati	2022/23	33	8.88	8.36	0.45	0.52
Omusati	2023/24	41	8.51	7.95	0.32	0.56

Kavango West moved from below the national benchmark (1.30 vs 1.36 t/ha, dryland 2022/23) to **+0.83 t/ha above** it once Nkurenkuru's pivot came online. The conformed dimensions answer that question without a manual join.

Semi-additive measures. Summing month-end stock over a quarter overstates it 2.4–4.6 times. The roll-up must take the last month:

fiscal_quarter_label	closing_stock_t_correct	closing_stock_t_wrong_sum_of_months	closing_value_nad_m
FY2026-FQ1	7,637.00	18,606.00	30.07
FY2026-FQ2	5,263.00	17,147.00	19.67
FY2026-FQ3	4,312.00	15,336.00	20.39
FY2026-FQ4	1,396.00	6,400.00	6.28

Technique 1: revenue forecasting (time series)

Question (BQ4): what revenue should the division plan for in the next 12 months, and how reliable is that number?

Method (`03_analytics/forecasting.py`):

- **Series:** monthly net revenue for each crop category and for the division total, April 2021 – September 2026 (66 months).
- **Back-test:** train on 54 months; hold out the last 12 (October 2025 – September 2026), a full season.
- **Models:**
  1. **seasonal naive**, the benchmark: same month last year;
  2. **ETS(A,Ad,A)**: additive error, damped trend, additive seasonality;
  3. **SARIMA(1,0,0)(0,1,1)$_{12}$** with drift.
- **Transformation:** `log1p` is used when a series has no near-zero months. Fruit revenue is zero outside the grape season, so fruit stays on the raw scale.
- **Metrics:**
  - MAE and RMSE;
  - MAPE, computed only where actuals are not near zero;
  - **WAPE**, the primary metric, because it is robust to small months;
  - **MASE** against the in-sample seasonal naive (Hyndman & Koehler, 2006);
  - empirical coverage of the 80 % and 95 % prediction intervals.

Table 31. Hold-out accuracy (Oct 2025 - Sep 2026) by category and model; best = lowest WAPE

category	model	scale	MAE_Nm	RMSE_Nm	MAPE_pct	WAPE_pct	MASE	cov80_pct	cov95_pct	mape_months	best
Cereals & grains	Seasonal naive	raw	1.04	1.21	15.57	15.32	0.89	83.33	100.00	12	
Cereals & grains	ETS(A,Ad,A)	log1p	1.12	1.37	16.93	16.55	0.96	91.67	100.00	12	
Cereals & grains	SARIMA(1,0,0)(0,1,1)12	log1p	0.62	0.81	8.95	9.20	0.53	100.00	100.00	12	**best**
Vegetables	Seasonal naive	raw	2.96	4.85	23.20	24.61	1.44	75.00	83.33	11	
Vegetables	ETS(A,Ad,A)	raw	3.39	4.74	32.38	28.24	1.65	91.67	100.00	11	
Vegetables	SARIMA(1,0,0)(0,1,1)12	raw	2.89	4.70	22.95	24.07	1.41	75.00	83.33	11	**best**
Fruit	Seasonal naive	raw	2.31	4.49	24.51	11.54	0.79	91.67	100.00	4	**best**
Fruit	ETS(A,Ad,A)	raw	2.32	4.53	24.50	11.62	0.79	100.00	100.00	4	
Fruit	SARIMA(1,0,0)(0,1,1)12	raw	2.76	4.04	17.00	13.78	0.94	91.67	100.00	4	
Division total	Seasonal naive	raw	4.93	6.67	16.43	12.71	1.07	83.33	100.00	12	**best**
Division total	ETS(A,Ad,A)	log1p	6.00	9.72	13.81	15.45	1.30	66.67	83.33	12	
Division total	SARIMA(1,0,0)(0,1,1)12	log1p	5.51	8.76	13.45	14.20	1.19	75.00	100.00	12	

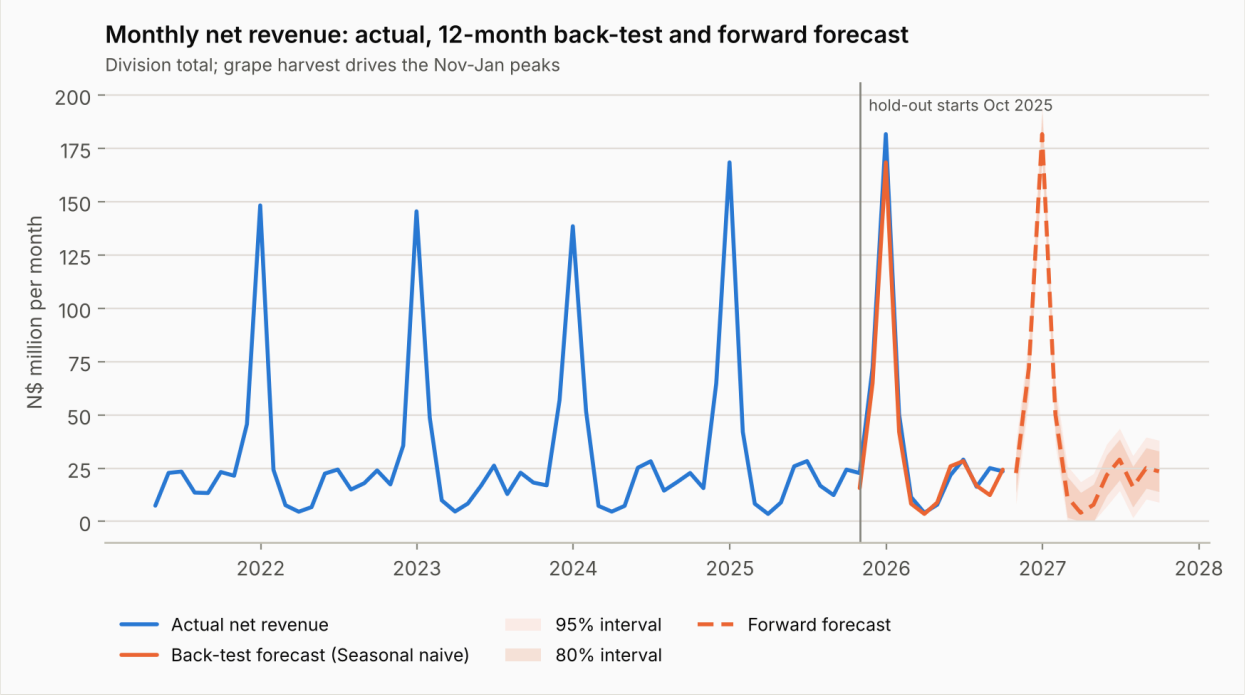


Figure 7. Division total: actual monthly net revenue, 12-month back-test of the selected model, and the forward forecast with 80 % and 95 % intervals

Findings.

1. At division level, the **seasonal naive model wins** (WAPE 12.7 %), and its 80 % interval covers 83 % of the hold-out months, close to nominal. The business is strongly seasonal and stable, and with 4.5 seasons of training data the extra parameters of ETS and SARIMA cost more variance than they remove. This is the classic benchmark result (Hyndman & Athanasopoulos, 2021).
2. **By category** the picture differs:
   - SARIMA is clearly best for cereals: WAPE 9.2 %, MASE 0.53.
   - SARIMA is marginally best for vegetables, but at 24 % WAPE, vegetables remain the least predictable.
   - The naive model is best for fruit (11.5 %).

   Picking the best model per category reduces the pooled category-level WAPE from 16.2 % to 15.0 %.
3. **Forward forecast:**
   - October 2026 – September 2027: N$ 465.6 m.
   - FY2027 will close at about **N$ 465.6 m**: N$ 123.5 m actual for April–September plus N$ 342.1 m forecast for October–March. That is **+1.5 %**, well below the 8 % growth objective (O1), so mix and margin must deliver the gap.

Model: Seasonal naive (lowest hold-out WAPE for the division total)

month	forecast_Nm	lo80_Nm	hi80_Nm	lo95_Nm	hi95_Nm
2026-10	22.70	13.20	32.20	8.20	37.20
2026-11	71.90	62.40	81.40	57.40	86.40
2026-12	181.70	172.20	191.20	167.20	196.20
2027-01	50.20	40.80	59.70	35.70	64.80
2027-02	11.50	2.00	20.90	0.00	26.00
2027-03	4.10	0.00	13.60	0.00	18.60
2027-04	7.90	0.00	17.30	0.00	22.40
2027-05	21.70	12.20	31.20	7.20	36.20
2027-06	29.00	19.60	38.50	14.50	43.50
2027-07	16.30	6.80	25.80	1.80	30.80
2027-08	25.10	15.60	34.50	10.50	39.60
2027-09	23.50	14.00	33.00	9.00	38.00

Forecast Oct 2026 - Sep 2027 total: N$ 465.6 m (sum of point forecasts; FY2027 H2 Oct 2026 - Mar 2027 = N$ 342.1 m)

Assumptions:

- The seasonal pattern persists.
- No structural break: grape area is stable, and no new export market opens.
- Errors are approximately normal for the intervals.

Limitations:

- There is a single 12-month hold-out, i.e. one season. Rolling-origin evaluation over several seasons would give more stable error estimates.
- No exogenous drivers yet: the FX rate, EU grape prices and rainfall are known leading indicators. A SARIMAX with them is the next step.
- The dashboard card reports the category-grain WAPE (16.2 % for the naive model), while the division-total WAPE is 12.7 %. Both are labelled.

In Power BI: page 3 shows actual and forecast with the 80 % band. A model slicer, read through `SELECTEDVALUE`, switches between the three models stored in `fact_forecast_monthly`.

File: `03_analytics/forecasting.py` (lines 46–90)

```
        def seasonal_naive(train: np.ndarray, h: int):
            reps = int(np.ceil(h / 12))
            fc = np.tile(train[-12:], reps)[:h]
            resid = train[12:] - train[:-12]
            sd = resid.std(ddof=1)
            k = np.sqrt(np.floor(np.arange(h) / 12) + 1)  # seasonal-naive variance grows with whole seasons
            return fc, fc - 1.2816 * sd * k, fc + 1.2816 * sd * k, fc - 1.96 * sd * k, fc + 1.96 * sd * k


        def use_log(train: np.ndarray) -> bool:
            return bool(train.min() > 0.05 * train.mean())


        def _as_series(train: np.ndarray) -> pd.Series:
            idx = pd.date_range("2021-04-30", periods=len(train), freq="ME")
            vals = np.log1p(np.maximum(train, 0)) if use_log(train) else np.asarray(train, dtype=float)
            return pd.Series(vals, index=idx)


        def _back(train: np.ndarray):
            return (lambda v: np.expm1(np.asarray(v))) if use_log(train) else (lambda v: np.asarray(v))


        def ets(train: np.ndarray, h: int):
            y = _as_series(train)
            m = ETSModel(y, error="add", trend="add", damped_trend=True, seasonal="add", seasonal_periods=12).fit(disp=False)
            pr = m.get_prediction(start=len(y), end=len(y) + h - 1)
            f80 = pr.summary_frame(alpha=0.20)
            f95 = pr.summary_frame(alpha=0.05)
            back = _back(train)
            return (back(f80["mean"]), back(f80["pi_lower"]), back(f80["pi_upper"]),
                    back(f95["pi_lower"]), back(f95["pi_upper"]))


        def sarima(train: np.ndarray, h: int):
            y = _as_series(train)
            m = SARIMAX(y, order=(1, 0, 0), seasonal_order=(0, 1, 1, 12), trend="c").fit(disp=False)
            fc = m.get_forecast(h)
            c80 = np.asarray(fc.conf_int(alpha=0.20))
            c95 = np.asarray(fc.conf_int(alpha=0.05))
            back = _back(train)
            return (back(fc.predicted_mean), back(c80[:, 0]), back(c80[:, 1]), back(c95[:, 0]), back(c95[:, 1]))


        MODELS = {"Seasonal naive": seasonal_naive, "ETS(A,Ad,A)": ets, "SARIMA(1,0,0)(0,1,1)12": sarima}
```

Technique 2: customer segmentation (K-means clustering)

Question (BQ4): which groups of customers behave alike, and where should sales effort and credit control go?

Method (`03_analytics/clustering.py`):

- **Features** for each durable customer code at the 30 September 2026 cut-off:
  - recency (days since the last invoice);
  - frequency (invoices in the last 12 months);
  - monetary value (12-month net revenue);
  - basket size (average kg per line);
  - product breadth.
- Skewed features are `log1p`-transformed, then all features are standardised.
- **K-means** with k = 2…8 (25 initialisations each) is evaluated on:
  - silhouette (Rousseeuw, 1987);
  - Davies–Bouldin;
  - Calinski–Harabasz;
  - the within-cluster SSE elbow;
  - **stability**: the adjusted Rand index over 20 re-seeded runs.
- **Selection rule:** among stable solutions (ARI ≥ 0.95) with k ≥ 3, take the highest Calinski–Harabasz.

Table 32. Cluster-count evaluation

k	sse	silhouette	davies_bouldin	calinski_harabasz	stability_ari
2	728.10	0.63	0.42	154.25	1.00
3	375.17	0.65	0.52	260.53	1.00
4	276.62	0.35	0.91	262.60	0.98
5	204.34	0.41	0.80	286.26	1.00
6	179.65	0.40	0.83	265.81	0.92
7	156.10	0.39	0.89	259.70	0.92
8	137.51	0.39	0.90	256.07	0.91

Chosen k = 5; mean silhouette = 0.409; stability (adjusted Rand index vs 20 re-seeded runs): mean 1.000, min 1.000

Why not the silhouette maximum? k = 3 has the best silhouette (0.65) but merges **84 %** of customers into one segment, which is not actionable. k = 5 keeps a reasonable silhouette (0.41), the best Calinski–Harabasz (286) and perfect stability (ARI 1.00). This is a documented trade-off, not metric-shopping: the rule was fixed before the profiles were inspected.

segment	customers	recency_days	frequency	median_rev_12m_N$k	avg_line_kg	product_breadth	segment_rev_12m_N$m	silhouette	rev_share_pct
Bulk & export key accounts	20	24.00	30.00	6,638.00	18,530.56	3.00	272.80	0.53	58.60
High-frequency small-basket buyers	73	0.00	190.00	533.00	156.80	15.00	114.20	0.38	24.50
Wholesale regulars	34	4.00	48.00	878.00	2,143.02	19.00	64.60	0.40	13.90
Regular small-basket buyers	94	5.00	49.00	106.00	261.93	13.00	14.00	0.33	3.00
Lapsed / at-risk accounts	19	989.00	0.00	0.00	340.31	14.00	0.00	0.80	0.00

Channel mix (customers, current channel):

segment	Export	Informal market	Institutional	Miller/Processor	Retail	Wholesale
Bulk & export key accounts	12	0	0	8	0	0
High-frequency small-basket buyers	0	37	6	0	29	1
Lapsed / at-risk accounts	0	9	0	0	8	2
Regular small-basket buyers	0	44	8	0	42	0
Wholesale regulars	0	0	0	0	0	34

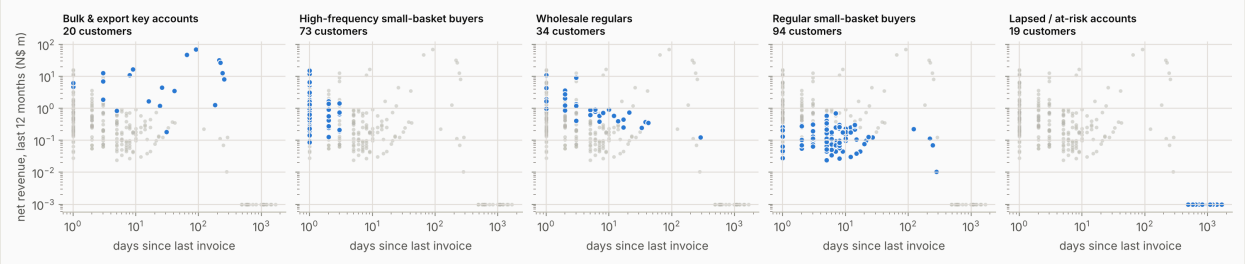


Figure 8. Customer segments: distribution of the standardised features per segment (K-means, k = 5)

Findings and actions.

- **Bulk and export key accounts** (20 customers, 58.6 % of 12-month revenue): 12 export buyers and 8 millers. Manage them as key accounts and watch concentration (O3).
- **High-frequency small-basket buyers** (73, 24.5 %) and **regular small-basket buyers** (94, 3.0 %) are the informal and retail long tail. Serve them at low cost-to-serve through route sales.
- **Wholesale regulars** (34, 13.9 %): candidates for volume agreements.
- **Lapsed / at-risk accounts** (19): last purchase about 989 days ago. A win-back list for the sales team, published on page 3.

The segments were **not given channel as an input**, yet they align with channels (table above). That is evidence that the behavioural features capture real commercial roles.

Assumptions:

- Euclidean distance on standardised log-features reflects business similarity.
- Clusters are roughly convex.

Limitations:

- K-means is sensitive to scaling and outliers, which the log transform mitigates.
- Segment names are interpretive.
- This is a static snapshot. It should be refreshed quarterly and tracked as an SCD attribute.

File: `03_analytics/clustering.py` (lines 76–100)

```
        def main():
            setup_style()
            df = load()
            X = df[FEATURES].copy()
            for c in LOGS:
                X[c] = np.log1p(X[c])
            X["recency_days"] = np.log1p(X["recency_days"])
            Z = StandardScaler().fit_transform(X)

            evals = []
            for k in range(2, 9):
                km = KMeans(n_clusters=k, n_init=25, random_state=42).fit(Z)
                evals.append(dict(k=k, sse=km.inertia_, silhouette=silhouette_score(Z, km.labels_),
                                  davies_bouldin=davies_bouldin_score(Z, km.labels_),
                                  calinski_harabasz=calinski_harabasz_score(Z, km.labels_)))
            ev = pd.DataFrame(evals)
            stab = {}
            for k in ev.k:
                ref = KMeans(n_clusters=k, n_init=50, random_state=42).fit(Z).labels_
                stab[k] = np.mean([adjusted_rand_score(ref, KMeans(n_clusters=k, n_init=10, random_state=s).fit(Z).labels_)
                                   for s in range(20)])
            ev["stability_ari"] = ev.k.map(stab)
            cand = ev[(ev.k >= 3) & (ev.stability_ari >= 0.95)]
            k_best = int(cand.sort_values("calinski_harabasz", ascending=False).iloc[0].k)
            km = KMeans(n_clusters=k_best, n_init=50, random_state=42).fit(Z)
```

Technique 3: what drives yield (regression and key influencers)

Question (BQ2): how much of the yield gap is weather, how much is irrigation, and how much is management? What would irrigating a dryland block be worth?

Method (`03_analytics/yield_drivers.py`):

- **Unit:** 1,163 completed field-block crop cycles.
- **Response:** ln(yield t/ha).
- **Explanatory variables**, assembled by **drill-across** on conformed keys:
  - irrigation method of the block in this cycle (planting register; §3.3);
  - dryland × ln(season rain ratio), with November–March rainfall at the farm divided by the farm's average (NASA POWER);
  - heat days with Tmax ≥ 35 °C in the flowering window;
  - ln(fertiliser index) from the cost ledger's input quantities;
  - crop and farm fixed effects.
- **Models:**

1. **OLS** with HC3 robust standard errors, for interpretable elasticities;
2. **random forest** with permutation importance, a non-linear "key influencer" ranking (Breiman, 2001).

- **Validation:** 5-fold random cross-validation, plus **leave-one-season-out** cross-validation, which tests generalisation to an unseen weather year.

n = 1163 completed field-block cycles; R2 = 0.989 (adj. 0.989) - dominated by between-crop differences; within-crop R2 = 0.861. Robust (HC3) standard errors; crop and farm fixed effects not shown.

term	coef	std_err	p_value	effect_pct	generator_truth
Irrigation: Centre pivot	1.0305	0.0723	0.0000	180.2550	
Irrigation: Drip	0.9633	0.0362	0.0000	162.0295	
Irrigation: Drip + shade-net	0.6266	0.0180	0.0000	87.1299	
Irrigation: Flood (canal)	0.8183	0.0379	0.0000	126.6688	
Dryland x ln(rain ratio)	0.7332	0.0361	0.0000	108.1728	~0.77 elasticity (f = 0.15 + 0.85 r^0.9)
Heat days (Tmax >= 35 C, flowering)	-0.0033	0.0006	0.0000	-0.3328	-0.2 % to -0.7 % per day by crop
ln(fertiliser index)	0.1239	0.0242	0.0000	13.1856	+0.2 to +0.3 elasticity

Crop-specific irrigation contrast (mean t/ha, all farms):

crop_name	irrigated_t_ha	dryland_t_ha	ratio
Watermelon	28.79	11.17	2.58
White maize	8.60	3.67	2.35

Table 33. Out-of-sample validation

scheme	model	R2_ln	RMSE_ln	MAE_t_ha	MAPE_pct
5-fold random CV	OLS fixed effects	0.989	0.116	2.188	8.579
5-fold random CV	Random forest	0.991	0.101	1.595	6.272
Leave-one-season-out	OLS fixed effects	0.987	0.125	2.374	9.171
Leave-one-season-out	Random forest	0.977	0.166	2.439	10.938

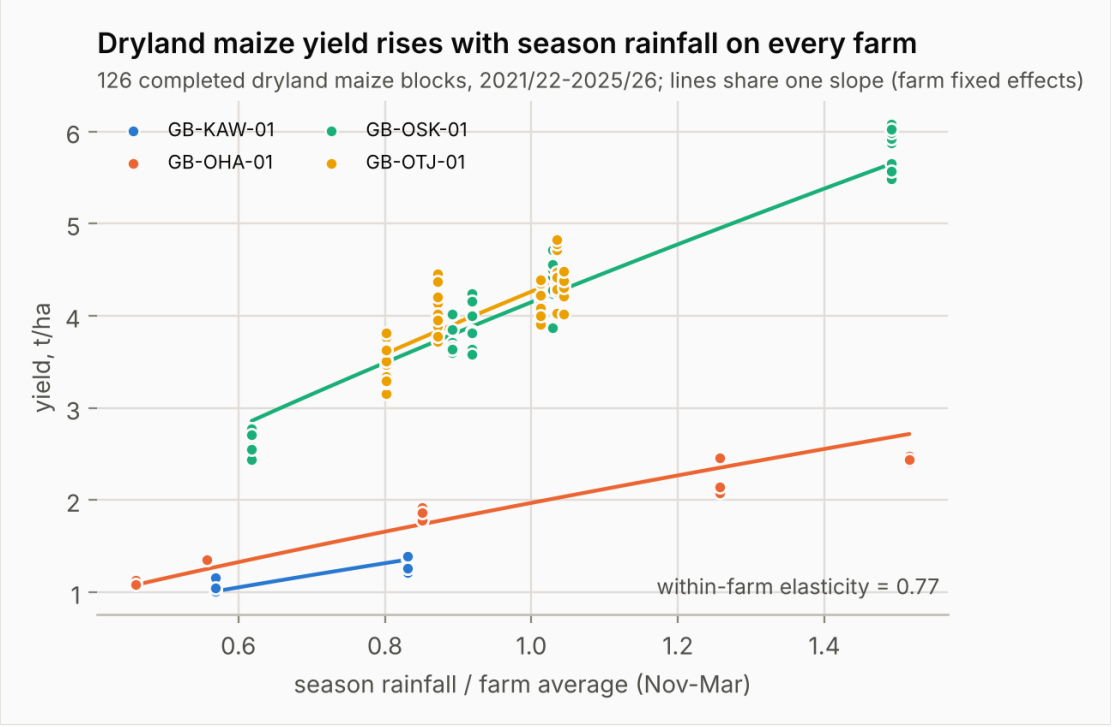


Figure 9. Within-farm rainfall response of dryland maize (one colour per farm, common fixed-effects slope)

Findings.

1. **Rainfall elasticity of dryland yield.**
   - The full model across all dryland crops gives **0.73** ($p < 0.001$).
   - Within farms, dryland maize gives **0.77**.
   - The generator's true elasticity is about 0.77, so **the method recovers the known truth**. This validates the whole pipeline: ETL, conformed keys and drill-across.

- Business meaning: a season with 30 % less rain than normal costs about 23 % of dryland yield. The 2023/24 drought (rain ratio 0.71) brought dryland maize down to 2.97 t/ha.

2. **Irrigation more than doubles yield** relative to dryland, holding crop and farm constant:
   - centre pivot: +180 %;
   - drip: +162 %;
   - flood: +127 %.

   For maize the raw contrast is 8.60 vs 3.67 t/ha (×2.35). An indicative business case for the next pivot:
   - At the NAB white-maize floor price of N$ 6,680 per tonne (The Namibian, n.d.), the 4.9 t/ha gap is worth about **N$ 33,000 per hectare per season**.
   - Irrigated production costs about N$ 17,000/ha more in direct cost: N$ 3,880/t × 8.60 t vs N$ 4,430/t × 3.67 t, averaged over 2022/23–2024/25.
   - The **margin gain is therefore about N$ 16,000 per hectare per season**, before the pivot's capital cost.
3. **Heat** costs 0.33 % of yield per day ≥ 35 °C during flowering, inside the true range (−0.2 % to −0.7 %).
4. **Fertiliser** has a positive effect (0.12), but it is **attenuated** against the true 0.2–0.3, for two reasons:
   - The ledger allocates fertiliser to blocks by area share, so the rate varies only between cycles.
   - It measures the true rate with error (classical attenuation bias).

   We report the attenuation rather than hide it.
5. **The random forest** ranks irrigation and crop first, then rainfall (permutation importance: drop in R^2). Leave-one-season-out cross-validation shows **OLS generalises better to an unseen season** (RMSE 0.125 vs 0.166 on the log scale), so OLS is the model for decisions. The forest is used only as a cross-check of the ranking.

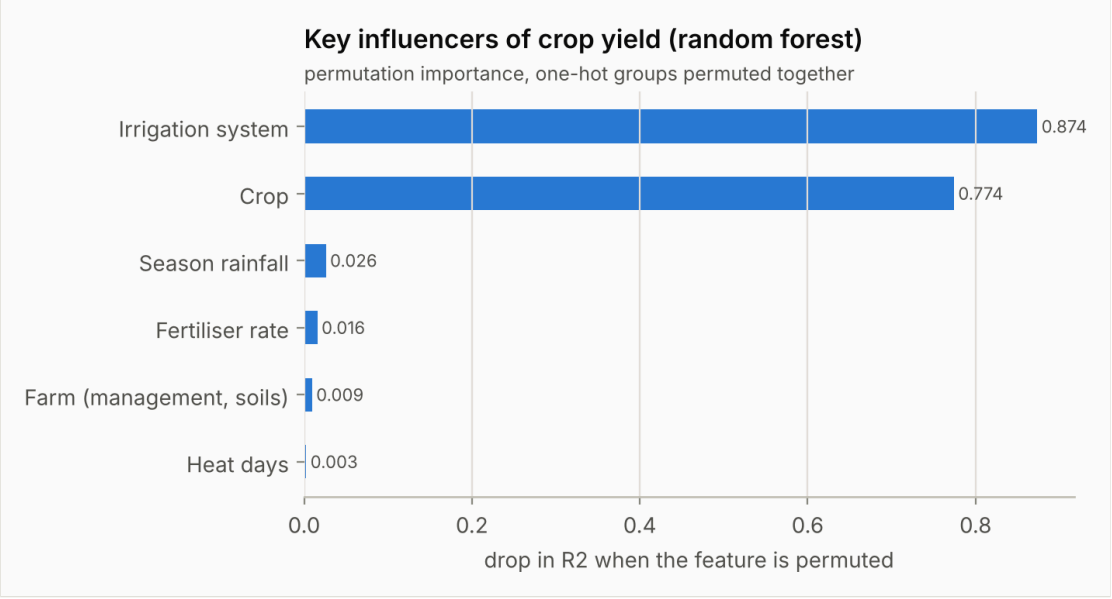


Figure 10. Random-forest permutation importance of yield drivers (one-hot groups permuted together)

Assumptions:

- Log-linear effects.
- Additive farm and crop effects.
- Rainfall at the farm's NASA POWER grid point represents the field.

Limitations:

- The data is observational: farms chose where to irrigate, so the irrigation effect is an association adjusted for crop and farm, not a randomised estimate.
- NASA POWER's grid resolution is about 0.5°.
- The data is synthetic. Recovering the truth validates the method, not real-world magnitudes. The real NASA POWER data (assumption A2) would test the latter.

File: `03_analytics/yield_drivers.py` (lines 80–87)

```
            df["dryland"] = (~df.is_irrigated).astype(int)
            df["dry_x_ln_rain"] = df.dryland * np.log(df.rain_ratio)
            df["irrigation"] = pd.Categorical(df.irrigation_type, categories=["Dryland", "Centre pivot", "Drip", "Drip + shade-net", "Flood
(canal)"])
            df["harvest_season"] = [d.year if d.month >= 10 else d.year - 1 for d in pd.to_datetime(df.first_harvest)]

            # ---------------- (1) OLS with crop and farm fixed effects, HC3 robust errors
            f = "ln_yield ~ C(crop_code) + C(farm_code) + C(irrigation, Treatment('Dryland')) + dry_x_ln_rain + heat_days + ln_fert_index"
            ols = smf.ols(f, data=df).fit(cov_type="HC3")
```

Technique 4: anomaly detection in farm logs (Isolation Forest vs rules)

Question: can an unsupervised model find keying errors in harvest logs that nobody wrote a rule for?

Method (`03_analytics/anomaly_detection.py`):

- **Isolation Forest** (Liu et al., 2008): 500 trees, contamination 0.15 %.
- **Data:** every accepted harvest record with its **original keyed** quantity, i.e. before the HV06 corrections.
- **Five features:**
  - ln(kg);
  - ln(kg per planted ha);
  - ratio to the median of the four neighbouring records of the block;
  - ratio to the cycle median;
  - block yield vs the leave-one-out median of the sister blocks.
- **Evaluation** against the generator's ground truth: 8 errors among 7,368 records, a base rate of 0.11 %.

Ground truth: 8 injected x10/x1000 keying errors among 7,368 accepted harvest records (base rate 0.109 %). Isolation Forest PR-AUC (average precision) = 0.484; 5 of the 8 errors are in the 25 highest-scored records.

detector	flagged	true_pos	precision	recall	f1
SQL rule HV06 (corrected)	7	7	1.000	0.875	0.933
Isolation Forest (all flags)	12	4	0.333	0.500	0.400
Isolation Forest + upward-deviation filter	4	4	1.000	0.500	0.667

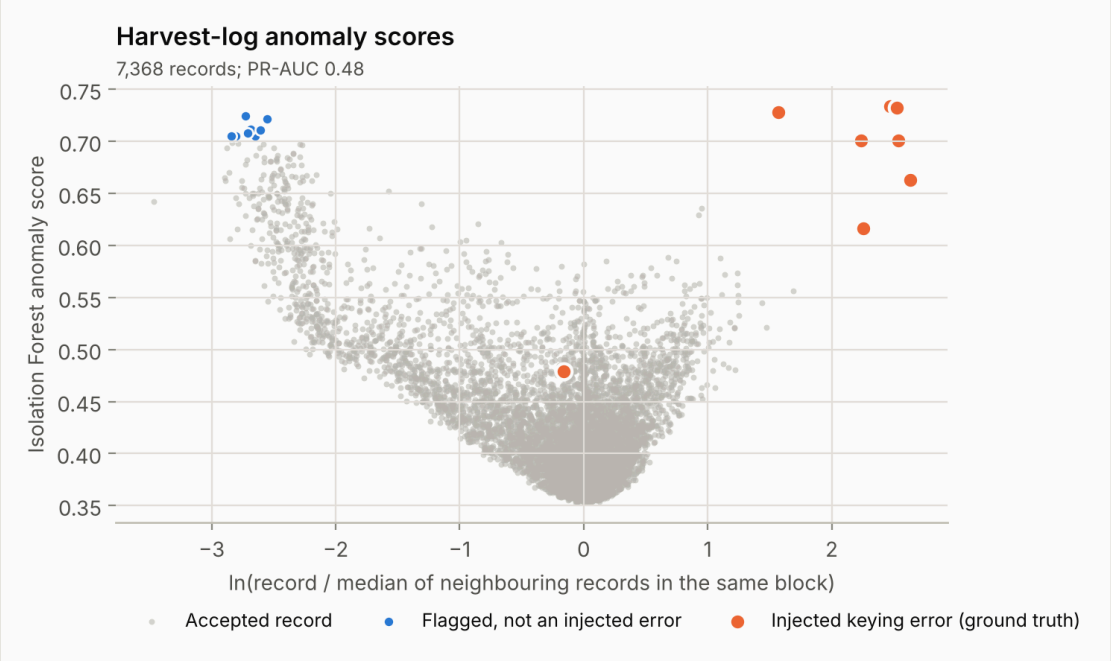


Figure 11. Isolation-Forest anomaly scores of harvest records; injected keying errors highlighted

Findings.

- The **domain rule wins**: HV06 has precision 1.00 and recall 0.875. The Isolation Forest, used blind, flags 12 records with precision 0.33.
- Restricting the Forest to *upward* deviations, which is the direction a ×10 error moves, gives precision 1.00 at recall 0.50.
- The Forest's real value is **triage**. 5 of the 8 errors sit in its top 25 records, i.e. 0.34 % of the log. The top 25 are written to `dw.harvest_anomaly` for the Farm Data Steward. A weekly review of 25 records would surface new error types that no rule anticipated.

Limitations:

- Only 8 positives: each one moves recall by 12.5 points.
- Results depend on the contamination setting.
- Legitimate extremes, such as large grape picks, look anomalous without domain features.

3.6 Interactive dashboard design

The report `GreenBasket_Agri.Report` has **four generated pages** on a 1280 × 720 canvas, plus a drill-through page and a tooltip page that are added in Desktop (`04_powerbi/BUILD_GUIDE.md` §5). Each page answers specific business questions for a specific audience.

Table 34. Report pages, audiences and the questions they answer

Page	Primary audience	Answers	Design pattern
1 Executive Summary	Executive committee, CFO (non-technical)	BQ1: are we on track, and where does revenue come from?	KPI band → trend → breakdowns → farm scorecard
2 Production & Yield	Zone and farm managers, agronomist	BQ2, BQ3: which farms and seasons miss plan, and why?	Matrix with drill-down + diagnostic charts
3 Advanced Analytics	Analysts, sales management (technical and semi-technical)	BQ4: outlook, segments, yield drivers	Model outputs with accuracy shown next to them
4 Operations & Data Quality	Inventory controller, data stewards, BI team (technical)	BQ5: is stock under control, and can we trust the numbers?	Operational KPIs + DQ / RLS check card
Farm Detail (drill-through)	Zone managers	Everything about one farm	Filtered copy of the key visuals
Revenue tip (tooltip)	All	Context on hover without leaving the page	Small multi-card page

Page designs

The four images below are **design mock-ups**. They are rendered from the PostgreSQL warehouse by `06_report/build_mockups.py` with the same layout, titles and measure logic as the PBIR pages. They show what the pages contain with real numbers. They are not Power BI screenshots: those are captured in Desktop after the refresh (`BUILD_GUIDE.md` §9) and replace these files with the same names.

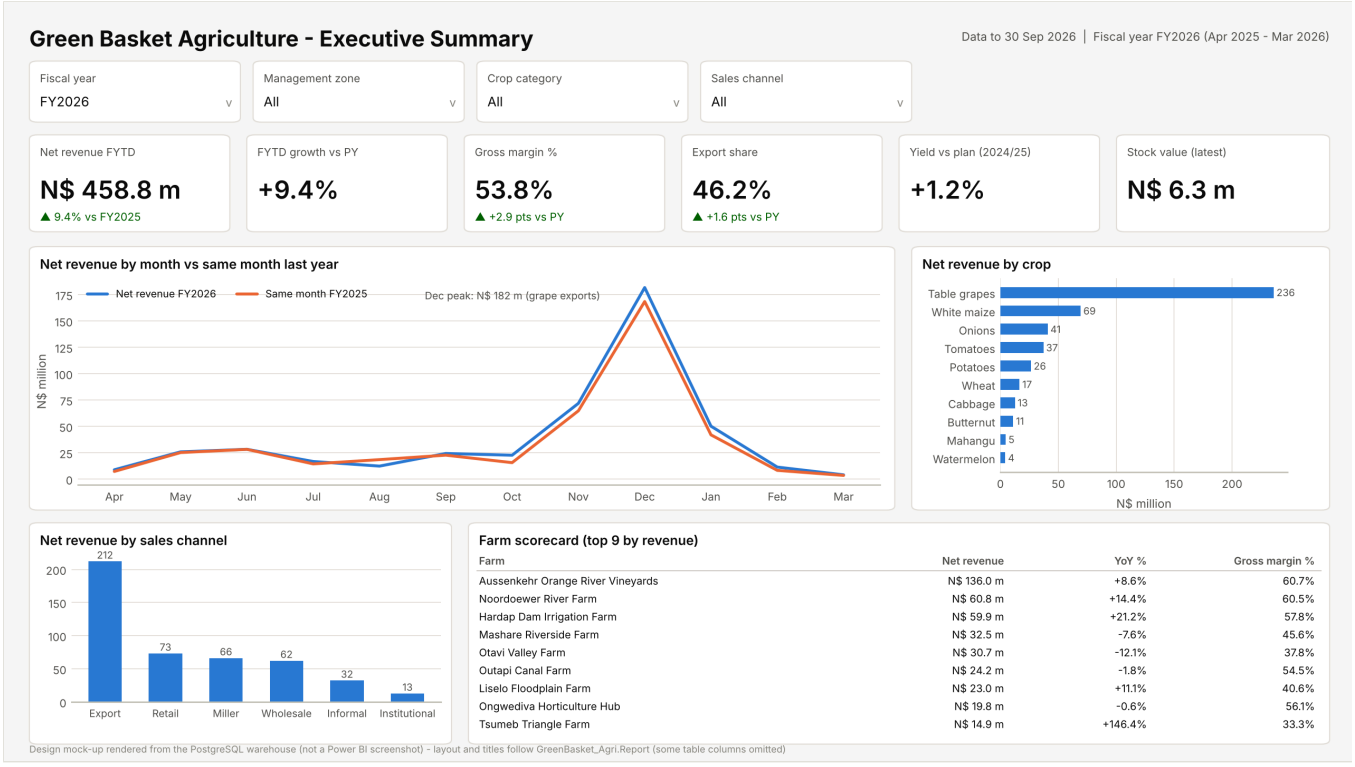


Figure 12. Page 1 - Executive Summary (FY2026 selected) - design mock-up from warehouse data

```
WIREFRAME - PAGE 1  Executive Summary                                  1280 x 720
+-------------------------------------------------------------------------------+
| Green Basket Agriculture - Executive Summary            [Data to 30 Sep 2026]  |
| [Fiscal year v] [Management zone v] [Crop category v] [Sales channel v]  <- synced |
| +---------+ +---------+ +---------+ +---------+ +---------+ +-----------+     |
| | Net rev | | FYTD    | | Gross   | | Export  | | Yield   | | Stock     |  KPI band |
| | FYTD    | | YoY %   | | margin% | | share % | | vs plan | | value     |  (cards)  |
| +---------+ +---------+ +---------+ +---------+ +---------+ +-----------+     |
| +-------------------------------------------+ +---------------------------+ |
| | LINE  Net revenue by month vs same month LY     | | BAR  Net revenue by crop   | |
| |  2 series + legend; FY>FQ>Month drill           | |  sorted desc; Category >   | |
| |  tooltip page "Revenue tip" on hover            | |  Crop > Product drill-down | |
| +-------------------------------------------+ +---------------------------+ |
| +-------------------------+ +-------------------------------------------------+ |
| | COLUMN  Revenue by      | | TABLE  Farm scorecard: revenue, YoY % (icons),  | |
| | sales channel           | | GM %, yield  -> right-click: drill through to farm | |
| +-------------------------+ +-------------------------------------------------+ |
+-------------------------------------------------------------------------------+
```

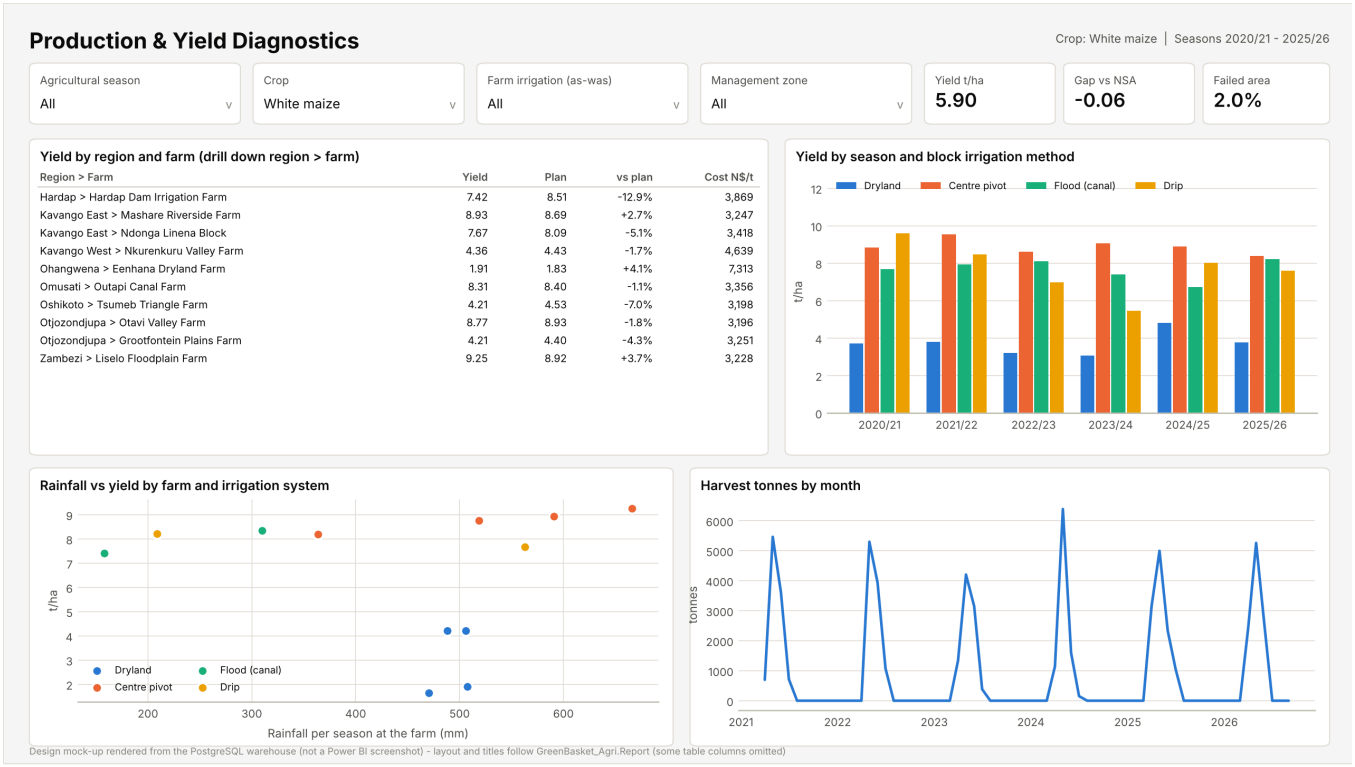


Figure 13. Page 2 - Production & Yield Diagnostics (crop = white maize) - design mock-up

```
WIREFRAME - PAGE 2  Production & Yield Diagnostics
+-------------------------------------------------------------------------------------+
| Production & Yield Diagnostics                                                      |
| [Season v] [Crop v] [Farm irrigation (as-was) v] [Zone v]  |Yield t/ha|Gap vs NSA|Failed|
| +-----------------------------------+ +-----------------------------------+ |
| | MATRIX  Region > Farm (expand / collapse)  | | CLUSTERED COLUMN  Yield by       | |
| |  Yield | Plan | vs plan (icons) | NSA |     | | season x block irrigation method | |
| |  Cost N$/t (data bars)                    | | legend; Dryland vs irrigated gap | |
| +-----------------------------------+ +-----------------------------------+ |
| +-----------------------------------+ +-----------------------------------+ |
| | SCATTER  Rainfall per season vs      | | LINE  Harvest tonnes by month          | |
| | yield, one point per farm, colour =  | |  (shows harvest windows; drill to day)  | |
| | farm irrigation system               | |                                         | |
| +-----------------------------------+ +-----------------------------------+ |
+-------------------------------------------------------------------------------------+
```

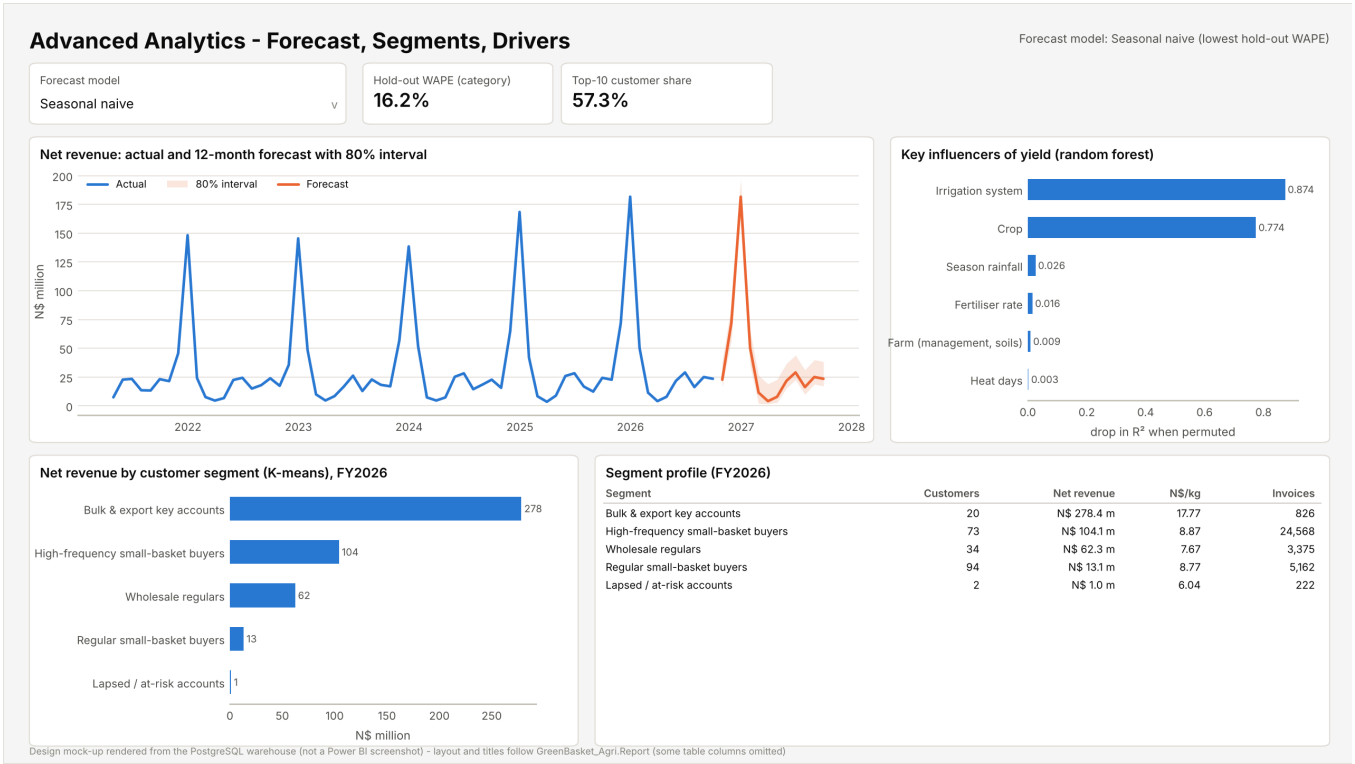


Figure 14. Page 3 - Advanced Analytics: forecast, key influencers and customer segments - design mock-up

```
WIREFRAME - PAGE 3  Advanced Analytics
+-------------------------------------------------------------------------------+
| Advanced Analytics - Forecast, Segments, Drivers                              |
| [Forecast model: (o) Seasonal naive ( ) ETS ( ) SARIMA]  |WAPE (category)| |Top-10 %| |
| +-----------------------------------------------+ +---------------------------+ |
| | LINE  Actual + 12-month forecast + 80 % band  | | BAR  Key influencers of   | |
| |  (model slicer switches series; band = 2 lines)| | yield (RF importance)     | |
| +-----------------------------------------------+ +---------------------------+ |
| +--------------------------+ +-----------------------------------------------+ |
| | BAR  Revenue by segment  | | TABLE  Segment profile: customers, revenue,   | |
| | (K-means), sorted        | | N$/kg, invoices; click a segment = cross-filter | |
| +--------------------------+ +-----------------------------------------------+ |
+-------------------------------------------------------------------------------+
```

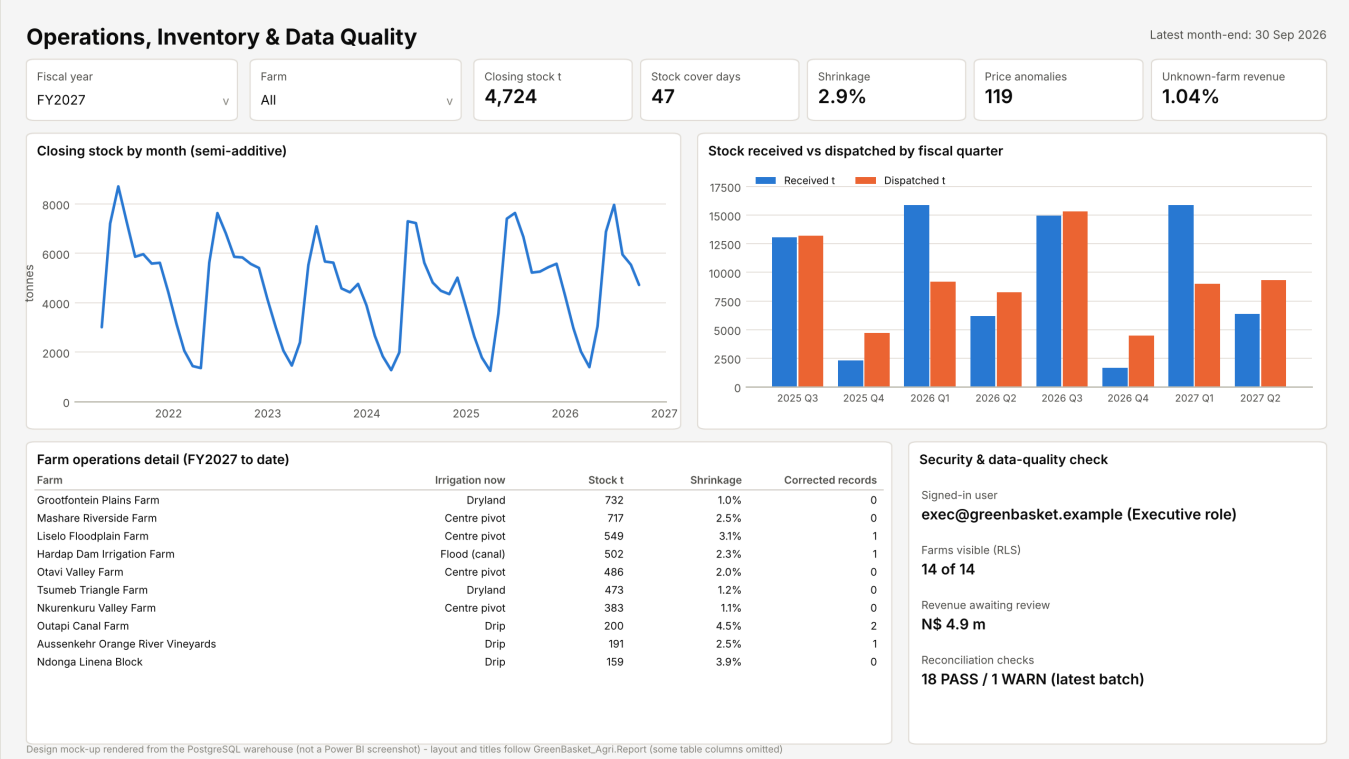


Figure 15. Page 4 - Operations, Inventory & Data Quality (latest month-end) - design mock-up

```
WIREFRAME - PAGE 4  Operations, Inventory & Data Quality
+-------------------------------------------------------------------------------+
| Operations, Inventory & Data Quality                                          |
| [Fiscal year v] [Farm v] |Closing t| |Cover days| |Shrinkage| |Price anomalies| |Unknown %| |
| +--------------------------------------+ +-----------------------------------+ |
| | LINE  Closing stock by month         | | COLUMN  Received vs dispatched by | | |
| |  (semi-additive: last snapshot)      | | fiscal quarter (2 series + legend) | |
| +--------------------------------------+ +-----------------------------------+ |
| +-----------------------------------------------+ +---------------------------+ |
| | TABLE  Farm operations: irrigation now, stock,  | | MULTI-ROW CARD  Security &  | |
| | cover, shrinkage, corrected records (HV06)      | | DQ: signed-in user, farms   | |
| | |                                             | | visible (RLS), N$ awaiting  | |
| | |                                             | | review, reconciliation      | |
| +-----------------------------------------------+ +---------------------------+ |
+-------------------------------------------------------------------------------+
```

```
WIREFRAME - DRILL-THROUGH  Farm Detail          (target of right-click on any farm)
+-------------------------------------------------------------------------------+
| < Back   Farm Detail: <farm name>  (zone, region, irrigation as-is, SCD version)  |
| |Net revenue| |Yield t/ha| |Cost N$/t| |GM %|   [Fiscal year v] (kept from source) |
| +-----------------------------------------+ +-------------------------------+ |
| | LINE  Net revenue by month (this farm)  | | TABLE  Crop cycles: crop,     | |
| | |                                       | | season, block, yield vs plan, | |
| +-----------------------------------------+ | irrigation method, status     | |
|                                               +-------------------------------+ |
+-------------------------------------------------------------------------------+
```

Purposeful chart choice and visual hierarchy

Table 35. Why each chart type was chosen

Question shape	Chart used	Why not the alternatives
Single headline numbers (revenue, margin, share)	KPI **cards** with the comparison underneath (▲/▼ + text)	A gauge wastes space and hides the comparison. A card answers in under 5 seconds.
Change over time with a comparison	**Line** with 2 series (actual vs last year) and a legend	Columns for 12 × 2 bars clutter. One axis only: no dual axes.

Question shape	Chart used	Why not the alternatives
Ranking of categories (crops, segments, drivers)	**Horizontal bar**, sorted descending, with direct value labels	Long crop and segment names stay readable. A pie cannot rank 10 crops.
Part of a whole over few categories (channels)	**Column**, sorted	A donut makes six close shares hard to compare.
Two measures across entities (rain vs yield)	**Scatter**, colour = irrigation system	The relationship *is* the message, and two axes of different units belong on a scatter, not a dual-axis line.
Hierarchical detail with several measures	**Matrix** (Region › Farm) with icons and data bars	Gives managers numbers they can quote. Drill-down keeps it compact.
Uncertainty of a forecast	**Line + interval band**	Point forecasts alone overstate certainty.

Visual hierarchy. Every page reads in a Z-pattern:

1. Title and data freshness.
2. Slicers in **one row** at the top.
3. The **KPI band**.
4. The main trend, top left and largest.
5. Breakdowns.
6. Detail tables at the bottom.

Above-the-fold space goes to what executives look at first. Detail is one click away (drill-down or drill-through), not crammed onto the page.

Consistent colour and typography.

- One custom theme (`theme/GreenBasket_Accessible.json`) applies everywhere.
- Text uses Segoe UI: titles 12 pt semibold, labels 10 pt, card callouts 22 pt.
- Colour is assigned **by role, never by rank**:
  - a single-series chart uses one blue (#2A78D6);
  - multi-series charts take the fixed categorical order (blue, orange, green, amber, …), so “Dryland” is always blue and “Centre pivot” always orange on every page;
  - status colours (good #006300, bad #D03B3B) appear only with an icon or text, never as a series colour.

Accessibility

Table 36. Accessibility measures and how they were verified

Requirement	Implementation	Verification
Colour-blind safe palette	8-colour categorical palette validated for lightness band, chroma, **CVD separation** (worst adjacent ΔE 9.1 under protanopia; ≥ 8 is the target) and normal-vision separation (ΔE ≥ 19.6)	`validate_palette.js` from the team’s dataviz kit: **all checks pass**
Colour never the only cue	Legends on every multi-series chart; direct data labels on bars; ▲/▼ icons with text on KPIs and status	Design review against the mock-ups
Contrast	Primary text #0B0B0B on white is **19.7:1**; secondary text #52514E is **7.9:1**; good #006300 is 7.5:1; bad #D03B3B is 4.8:1, always paired with an icon. Three palette hues (green, amber, pink) fall below 3:1 against white, so those series always carry labels or a legend, and *Show as table* is available	WCAG 2.2 contrast formula (W3C, 2023)
Screen readers	**Alt text on every data visual**, generated in the PBIR (e.g. “Line chart of monthly net revenue compared with the same month of the previous year…”); text boxes are read as their own text	`build_pbip.py` **fails the build** if a data visual has no alt text
Keyboard navigation	Tab order is written into the PBIR in reading order: title → slicers → cards → charts → tables. Every chart supports *Show as a table* (Alt+Shift+F11)	Generated by `build_pbip.py`; confirm in *Selection pane › Tab order* (`BUILD_GUIDE.md` §5.8)
Readable sizes	No text under 9 pt; 1280 × 720 *Fit to page*; descriptive titles instead of abbreviations	Mock-up review

Interactivity, used on purpose

- **Slicers.** Fiscal year, zone, crop category and channel are **synced** across pages 1, 2 and 4, so a manager keeps the context when changing page. They are dropdowns, to save space, except the forecast model, which is a radio list because all three options should be visible.
- **Cross-filtering and cross-highlighting.**
  - Clicking a crop bar filters the trend and the scorecard.
  - KPI cards are set **not** to react to cross-highlighting (*Edit interactions*), so the headline stays stable while users explore.
- **Drill-down.**
  - Fiscal Calendar on the trend: FY › FQ › Month.
  - Category › Crop › Product on the crop chart.
  - Region › Farm in the yield matrix.
- **Drill-through.** *Farm Detail* receives the farm and keeps all filters, so a zone manager can go from “this farm is −7 % vs plan” to its crop cycles in one right-click.
- **Report-page tooltip.** *Revenue tip* shows YoY %, gross margin and export share when hovering over a month.

- **Bookmarks.**
  - *Executive view*: resets slicers to the current fiscal year.
  - *Analyst view*: reveals the Metric Selector visual.
  - Two buttons switch between them.
- **Field parameter.** *Metric Selector* swaps the trend measure among six KPIs. This is self-service without building visuals.
- **Dynamic titles.** Measures in folder *0 Labels* (e.g. `Title Revenue Trend`) put the selected fiscal year in the chart title, so screenshots and exports are self-describing.
- **Security-aware content.** RLS filters every visual automatically. Page 4's check card shows who is signed in and how many farms they can see.

Design rationale: serving technical and non-technical users

Non-technical users get:

- plain-language titles that state the question ("Net revenue by month vs same month last year");
- KPI cards with explicit comparisons;
- no jargon on pages 1 and 2;
- the **app** experience with RLS, so a zone manager sees only their zone and nothing to configure.

Technical users get:

- the model accuracy (WAPE, model choice) next to the forecast, not hidden;
- the DQ and reconciliation status on page 4, linked to the audit tables;
- a model whose display folders, descriptions on every measure and hidden keys make building their own analyses safe.

The same semantic model serves both, so **every number is identical** wherever it appears. That is the "single trusted view" in the problem statement.

3.7 Performance evaluation and optimisation

Baseline and measurement method

Performance was measured at the two layers where the solution spends time.

1. Warehouse queries that feed refreshes and drill-throughs (`03_analytics/performance_benchmark.py`):

- Six representative queries ran on PostgreSQL 16 against the 302,903-row `fact_sales`.
- Each was run **7 times**, and we report the **median**. `EXPLAIN (ANALYZE, BUFFERS)` recorded the access path.
- The **baseline** is the warehouse straight after loading, with only primary keys and unique constraints.
- The **optimised** state is after `02_sql/13_optimisation.sql`, which the pipeline now runs after every full load.

2. Model size and refresh volume.

- The VertiPaq engine is not available outside Power BI. The import was therefore reproduced in **DuckDB**, a columnar engine with dictionary and run-length compression comparable in kind to VertiPaq.
- Two variants were compared:
  - a **naive import**: every column of every `dw` table;
  - the **optimised import**: the `mart` views the model actually uses.
- We report column counts, cardinality, compressed size, bytes streamed from PostgreSQL, and rows per refresh.

3. Power BI Desktop (to be captured by the team). Performance Analyzer visual timings, DAX Studio server timings and VertiPaq Analyzer model size can only be produced in Desktop. The protocol is fixed in `04_powerbi/BUILD_GUIDE.md` §7:

- three refreshes per page, using the median;
- cold-cache server timings for the three heaviest measures;
- a `.vpax` export.

The results go into the table at the end of this section. **The measured numbers below come from the warehouse and the columnar proxy only. No Power BI timing has been estimated or invented.**

Optimisation techniques with before-and-after evidence

Table 37. Warehouse query latency before and after optimisation (median of 7 runs, ms)

query	baseline_ms	optimised_ms	speedup_x	baseline_access	optimised_access
Q1 Monthly revenue by crop category, FY2026	31.82	27.34	1.16	Parallel Seq Scan on fact_sales	Bitmap Heap Scan on fact_sales
Q2 Revenue by zone x channel, one month (slicer)	18.45	4.75	3.88	Parallel Seq Scan on fact_sales	Bitmap Heap Scan on fact_sales
Q3 Top-10 customers, last 90 days	20.79	6.43	3.23	Parallel Seq Scan on fact_sales	Index Only Scan on fact_sales
Q4 Customer drill-through (one customer, one quarter)	15.74	0.04	414.11	Parallel Seq Scan on fact_sales	Index Only Scan on fact_sales
Q5 Incremental-refresh partition (last 10 days)	14.71	1.18	12.44	Parallel Seq Scan on fact_sales	Bitmap Heap Scan on fact_sales
Q6 Monthly revenue by channel (aggregation table vs detail)	99.29	5.95	16.68	Parallel Seq Scan on fact_sales	Seq Scan on agg_sales_monthly

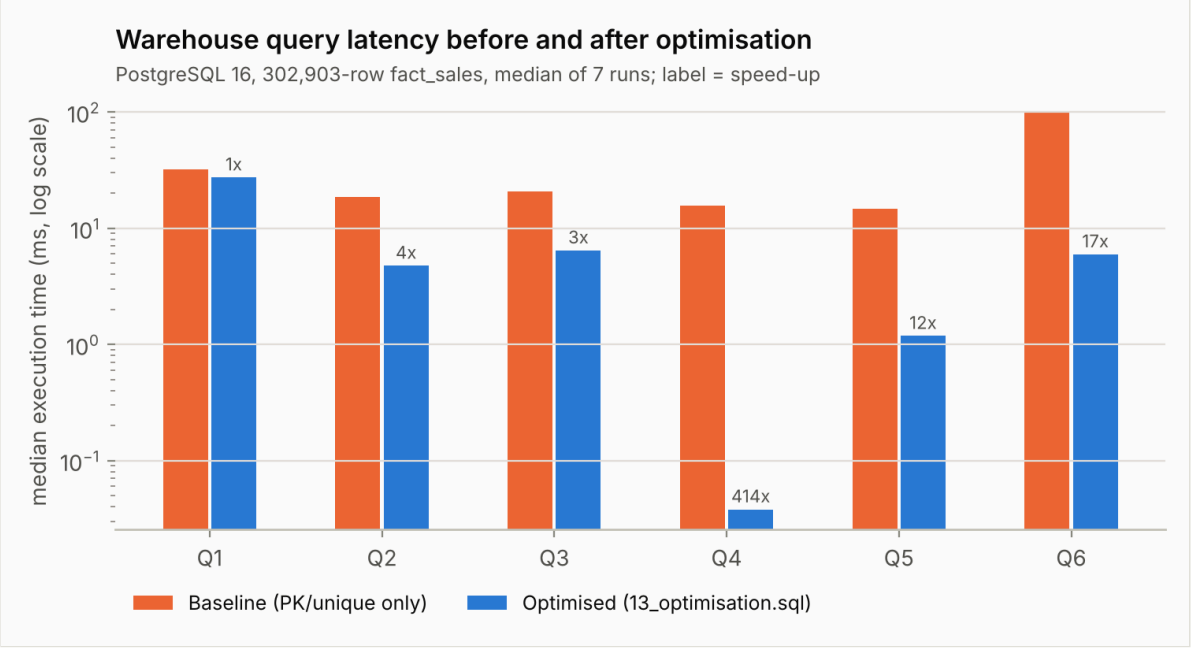


Figure 16. Query latency before and after optimisation (log scale; label = speed-up)

Technique 1: warehouse indexing (B-tree, BRIN and covering indexes).

- B-tree indexes on the fact foreign keys turn slicer-driven queries into bitmap scans:
  - Q2, one month × zone × channel: **≈ 3.9× faster**;
  - Q3, top-10 customers over 90 days: **≈ 3.2×**.
- A **BRIN** index on `invoice_date_key` with 16 pages per range costs **24 kB**, against 2.1 MB for a B-tree on a key column, because invoices arrive in date order. Q5, the incremental-refresh partition, fell from 14.7 to 1.2 ms: **≈ 12×**.
- The **covering index** `(customer_key, invoice_date_key) INCLUDE (net_revenue_nad, quantity_kg)` lets the drill-through query for one customer and one quarter run as an **index-only scan**: 15.7 → 0.04 ms (≈ 400×).

File: `02_sql/13_optimisation.sql` (lines 10–25)

```
        CREATE INDEX IF NOT EXISTS ix_fact_sales_customer  ON dw.fact_sales (customer_key);
        CREATE INDEX IF NOT EXISTS ix_fact_sales_product   ON dw.fact_sales (product_key);
        CREATE INDEX IF NOT EXISTS ix_fact_sales_farm      ON dw.fact_sales (farm_key);
        -- 2. BRIN on the date key: rows are loaded in date order, so a block-range index is tiny and
        --    serves date-range filters (incl. Power BI incremental-refresh partitions) almost as well as a B-tree
        CREATE INDEX IF NOT EXISTS brin_fact_sales_invoice_date ON dw.fact_sales USING brin (invoice_date_key) WITH (pages_per_range = 16);
        -- 3. covering index for customer drill-through (index-only scans)
        CREATE INDEX IF NOT EXISTS ix_fact_sales_customer_date ON dw.fact_sales (customer_key, invoice_date_key) INCLUDE (net_revenue_nad,
quantity_kg);
        CREATE INDEX IF NOT EXISTS ix_fact_harvest_date        ON dw.fact_harvest (harvest_date_key);
        CREATE INDEX IF NOT EXISTS ix_fact_harvest_farm_crop   ON dw.fact_harvest (farm_key, crop_key);
        CREATE INDEX IF NOT EXISTS ix_fact_cost_date           ON dw.fact_production_cost (posting_date_key);
        CREATE INDEX IF NOT EXISTS ix_fact_cost_farm_crop      ON dw.fact_production_cost (farm_key, crop_key);
        CREATE INDEX IF NOT EXISTS ix_fact_cycle_farm_crop     ON dw.fact_crop_cycle (farm_key, crop_key);
        -- 4. SCD2 range lookups used by every fact load
        CREATE INDEX IF NOT EXISTS ix_dim_customer_code_range ON dw.dim_customer (customer_code, scd_effective_from, scd_effective_to);
        CREATE INDEX IF NOT EXISTS ix_dim_farm_code_range     ON dw.dim_farm (farm_code, scd_effective_from, scd_effective_to);
```

Technique 2: aggregation table. The materialized view `mart.agg_sales_monthly` pre-aggregates revenue by month, product, customer channel and farm. Q6, monthly revenue by channel, drops from 99.3 ms to 6.0 ms, **≈ 17× faster**. The same table is the ready-made Import aggregation for a composite model once detail volumes grow (§3.4). It is refreshed by `REFRESH MATERIALIZED VIEW` after each load.

File: `02_sql/13_optimisation.sql` (lines 28–37)

```
        CREATE MATERIALIZED VIEW mart.agg_sales_monthly AS
        SELECT d.month_end_date, to_char(d.month_end_date, 'YYYYMMDD')::int AS month_date_key,
               f.product_key, f.farm_key, c.channel,
               sum(f.net_revenue_nad) AS net_revenue_nad, sum(f.cogs_nad) AS cogs_nad, sum(f.quantity_kg) AS quantity_kg,
               count(*) AS invoice_lines
        FROM dw.fact_sales f
        JOIN dw.dim_date d ON d.date_key = f.invoice_date_key
        JOIN dw.dim_customer c ON c.customer_key = f.customer_key
        GROUP BY 1, 2, 3, 4, 5;
        CREATE INDEX IF NOT EXISTS ix_agg_sales_monthly_date ON mart.agg_sales_monthly (month_date_key);
```

Technique 3: removing unused columns, reducing cardinality and using the right data types.

Table 38. Model size and refresh volume, naive vs optimised import

metric	naive	optimised
Columns imported (14 tables)	243	165

metric	naive	optimised
fact_sales columns	24	14
Columnar store size, all tables (MB, DuckDB proxy)	16.01	10.76
Bytes streamed from PostgreSQL at refresh (MB)	46.50	27.60
Extract + load wall time (s)	2.17	1.27
Highest-cardinality fact_sales column	sales_line_key (290,971)	invoice_id (198,059)
Rows refreshed: full vs incremental (10-day partition)	302903	2162
Refresh query time: full vs incremental (s)	0.01	0.00

fact_sales columns by cardinality - naive import:

- The `mart` views expose only what the model uses: **165 columns instead of 243 (−32 %)**. `fact_sales` goes from 24 columns to 14.
- Removed:
  - the fact surrogate key `sales_line_key` (290,971 distinct values, the most expensive column for VertiPaq and never used in a visual);
  - audit columns;
  - the stored `gross_margin_nad`, now a measure;
  - FX and sales-rep columns.
- The text `invoice_no` is replaced by an integer `invoice_id`, so the highest-cardinality column is now an integer. It is still needed for the *Invoices* distinct count.
- Integer smart date keys replace dates on the facts.
- Result: compressed size **16.0 → 10.8 MB (−33 %)**, bytes streamed at refresh **46.5 → 27.6 MB (−41 %)**, and extract-plus-load time **2.17 → 1.27 s (−41 %)**.

Technique 4: incremental refresh with query folding.

- The `Sales` partition filter uses integer keys that **fold** into a `WHERE invoice_date_key >= … AND < …` predicate. The predicate is answered by the BRIN index (Q5).
- A daily refresh re-reads **2,162 rows instead of 302,903 (−99.3 %)**. The other facts are small enough to refresh in full.

Technique 5: DAX refactoring and model design. Measured in Desktop by the team.

- `VAR` caches repeated sub-expressions. `Net Revenue YoY %` evaluates `[Net Revenue PY]` once, not three times.
- `KEEPFILTERS` with a column predicate replaces `FILTER(ALL(...))` table iterators.
- `Closing Stock` finds the last snapshot with one `MAX` over an integer key and then a single `SUM`. It avoids a `SUMX` over dates with `LASTNONBLANK`.
- `REMOVEFILTERS` on the five product and farm dimensions replaced an `ALLEXCEPT` pattern whose behaviour across snowflaked tables was harder to reason about.
- All relationships are single-direction and many-to-one, with no bidirectional filter propagation.
- Auto date/time is off, which removes one hidden date table per date column.

Table 39. Power BI Desktop measurements (protocol in BUILD_GUIDE.md section 7; the team records the values here)

Measurement	Tool	Target	Result
Slowest visual per page, median of 3 refreshes (ms)	Performance Analyzer	< 1,000 ms	recorded in Desktop by the team
`Net Revenue FYTD PY` by month: total, SE, FE (ms), cold cache	DAX Studio Server Timings	< 300 ms	recorded in Desktop by the team
Model size, naive vs optimised import	VertiPaq Analyzer (`.vpax`)	≤ 15 MB	recorded in Desktop by the team (proxy: 10.8 MB)
Full refresh time (PostgreSQL mode)	Desktop refresh	< 60 s	recorded in Desktop by the team

Evaluation against the KPIs and objectives of §3.1

Table 40. The BI solution evaluated against the business objectives and KPIs

Objective / KPI	Value delivered by the solution	Status	Comment
O1 Revenue growth ≥ 8 %, GM ≥ 50 %	FY2026 N$ 458.8 m, **+9.4 %**; GM **53.8 %**	Met in FY2026; **at risk** in FY2027	The forecast shows +1.5 % for FY2027. The dashboard makes the gap visible six months ahead
O2 Yield at or above plan and benchmark; failed area < 3 %	Irrigated maize 8.68 vs plan 8.63 t/ha (2024/25); maize gap vs NSA −0.06 t/ha; failed area 2.0 %	Met (irrigated); weather-dependent (dryland)	Dryland −27 % in the drought year; the regression quantifies why
O3 Top-10 share < 50 %; export share 40–50 %	Top-10 share **57.3 %**; export **46.2 %**	**Not met** (concentration)	Segmentation names the 20 key accounts and 19 lapsed accounts to win back
O4 Cost per tonne −5 % real	Measurable per farm, crop and season for the first time (e.g. dryland maize N$ 3,270/t in 2024/25 vs N$ 5,120/t in 2023/24)	Baseline established	Cost per tonne is dominated by weather through yield; compare like-for-like seasons

Objective / KPI	Value delivered by the solution	Status	Comment
O5 One trusted source, fast close, DQ ≥ 98 %	100 % of KPIs come from the semantic model; the full rebuild runs in about 3 minutes, so the month-end pack is available the same day; 236 of 237 injected defects caught with 0 false positives; reconciliation 18 PASS / 1 WARN	**Met**	The WARN is a business action (N$ 4.9 m of mispriced lines to credit)
Solution: freshness	Daily refresh; incremental 2,162 rows	Met	
Solution: query speed	Warehouse queries 0.04–27 ms after optimisation	Met	Power BI visual timings to be confirmed in Desktop

Overall. The solution answers all five business questions from one governed model. It already exposes two uncomfortable facts that the Excel process hid:

- customer concentration (57 %) above the risk appetite;
- a flat FY2027 outlook.

Those insights justify the investment more than any technical metric.

3.8 Data governance and security

Ownership and stewardship

Governance follows a federated model:

- **Business owners** are accountable for the meaning and the access rights of their domain's data.
- **Stewards** run the day-to-day data-quality work.
- **IT** is the custodian of the platform.

Every DQ rule in `audit.dq_rule` names its steward role, so an issue in the log has an owner from the moment it is created.

Table 41. Data governance roles

Role	Held by	Accountable for	Key artefacts
Data owner: commercial	CFO	Sales, customer and cost data; approves who sees revenue and margin (RLS mapping, app audiences)	`sec.user_zone_access`, workspace and app permissions
Data owner: production	Agriculture operations manager	Farm, planting, harvest and stock data; KPI definitions for yield	KPI glossary (measure descriptions)
Farm data steward	Senior farm administrator	Rules HV01–HV08, PL01–PL03; harvest quarantine	`audit.dq_issue_log`, page 4 "Corrected records"
Sales data steward	Sales administration lead	SA01–SA08; price anomalies (R19); inferred customers	R19 queue, credit notes
Finance data steward	Financial accountant	CL01–CL04; ledger control totals (R09)	Reconciliation report
Master data steward	Commercial administrator	MD01–MD05; alias and xref tables; change log with effective dates (SCD2)	Master-data workbook, `ref.*`
External data steward	BI analyst	WE, MP, FX, CE rules; refresh of public sources	`fetch_public_data.py`, source licences
Inventory controller	Packhouse manager	ST01–ST03; month-end counts	Stock-flow checks R07, R08
Data custodian	IT / DBA	Servers, backups, roles, TLS, gateway, patching	`pg_dump` backups, `10_security_grants.sql`
BI developer	BI team (2)	Pipeline, semantic model, report, performance	Git repository (PBIP, SQL, Python), release notes

Table 42. RACI for recurring governance activities (R = responsible, A = accountable, C = consulted, I = informed)

Activity	Data owner	Steward	BI developer	Custodian	Consumers
Define or change a KPI	A	C	R	–	C
Grant or revoke data access (RLS map, app audience)	A	–	R	C	I
Maintain master data and mappings	A	R	C	–	I
Resolve DQ issues and quarantine	A	R	C	–	I
Run, monitor and recover the pipeline	I	I	R	A	–
Change the model or report (release)	A	C	R	C	I
Security review (quarterly)	A	C	R	R	–

Data-quality rules and monitoring

The 37 rules (§3.3) cover seven dimensions: validity, completeness, consistency, uniqueness, accuracy, integrity and timeliness. They are **data**, not code comments. `audit.dq_rule` stores each rule's dimension, action, owner and description, so the catalogue is queryable and appears in `mart.dq_summary`. Five operating thresholds turn the rules into a managed process:

Table 43. DQ service levels

Indicator	Threshold	Source	Action when breached
Sales lines on Unknown farm	≤ 2 % of lines (now 0.81 %)	R11	Sales steward chases the farm of origin with dispatch
Open inferred customers	0 older than 30 days	R12, SA04	Master data steward completes the record
Price-anomaly lines awaiting review	0 older than 5 working days	R19, SA07	Credit note or confirmation
Harvest quarantine	0 rows older than 7 days in season	HV01–HV03	Farm resubmits the corrected log
Reconciliation	All PASS, or WARN with a documented explanation	R01–R19	Batch is not published until explained

The issue lifecycle: **detected** (a rule fires and the raw value is logged) → **routed** (to the owner role) → **resolved** (the source is corrected and the next batch reloads it) → **verified** (the rule stops firing and the reconciliation passes). Because the pipeline is idempotent, a correction at the source flows through on the next run with no manual patching in the warehouse.

Metadata and lineage

- **Technical metadata.** Every warehouse column carries a `COMMENT ON` holding a business definition and its source. The data dictionary (Appendix A, plus an `.xlsx`) is generated from the catalogue, so it cannot drift.
- **Business metadata.**
  - Each of the 79 measures has a description shown as a tooltip in the Power BI field list.
  - Display folders group measures by business area.
  - The KPI definitions in §3.1 are the glossary.
- **Operational metadata.** `audit.etl_batch` (batch, type, start, duration, status) and `audit.etl_step_log` (rows in, out and rejected per step) record every run. Every warehouse row carries `dw_batch_id`.
- **Lineage, to the row.**
  - Staging keeps `source_file` and `row_num`.
  - `fact_harvest` keeps the source row.
  - Facts carry degenerate keys (`invoice_no` + `line_no`, `posting_id`, `cycle_id`).
  - After publishing, the Power BI service's lineage view continues the chain to reports and dashboards. If the tenant has Microsoft Purview, the PostgreSQL source and the Power BI items can be scanned into its data map.

Lineage example. The executive card "Net revenue FYTD = N$ 458.8 m" traces back as follows:

1. Measure `[Net Revenue FYTD]`.
2. `[Net Revenue]`.
3. `'Sales'[Net Revenue]`.
4. `mart.fact_sales.net_revenue_nad`.
5. `dw.fact_sales`, batch 1.
6. `stg.sales_clean`, with FX and discount derivation and rules SA01–SA07.
7. `stg.sales_raw`.
8. The source row in `sales_invoice_lines_FY2026.csv.gz`.

Control total R05 proves that nothing was gained or lost along the chain.

Security and privacy

Database layer.

- Two least-privilege login roles:
  - `gb_etl_loader` writes `stg`, `ref`, `dw` and `audit`;
  - `gb_pbi_reader` can only `SELECT` from `mart`.
- `PUBLIC` is revoked on every schema.
- In production the connection requires TLS (`sslmode=require`) through the on-premises data gateway, whose credentials are stored encrypted in the service.
- Backups (`pg_dump -Fc`) are encrypted at rest and tested by restore (`BUILD_GUIDE.md` §1).

Power BI layer.

Table 44. Workspace roles and distribution

Workspace role	Who	Rights	RLS applies?
Admin	BI lead (1 named person + 1 deputy)	Everything, including access management	No
Member	BI developers	Publish and update content, share	No
Contributor	None by default (temporary for testers)	Edit content	No
Viewer	Not used directly	–	Yes
App audience "Executive" / "Zone managers" / "Export desk"	Entra ID groups	View the app; no download of the semantic model; *Build* permission only for named analysts	**Yes**: roles §3.4

- **Sensitivity labels** (Microsoft Purview Information Protection):
  - The semantic model and report are labelled **Confidential – Commercial**, because they hold revenue, margin and customer data.
  - The label is inherited by exports to Excel, PowerPoint and PDF and is enforced on downloaded files.
  - A future public, aggregated "industry benchmark" report using only NSA, World Bank and weather data could be labelled **Public**.
- **Export controls.** Export of underlying data is disabled for the zone-manager audience. Export of summarised data is allowed.

Anonymisation and data minimisation.

- The warehouse holds **no personal identifiers**:
  - no ID numbers, phone numbers, e-mail addresses or bank details;
  - customers are businesses identified by codes;
  - informal traders are pseudonymous ("Informal Trader OD-001");
  - farm managers appear only as codes (MGR-1xx);
  - recorders in the harvest logs appear only as codes.

- The only user identities are the business UPNs in `sec.user_zone_access`, needed for RLS.
- All internal data in this project is synthetic (assumption A1).

Legal and ethical considerations

Table 45. Legal framework applied

Instrument	Relevance	How the solution complies
Constitution of the Republic of Namibia, Article 13 (right to privacy)	Personal information about individuals	Minimisation: no personal identifiers; access by role
Namibian Data Protection Bill (2023 draft; returned to Parliament in 2025; not yet enacted as at the sources consulted)	Expected principles: lawful and fair processing, purpose limitation, minimisation, security safeguards, data-subject rights, limits on cross-border transfer (Afriwise, 2023; Windhoek Express, 2025b; Chambers and Partners, 2026)	Designed to comply in advance: purpose-bound data (management reporting), minimisation, documented security controls, retention rules, and a data-processing record (this report). Re-assess when the Act is promulgated.
Electronic Transactions Act 4 of 2019 (Namibia)	Legal recognition of electronic records	Lossless landing with source-file lineage keeps electronic records trustworthy and traceable
Statistics Act 9 of 2011 (Namibia)	Use of official statistics (NSA)	NSA data is used as published, attributed, and not presented as Green Basket's own data
EU GDPR / UK GDPR	Personal data of contact persons at European export buyers, where processed	Contacts are not loaded into the warehouse; if a CRM is added later, GDPR principles apply, with a lawful basis of contract or legitimate interest
POPIA (South Africa)	Applies if processing takes place in South Africa (e.g. a hosted ERP)	Same controls; the hosting location is documented in the data-processing record
EU Regulation (EC) 178/2002, Article 18 (food traceability) and GlobalG.A.P.	Export buyers must trace produce one step back	Farm of origin on invoice lines (SA05 monitored), block-level harvest lineage, and certification status as an SCD2 attribute
Licences of public data	World Bank (CC BY 4.0), NASA POWER (free with attribution), FRED (free with attribution)	Attributed in the report and dashboards' "About" text; terms recorded by the external data steward

Ethical use.

- **No harm to individuals.**
  - Anomaly flags and "corrected records" counts help stewards improve data. They are not used to rate or discipline farm staff.
  - Segment labels such as "lapsed" describe purchasing behaviour, not creditworthiness, and are not used for automated credit decisions.
- **Honest uncertainty.**
  - Forecasts are shown with intervals and their hold-out error.
  - Regression effects are labelled associations, not causal effects (§3.5).
- **Transparency about data provenance.** Synthetic data and stand-in files are labelled as such (assumption A2). Design mock-ups are labelled as mock-ups (§3.6).

Governance checklist for ongoing maintenance

Table 46. Governance checklist

Frequency	Check	Owner	Evidence
Every run (daily)	Batch status SUCCEEDED; all reconciliation checks PASS or WARN explained; no step with unexpected rejects	BI developer	`audit.etl_batch`, `mart.recon_latest`
Daily in season	Harvest quarantine cleared; HV06 corrections reviewed with the farm	Farm data steward	Page 4, `dq_issue_log`
Weekly	Price anomalies (R19) actioned; inferred customers completed; top-25 harvest anomalies reviewed	Sales and master-data stewards	R19, R12, `dw.harvest_anomaly`
Monthly	Stock-flow and ledger control totals agreed with finance; refresh history and gateway health checked	Finance steward, custodian	R07–R10, service refresh log
Monthly	Access review of RLS mapping vs HR leavers and movers	Data owner	`sec.user_zone_access` diff
Quarterly	Workspace roles, app audiences and Build permissions reviewed; sensitivity labels checked; test restore of the backup	Data owner, custodian	Access review sign-off
Quarterly	Model performance (Performance Analyzer, VertiPaq); unused measures and columns removed; forecast and segments retrained and accuracy logged	BI developer	Benchmark files, release notes
Annually / on change	DQ rule catalogue reviewed; legal register updated (e.g. when the Data Protection Act is promulgated); public data licences re-checked	Data owners, external steward	Updated §3.8 register
Every release	Changes made in Git (PBIP diff reviewed); pipeline re-run from raw; RLS test matrix re-run	BI developer	Pull request, test evidence

3.9 Presentation, Q&A and individual reflection

10-minute presentation outline

The deck is `07_presentation/GreenBasket_ABI820S_Presentation.pptx`: 13 slides plus 3 backup slides, with speaker notes on every slide. It is built by `build_presentation.js` from `deck_data.json`, which `deck_data.py` extracts from the warehouse, so every chart on the slides is native, editable and matches this report. Every member presents the part they built, so each can answer questions about it (rubric criterion 9).

Table 47. Presentation plan (10 minutes + 5 minutes Q&A)

Time	Slide	Content and key message	Presenter
0:00–0:30	1 Title	Green Basket cc: one trusted view of production, yield, cost and revenue	Member 1
0:30–1:30	2 The problem	Nine sources, no single truth: 2,587 duplicate invoice lines, 47 crop spellings, ×10 keying errors	Member 1
1:30–2:00	3 Questions and KPIs	Five business questions mapped to 5 objectives and their KPIs	Member 1
2:00–2:45	4 Architecture	ELT in PostgreSQL 16: landing → staging → star schema → mart → Power BI	Member 3
2:45–3:45	5 Dimensional model	Bus matrix, grain, snowflake trade-off, SCD2: *Nkurenkuru before and after the pivot*	Member 2
3:45–4:45	6 Data quality	37 rules; **236 of 237 injected defects caught, 0 false positives**; the two bugs our validation found	Member 3
4:45–5:15	7 Reconciliation and governance	19 checks (18 PASS, 1 WARN = N$ 4.9 m to credit); owners, RLS, labels	Member 3
5:15–6:00	8 Semantic model	Import + incremental refresh; 79 measures; like-for-like FYTD; dynamic RLS	Member 4
6:00–7:30	9 **Live demo**	Page 1 (FY2026 → drill to quarter) → page 2 (matrix drill Region › Farm, drill-through) → View as zm.northcentral	Member 4
7:30–8:15	10 Forecast and segments	Seasonal naive wins (WAPE 12.7 %); FY2027 ≈ N$ 465.6 m (+1.5 %); 5 segments, 20 key accounts = 59 %	Member 5
8:15–8:45	11 Yield drivers and anomalies	Elasticity 0.77 recovers the truth; a pivot is worth about N$ 16,000/ha per season; rules beat Isolation Forest	Member 5
8:45–9:15	12 Performance	Indexes up to ≈ 400×, aggregate ≈ 17×, −32 % columns, −99 % refresh rows	Member 5
9:15–10:00	13 Recommendations	Irrigate maize, diversify buyers, run the DQ process, publish the app	Member 1

Demo safety net. If Power BI or the network fails, slide 9 and backup slides 14–16 hold the four page images, so the demo can continue from them. Rehearse the demo path twice, with a stopwatch.

Q&A bank (5 minutes)

Table 48. Anticipated questions and model answers (owner = member who answers)

#	Question	Model answer	Owner
1	Why a snowflake at all? Isn't a star always better for Power BI?	Crop is a **conformed grain**: six facts are recorded per crop, not per product. Flattening crop into product would force fake product rows or a second crop dimension. We snowflaked only there and at region; customer geography is flattened. The cost is two tiny relationships (12 and 5 rows).	M2
2	How do you know your SCD Type 2 works?	Check R13 (no overlaps, gaps or duplicate current rows), the version table for GB-KAW-01, and the business effect: dryland seasons stay at 0.56–2.23 t/ha and pivot seasons at 7.4–8.6. The day-2 delta created exactly one new version (GB-ERO-01). Re-running creates none, because of the hash compare.	M2
3	Your data is synthetic. Why should we believe the results?	Because we know the truth. The generator logs every defect and the true response model. Our DQ rules found 236 of 237 injected errors; the regression recovered the true rainfall elasticity (0.77). With real data you cannot measure precision and recall. The pipeline reads the real public files unchanged (`fetch_public_data.py`).	M3 / M5

#	Question	Model answer	Owner
4	What was the hardest bug?	HV06 masked itself: in a two-block cycle the median included the outlier, so a 47 t/ha maize block passed. We switched to a leave-one-out median, and recall went from 6/8 to 7/8 with zero false positives.	M3
5	Why PostgreSQL rather than SQL Server Express?	Express caps a database at 10 GB and has no Agent. PostgreSQL parses the NASA JSON natively, has BRIN indexes, and has no edition limits. Power BI connects through the built-in connector and the gateway.	M3
6	Why Import and not DirectQuery?	303 k rows compress to about 11 MB. Import gives millisecond visuals and full DAX. Freshness needs are daily. DirectQuery would add gateway latency to every visual. Incremental refresh keeps refresh to about 2,200 rows.	M4
7	How does your FYTD avoid comparing a half year with a full year?	`Net Revenue FYTD PY` stops at `EDATE([Last Sales Date], −12)` with `KEEPFILTERS`, i.e. the same day last year. Without it, FY2027 would show −73 %.	M4
8	Show us that RLS works.	View as zm.northcentral: 4 farms, N$ 62.7 m FY2026. ops.north: 8 farms, N$ 139.6 m. An unmapped user sees blank. Expected values come from SQL. Note that RLS applies to Viewers and app users, not workspace members.	M4
9	Why is the simplest forecast the best?	With 4.5 seasons of a very regular seasonal pattern, extra parameters add variance. Seasonal naive has WAPE 12.7 %, with 80 % interval coverage of 83 %. By category, SARIMA wins for cereals (9.2 %), so we keep model choice per series. A rolling-origin test is the next step.	M5
10	Silhouette is best at k = 3; why choose k = 5?	k = 3 puts 84 % of customers in one cluster, which is not actionable. Our pre-set rule picks the best Calinski–Harabasz among stable solutions (ARI ≥ 0.95, k ≥ 3), giving k = 5 with silhouette 0.41.	M5
11	Is irrigation *causing* the yield doubling?	It is an adjusted association: crop and farm effects are controlled, but farms chose where to irrigate. Nkurenkuru's before/after with an SCD2 farm version is the closest thing to a natural experiment.	M5
12	What is the business value?	Three decisions: (1) pivot investment, worth about N$ 16,000/ha per season in margin; (2) concentration risk, with 57 % of revenue from 10 buyers; (3) N$ 4.9 m of mispriced invoices found. Plus a same-day month-end pack instead of days of Excel work.	M1
13	What about privacy law in Namibia?	The Data Protection Bill is not yet in force. We designed to its principles anyway: minimisation (no personal identifiers), purpose limitation, security controls, RLS and sensitivity labels. GDPR applies only if EU contact data is processed, and we do not load it.	M3
14	How would this scale to 10× the data?	Incremental refresh on `fact_sales`, BRIN on the date key, and the monthly aggregate as an Import aggregation over a DirectQuery detail table (composite model). PostgreSQL has no 10 GB ceiling.	M5
15	What would you do next?	Real NASA, World Bank and FRED data; SARIMAX with FX and EU prices; rolling-origin back-tests; block-level soil data; publish the app with Purview labels; automate the pipeline with `pg_cron` and alerting.	M1
16	Which part did *you* build?	Each member answers from the reflection: their files, a decision they made, a problem they solved.	All

Individual reflection templates

Each member writes their own reflection **in their own words**, about 300–400 words. The prompts below structure it. The team must not edit another member's reflection. The contribution factors are agreed at a team meeting and signed by all five members.

Table 49. Individual contribution factor statement (agreed by the team; 1.0 = full, equal contribution)

Member	Role	Sections and files owned	Agreed contribution factor	Signature
Member 1	Project lead and business analyst	3.1, 3.9; slides 1–3, 13	_____	_____
Member 2	Data warehouse architect	3.2; `03_ddl_warehouse.sql`, `06_etl_load_dimensions.sql`, data dictionary	_____	_____
Member 3	ETL, data quality and governance	3.3, 3.8; `02_ddl_staging_ref.sql`, `05_etl_transform.sql`, `08_reconciliation.sql`, generator	_____	_____
Member 4	Power BI developer	3.4, 3.6; `model_spec.py`, `build_pbip.py`, `.pbix` / `.pbit`, RLS tests	_____	_____
Member 5	Analytics and performance lead	3.5, 3.7; `03_analytics/*.py`, `13_optimisation.sql`	_____	_____

Reflection prompts (one page per member):

1. **My contribution.** Which files, sections and decisions are mine? Cite commits, file names and dates.
2. **One technical decision I can defend.** For example: the grain of `fact_crop_cycle`, the HV06 redesign, the FYTD cut-off, the k = 5 rule. State the alternative I rejected and why.
3. **A problem I hit and how I solved it.** Describe the symptom, the diagnosis and the fix.
4. **What I learned** about BI and analytics practice: not only tools, but trade-offs, data quality and governance.
5. **What I would do differently** with more time or real data.
6. **Use of AI tools.** What I used an AI assistant for, and how I verified its output, in line with the NUST policy.
7. **Teamwork.** How work was divided and integrated, and how disagreements were resolved.

4 Submission package, self-assessment and limitations

Submission package checklist

Table 50. Every required deliverable mapped to its file(s) and to where it appears in this report

#	Required deliverable	File(s) in the package	In this report	Status
1	Final report (PDF)	`06_report/GreenBasket_ABI820S_Final_Report.pdf`; built by `06_report/build_report.py` from `06_report/src/*.md`	–	✔ Done
2a	Power BI file (`.pbix`)	**To be saved in Power BI Desktop** from `04_powerbi/GreenBasket_Agri.pbip` (`BUILD_GUIDE.md` §8). Source: `GreenBasket_Agri.SemanticModel/` (TMDL) and `GreenBasket_Agri.Report/` (PBIR), validated by Microsoft TOM and the PBIR schemas	§3.4, §3.6	⚠ Team action (≈ 1 h on Windows)
2b	Power BI template (`.pbit`)	**To be exported in Desktop**: *File › Export › Power BI template*	§3.4	⚠ Team action
2c	Model fallbacks	`GreenBasket_model_script.tmdl` (TMDL view), `measures.dax`, `data_csv/*.csv.gz` (CSV mode)	Appendix B	✔ Done
3a	SQL scripts: DDL, staging, load procedures	`02_sql/00_…13_*.sql`, `02_sql/run_pipeline.py`	§3.3, Appendix C	✔ Done
3b	Python scripts	`01_data/generate_synthetic_data.py`, `01_data/fetch_public_data.py`, `03_analytics/*.py` (11 scripts), `04_powerbi/*.py`, `run_all.py`	§3.3, §3.5, §3.7	✔ Done
3c	Warehouse backup and DDL	`05_backup/green_basket_dw_pg16_warehouse.backup` (pg_dump custom format, restore-tested); `05_backup/green_basket_dw_schema_ddl.sql`	§3.3, `BUILD_GUIDE.md` §1	✔ Done
4a	Data dictionary	Appendix A; `06_report/data_dictionary.md` and `.xlsx` (generated from the catalogue)	§3.2, Appendix A	✔ Done
4b	Schema / ERD diagrams (embedded)	ERD 1 and ERD 2 in §3.2; sources `06_report/diagrams/*.mmd`	§3.2, Appendix D	✔ Done
5	Datasets or reproducible links	`01_data/raw/` (synthetic extracts + `generator_manifest.json`); regenerated byte-identically with seed 20261028; public sources by `fetch_public_data.py` with the links in §3.3	§3.3	✔ Done
6	PowerPoint presentation	`07_presentation/GreenBasket_ABI820S_Presentation.pptx` (13 slides + 3 backup, speaker notes, native charts)	§3.9	✔ Done
–	Run instructions	`README.md`, `requirements.txt`, `04_powerbi/BUILD_GUIDE.md`	–	✔ Done
–	Screenshots (Power BI, pgAdmin)	Captured by the team (`BUILD_GUIDE.md` §9); the mock-ups are replaced under the same file names	§3.3, §3.4, §3.6, §3.7	⚠ Team action

Marking rubric self-assessment

Table 51. Self-assessment against the 100-mark rubric (evidence for "Excellent" and what still has to happen)

#	Criterion (marks)	Evidence in this submission	Self-assessed band	Remaining risk / action
1	Business case and requirements (5)	Organisation profile with farm portfolio; 5 SMART objectives with baselines; problem quantified from profiling; 5 BQs with KPI definitions and pages; 9 stakeholder groups (technical and non-technical) with needs; BPMN flowchart with data capture points and design insights (§3.1)	Excellent	None
2	DW design (15)	Bus matrix (9 processes × 10 dimensions); grain and additivity for all 9 facts; 2 ERDs; 9 dimensions incl. a complete date dimension with Namibian holidays; justified snowflake (trade-off table); SCD2 with replay, hash and evidence of business effect; alternative designs compared; full data dictionary (§3.2, App. A)	Excellent	None
3	ETL, integration, DQ (10)	One-command reproducible pipeline; 9 sources with 6 conformance mechanisms; profiling; 37 rules **validated against ground truth (236/237, 0 FP)**; 19 reconciliation checks; load evidence (§3.3)	Excellent	External data are layout-exact stand-ins (A2); run `fetch_public_data.py` for the real files

#	Criterion (marks)	Evidence in this submission	Self-assessed band	Remaining risk / action
4	Semantic model and DAX (10)	Import + incremental refresh justified; 34 single-direction relationships; hidden columns, folders, sort-by; 79 measures incl. FYTD like-for-like, YoY, MAT, rolling, `CALCULATE` patterns; static + dynamic RLS with an SQL-derived test matrix (§3.4)	Excellent **if** the `.pbix` is built	Penalty cap if the `.pbix` is missing. DAX was validated by TOM and a static checker, but not yet executed in the VertiPaq engine: verify the expected KPI values (`BUILD_GUIDE.md` §3)
5	Advanced analytics (15)	Roll-up, drill-down, slice, dice, pivot + cube, drill-across, semi-additive with outputs; four techniques (forecasting, clustering, regression/RF, anomaly detection) with metrics, assumptions, limitations and code; ground-truth recovery (§3.5)	Excellent	None
6	Dashboards (15)	4 pages + drill-through + tooltip; chart rationale; hierarchy; validated CVD-safe palette, alt text, tab order, contrast; interactivity; wireframes; rationale for both audiences (§3.6)	Very good → Excellent after the Desktop build	Pages are judged in the real report: build them, add the drill-through, tooltip and bookmarks, and replace the mock-ups with screenshots
7	Performance (5)	Baseline method; 5 techniques with quantified before/after (up to ≈ 400×, −32 % columns, −99.3 % refresh rows); evaluation against objectives (§3.7)	Excellent (warehouse); pending (Power BI timings)	Record Performance Analyzer, DAX Studio and VertiPaq values in the §3.7 table
8	Governance, security, ethics (5)	Roles, RACI, DQ service levels, lineage example, DB and Power BI security, sensitivity labels, minimisation, Namibian and international legal register, checklist (§3.8)	Excellent	None
9	Report, presentation, individual contribution (20)	Structured report with APA references; 13-slide deck (+3 backup) with notes; demo plan; Q&A bank; reflection templates and contribution statement (§3.9)	Depends on delivery	**Each member must explain their own part in depth.** Rehearse the demo; write reflections personally; disclose AI use per NUST policy

Limitations and next steps

1. **Synthetic internal data and stand-in external files** (A1, A2).
   - The pipeline and analytics are validated against known truth, but business magnitudes are illustrative.
   - **Next:** load the real NASA POWER, World Bank and FRED files with `fetch_public_data.py`, and transcribe the NSA/MAFWLR tables.
2. **Power BI artefacts not yet produced in Desktop.** The PBIP is schema-valid, but `.pbix`/`.pbit` creation, execution-level DAX verification, screenshots and Desktop timings require Windows. The steps are scripted in `BUILD_GUIDE.md`.
3. **Forecast validation uses one hold-out season. Next:** rolling-origin evaluation and exogenous drivers (FX, EU grape prices).
4. **Fertiliser measured at cycle level** from ledger allocations, which attenuates its estimated effect. **Next:** record input applications per block in the planting register.
5. **Yield effects are observational. Next:** paired-block trials when a farm converts to irrigation, using the SCD2 history to compare before and after.
6. **Operationalisation. Next:**
   - schedule `run_all.py` with `pg_cron` or Task Scheduler;
   - alert on any reconciliation WARN or FAIL;
   - deploy to a Pro workspace with the gateway, app audiences and Purview labels.

References

Afriwise. (2023). *Securing data privacy: Key highlights of Namibia's Data Protection Bill, 2023*. https://www.afriwise.com/blog/securing-data-privacy-key-highlights-of-namibias-data-protection-bill-2023

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Chambers and Partners. (2026). *Doing business in 2026: Namibia* [Global practice guide]. https://gpg-pdf.chambers.com/Doing-Business-In-2026/717/

Federal Reserve Bank of St. Louis. (2026). *FRED economic data: DEXSFUS, DEXUSEU, DEXUSUK* [Data set]. https://fred.stlouisfed.org/

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting, 22*(4), 679–688. https://doi.org/10.1016/j.ijforecast.2006.03.001

Integrated Food Security Phase Classification. (2025). *Namibia: Acute food insecurity analysis, July 2025 – June 2026*. https://www.foodsecurityportal.org/sites/default/files/2025-10/IPC_Namibia_Acute_Food_Insecurity_Jul2025_Jun2026_Report.pdf

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17

Microsoft. (2024a). *Power BI Desktop projects: Report folder*. Microsoft Learn. https://learn.microsoft.com/power-bi/developer/projects/projects-report

Microsoft. (2024b). *Understand star schema and the importance for Power BI*. Microsoft Learn. https://learn.microsoft.com/power-bi/guidance/star-schema

Microsoft. (2024c). *Editions and supported features of SQL Server 2022*. Microsoft Learn. https://learn.microsoft.com/sql/sql-server/editions-and-components-of-sql-server-2022

Microsoft. (2024d). *Power BI Desktop projects: Semantic model folder*. Microsoft Learn. https://learn.microsoft.com/power-bi/developer/projects/projects-dataset

Microsoft Fabric Community. (2025). *PBIR format* [Online forum thread]. https://community.fabric.microsoft.com/t5/Desktop/PBIR-format/m-p/4669466

NASA Langley Research Center. (2024). *Prediction Of Worldwide Energy Resources (POWER): Daily point API* [Data set]. https://power.larc.nasa.gov/

Namibian Agronomic Board. (2021). *White maize market report 2020/2021*. https://www.nab.com.na/wp-content/uploads/2021/05/D-White-maize-market-report-2020-2021.pdf

Namibian Agronomic Board. (n.d.). *Market share promotion*. Retrieved October 27, 2026, from https://www.nab.com.na/horticulture/market-share-price/

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

The Namibian. (n.d.). *White maize producer price at N$6 680*. Retrieved October 27, 2026, from https://namibian.com.na/?p=681300

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Windhoek Express. (2025a). *Namibia produced 94,262 tonnes of grain in 2025*. https://www.we.com.na/mw-main/namibia-produced-94262-tonnes-of-grain-in-2025-NMH003391-3015-3434

Windhoek Express. (2025b, August 18). *Data protection bill returns to parliament*. https://www.we.com.na/technology-we/data-protection-bill-returns-to-parliament2025-08-18

World Bank. (2026). *Commodity markets: "Pink Sheet" monthly commodity prices* [Data set]. https://www.worldbank.org/en/research/commodity-markets

Appendix A – Data dictionary

Generated from the PostgreSQL catalog (`COMMENT ON ...`) by `03_analytics/build_data_dictionary.py`.

dw.fact_crop_cycle

Accumulating snapshot. GRAIN: one row per planted field block per crop cycle; milestone dates and measures are updated as the cycle moves from Growing to Harvesting to Completed.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	`crop_cycle_key`	bigint	N	PK	Fact surrogate key.	ETL identity
2	`cycle_id`	character varying(24)	N		Degenerate: crop-cycle id farm/crop/year+slot, e.g. GB-KAW-01/MZW/2023S.	planting register
3	`block_id`	character varying(6)	N		Degenerate: field block code.	planting register
4	`farm_key`	integer	N	FK → dim_farm.farm_key	FK to the dim_farm version valid on the planting date (as-was irrigation regime).	planting register farm_code
5	`crop_key`	smallint	N	FK → dim_crop.crop_key	FK to dim_crop.	planting register crop_code
6	`planting_date_key`	integer	N	FK → dim_date.date_key	Milestone 1 (role-playing date, active in BI): planting / bud-break date.	planting register (rule PL01)
7	`first_harvest_date_key`	integer	N	FK → dim_date.date_key	Milestone 2 (role-playing date): first harvest record (-1 = not yet).	derived from fact_harvest
8	`last_harvest_date_key`	integer	N	FK → dim_date.date_key	Milestone 3 (role-playing date): latest harvest record to date (-1 = not yet).	derived from fact_harvest
9	`harvest_season_end_date_key`	integer	N	FK → dim_date.date_key	Role-playing date: 30 September closing the agricultural season of the (expected) first harvest; conforms with fact_crop_estimate.	derived
10	`cycle_status`	character varying(10)	N		Growing, Harvesting, Completed or Failed as at the extract date.	planting register validated against harvest log (rule PL03)
11	`variety`	character varying(30)	Y		Cultivar planted.	planting register
12	`irrigation_method`	character varying(20)	N		Degenerate: irrigation method of THIS block in this cycle (Dryland, Centre pivot, Drip, ...); can differ from the farm's main system, e.g. a borehole pivot block on a dryland farm.	planting register (fallback: dim_farm as-was)
13	`is_irrigated`	boolean	N		Degenerate: true when the block was irrigated in this cycle.	derived from irrigation_method
14	`planted_ha`	numeric(10,2)	N		Planted area of the block in hectares (additive).	planting register (rule PL02 imputes)
15	`harvested_ha`	numeric(10,2)	Y		Harvested area in hectares once Completed (0 if Failed, NULL while open) (additive).	planting register
16	`planned_yield_t_ha`	numeric(8,2)	Y		Budgeted yield t/ha at planting (non-additive).	planting register
17	`planned_production_kg`	numeric(14,1)	Y		Budgeted production = planned yield x planted ha x 1000 (additive).	derived
18	`harvested_kg`	numeric(14,1)	N		Cumulative harvested kg to date (additive).	aggregated from fact_harvest

#	Column	Data type	Null	Key	Business definition	Source / lineage
19	harvest_event_count	integer	N		Number of harvest records to date (additive).	aggregated from fact_harvest
20	days_to_first_harvest	integer	Y		Days from planting to first harvest (non-additive; average in BI).	derived
21	harvest_duration_days	integer	Y		Days between first and last harvest record (non-additive).	derived
22	direct_cost_nad	numeric(16,2)	N		Crop-cycle direct cost allocated to the block by planted-area share, N$ (additive).	cost ledger cost_centre = cycle_id
23	fertiliser_kg	numeric(14,1)	N		Fertiliser applied, allocated by area share (additive).	cost ledger input_qty (CC02)
24	planted_ha_imputed	boolean	N		True when planted_ha was imputed by rule PL02.	ETL
25	dw_batch_id	integer	N		Audit: ETL batch that last updated the row.	ETL
26	dw_updated_at	timestamp with time zone	N		Audit: last update timestamp (accumulating snapshot rows are updated in place).	ETL

dw.fact_crop_estimate

Benchmark fact (external). GRAIN: one row per agricultural season x region x crop x farming sector. Area/production additive; yield non-additive (recompute as production/area).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	crop_estimate_key	bigint	N	PK	Fact surrogate key.	ETL identity
2	season_end_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: 30 September closing the agricultural season.	NSA season text e.g. 2023/2024
3	region_key	smallint	N	FK → dim_region.region_key	FK to dim_region (conformed geography).	NSA region text via ref.region_alias
4	crop_key	smallint	N	FK → dim_crop.crop_key	FK to dim_crop (rule CE01 excludes out-of-scope crops).	NSA crop text via ref.crop_alias
5	farming_sector	character varying(10)	N		Communal or Commercial farming sector.	NSA / MAFWLR table
6	area_planted_ha	numeric(12,1)	Y		Area planted, ha (additive).	NSA / MAFWLR crop estimates
7	production_t	numeric(14,1)	Y		Production, tonnes (additive).	NSA / MAFWLR crop estimates
8	yield_t_ha	numeric(8,3)	Y		Published (or derived) yield, t/ha (non-additive).	NSA (rule CE02)
9	is_yield_derived	boolean	N		True when yield was derived as production/area.	ETL
10	release_date	date	Y		Publication date of the estimate.	NSA
11	source_table	character varying(80)	Y		Lineage: report/table of origin.	NSA
12	dw_batch_id	integer	N		Audit: ETL batch.	ETL

dw.fact_forecast_monthly

Model output. GRAIN: one row per month x crop category x model x scenario (12-month back-test on hold-out data, then 12-month forward forecast).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	forecast_key	bigint	N	PK	Surrogate key.	ETL identity
2	month_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: month-end of the forecast month.	03_analytics/forecasting.py
3	crop_category_key	smallint	N	FK → dim_crop_category.crop_category_key	FK to dim_crop_category.	forecasting.py

#	Column	Data type	Null	Key	Business definition	Source / lineage
4	model_name	character varying(30)	N		Model: Seasonal naive, Holt-Winters ETS, SARIMA.	forecasting.py
5	scenario	character varying(12)	N		Backtest (hold-out evaluation) or Forecast (forward).	forecasting.py
6	actual_net_revenue_nad	numeric(16,2)	Y		Actual net revenue for back-test months, N$.	fact_sales
7	forecast_net_revenue_nad	numeric(16,2)	N		Point forecast of net revenue, N$.	forecasting.py
8	lower_80_nad	numeric(16,2)	Y		Lower bound of the 80% prediction interval.	forecasting.py
9	upper_80_nad	numeric(16,2)	Y		Upper bound of the 80% prediction interval.	forecasting.py
10	lower_95_nad	numeric(16,2)	Y		Lower bound of the 95% prediction interval.	forecasting.py
11	upper_95_nad	numeric(16,2)	Y		Upper bound of the 95% prediction interval.	forecasting.py
12	model_run_at	timestamp with time zone	N		Audit: model run timestamp.	forecasting.py

dw.fact_harvest

Transaction fact. GRAIN: one row per harvest record = farm x crop x field block x harvest day (weighbridge/packhouse slip).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	harvest_event_key	bigint	N	PK	Fact surrogate key.	ETL identity
2	harvest_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: harvest day.	harvest log date (template-specific format)
3	farm_key	integer	N	FK → dim_farm.farm_key	FK to the dim_farm version valid on the harvest date.	harvest log farm code (rule HV08)
4	crop_key	smallint	N	FK → dim_crop.crop_key	FK to dim_crop (crop grain - conformed rollup of dim_product).	harvest log crop text via ref.crop_alias
5	cycle_id	character varying(24)	Y		Degenerate: crop-cycle identifier from the planting register (links to fact_crop_cycle).	ETL block matching
6	block_id	character varying(6)	N		Degenerate: field block / pivot / vineyard block code.	harvest log
7	recorder_code	character varying(10)	Y		Pseudonymised clerk code that captured the record.	harvest log
8	harvested_kg	numeric(14,1)	N		Field mass harvested in kg after unit standardisation (additive).	harvest log quantity x ref.uom_conversion
9	is_corrected	boolean	N		True when rule HV06 corrected a unit/decimal error.	ETL
10	source_file	character varying(40)	N		Lineage: source file name.	ETL
11	source_row_num	integer	N		Lineage: row number in the source file.	ETL
12	dw_batch_id	integer	N		Audit: ETL batch.	ETL

dw.fact_inventory_monthly

Periodic snapshot. GRAIN: one row per farm store x product x month-end. Stock levels are SEMI-ADDITIVE (sum across farms/products, never across months); flows are additive.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	inventory_snapshot_key	bigint	N	PK	Fact surrogate key.	ETL identity
2	snapshot_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: month-end count date.	stock system period

#	Column	Data type	Null	Key	Business definition	Source / lineage
3	farm_key	integer	N	FK → dim_farm.farm_key	FK to the dim_farm version valid at month-end.	stock system farm_code
4	product_key	integer	N	FK → dim_product.product_key	FK to dim_product.	stock system product_code
5	opening_stock_kg	numeric(14,1)	N		Stock on hand at the start of the month, kg (semi-additive).	stock system
6	received_kg	numeric(14,1)	N		Graded produce received from the field in the month, kg (additive).	stock system
7	dispatched_kg	numeric(14,1)	N		Produce dispatched to customers in the month, kg (additive; reconciles to ERP dispatches).	stock system
8	shrinkage_kg	numeric(14,1)	N		Losses (spoilage, moisture, theft) in the month, kg (additive).	stock system
9	closing_stock_kg	numeric(14,1)	N		Stock on hand at month-end, kg (semi-additive: use closing-balance logic over time).	stock system (rule ST01)
10	closing_stock_value_nad	numeric(16,2)	N		Closing stock at standard cost, N$ (semi-additive).	stock system
11	is_imputed	boolean	N		True when the snapshot row was imputed for a missing month (rule ST03).	ETL
12	is_sign_corrected	boolean	N		True when rule ST01 corrected a sign error.	ETL
13	dw_batch_id	integer	N		Audit: ETL batch.	ETL

dw.fact_market_price_monthly

Periodic fact (external). GRAIN: one row per commodity series per month (keyed on the month-end date). Prices are NON-ADDITIVE (average only).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	market_price_key	bigint	N	PK	Fact surrogate key.	ETL identity
2	month_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: last day of the price month.	Pink Sheet period YYYYMmm
3	commodity_key	smallint	N	FK → dim_commodity.commodity_key	FK to dim_commodity.	Pink Sheet column header
4	crop_key	smallint	N	FK → dim_crop.crop_key	FK to dim_crop of the benchmarked crop (-2 for input series).	dim_commodity mapping
5	price_usd	numeric(12,2)	N		Monthly average price in USD per published unit (non-additive).	World Bank Pink Sheet (rule MP01)
6	fx_nad_per_usd	numeric(12,6)	N		Monthly average N$ per USD (non-additive).	FRED DEXSFUS (NAD = ZAR)
7	price_nad	numeric(14,2)	N		Price converted to N$ per published unit (non-additive).	derived
8	is_imputed	boolean	N		True when the price was interpolated (rule MP01).	ETL
9	dw_batch_id	integer	N		Audit: ETL batch.	ETL

dw.fact_production_cost

Transaction fact. GRAIN: one row per cost-ledger posting (farm x crop cycle or overhead x cost category x posting date).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	cost_posting_key	bigint	N	PK	Fact surrogate key.	ETL identity
2	posting_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: posting date.	cost ledger

#	Column	Data type	Null	Key	Business definition	Source / lineage
3	farm_key	integer	N	FK → dim_farm.farm_key	FK to the dim_farm version valid on the posting date.	cost ledger farm_code
4	crop_key	smallint	N	FK → dim_crop.crop_key	FK to dim_crop (-2 farm overhead, -1 crop missing - rule CL03).	cost ledger crop_code
5	cost_category_key	smallint	N	FK → dim_cost_category.cost_category_key	FK to dim_cost_category.	cost ledger free text via ref.cost_category_alias (rule CL02)
6	posting_id	character varying(24)	N		Degenerate: journal posting id (suffix -R reversal, -C correction).	cost ledger
7	cost_centre	character varying(30)	N		Degenerate: cost centre = crop cycle id, FARM/OVERHEAD or FARM/UNALLOCATED.	cost ledger
8	supplier_code	character varying(8)	Y		Pseudonymised supplier code.	cost ledger
9	amount_nad	numeric(16,2)	N		Posted cost amount in N$ (additive; reversals negative).	cost ledger (rule CL04)
10	input_qty	numeric(14,2)	Y		Physical input quantity where captured (additive within one category).	cost ledger
11	input_uom	character varying(6)	Y		Unit of input_qty.	cost ledger
12	is_reversal	boolean	N		True for reversal postings.	derived from posting_id
13	dw_batch_id	integer	N		Audit: ETL batch.	ETL

dw.fact_sales

Transaction fact. GRAIN: one row per ERP invoice or credit-note line. All monetary measures are additive (N$).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	sales_line_key	bigint	N	PK	Fact surrogate key (technical; hidden in BI).	ETL identity
2	invoice_date_key	integer	N	FK → dim_date.date_key	FK to dim_date: invoice (revenue recognition) date.	ERP invoice_date
3	dispatch_date_key	integer	N	FK → dim_date.date_key	FK to dim_date (role-playing): physical dispatch date used for stock reconciliation.	ERP dispatch_date
4	product_key	integer	N	FK → dim_product.product_key	FK to dim_product.	ERP product_code (rule SA02)
5	customer_key	integer	N	FK → dim_customer.customer_key	FK to the dim_customer version valid on the invoice date.	ERP customer_code (rules SA03/SA04)
6	farm_key	integer	N	FK → dim_farm.farm_key	FK to the dim_farm version of the farm of origin on the invoice date (-1 when missing, rule SA05).	ERP farm_code
7	invoice_no	character varying(16)	N		Degenerate dimension: ERP document number.	ERP
8	line_no	smallint	N		Degenerate dimension: line number within the document.	ERP
9	doc_type	character(3)	N		INV = invoice, CRN = credit note (quality claim).	ERP
10	ref_invoice_no	character varying(16)	Y		For credit notes: the invoice being credited.	ERP
11	sales_rep_code	character varying(6)	Y		Pseudonymised sales representative code.	ERP

#	Column	Data type	Null	Key	Business definition	Source / lineage
12	currency_code	character(3)	N		Invoicing currency (rule SA06 defaults missing values).	ERP / customer master
13	quantity_sales_uom	numeric(14,3)	N		Quantity in the invoice unit of measure (negative on credit notes). Additive only within one product.	ERP quantity
14	quantity_kg	numeric(14,3)	N		Quantity converted to kilograms (additive).	derived quantity x dim_product.kg_per_sales_uom
15	unit_price_txn	numeric(14,4)	N		Unit price in transaction currency per sales UoM (non-additive).	ERP
16	fx_rate_nad	numeric(12,6)	N		N$ per unit of transaction currency, monthly average of the invoice month (non-additive).	ref.fx_rate_monthly (FRED)
17	gross_revenue_nad	numeric(16,2)	N		quantity x unit price x FX before discount, N$ (additive).	derived
18	discount_nad	numeric(16,2)	N		Discount granted, N$ (additive).	derived from ERP discount_pct
19	net_revenue_nad	numeric(16,2)	N		Gross revenue less discount, N$ (additive) - the headline revenue measure.	derived
20	cogs_nad	numeric(16,2)	N		Standard cost of goods sold, N$ (0 on credit notes: returned produce is written off).	ERP unit_std_cost_nad x quantity
21	freight_cost_nad	numeric(16,2)	N		Outbound freight cost borne by Green Basket, N$ (additive).	ERP freight_nad
22	gross_margin_nad	numeric(16,2)	Y		Net revenue less COGS, N$ (additive, computed column).	derived
23	is_price_anomaly	boolean	N		True when the N$/kg price deviates > 4x from the product-month median (rule SA07).	ETL
24	dw_batch_id	integer	N		Audit: ETL batch that loaded the row.	ETL

dw.fact_weather_daily

Periodic fact (external). GRAIN: one row per farm location per day. Rainfall additive over time but averaged across locations; temperatures non-additive.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	weather_key	bigint	N	PK	Fact surrogate key.	ETL identity
2	date_key	integer	N	FK → dim_date.date_key	FK to dim_date.	NASA POWER date key YYYYMMDD
3	farm_key	integer	N	FK → dim_farm.farm_key	FK to the dim_farm version valid on the day (weather point = farm coordinates).	NASA POWER request per farm
4	rainfall_mm	numeric(7,2)	Y		Corrected precipitation, mm/day (PRECTOTCORR).	NASA POWER (rule WE02)
5	t_max_c	numeric(5,2)	Y		Maximum 2 m air temperature, deg C (T2M_MAX).	NASA POWER (rule WE01)
6	t_min_c	numeric(5,2)	Y		Minimum 2 m air temperature, deg C (T2M_MIN).	NASA POWER (rule WE01)
7	rh_pct	numeric(5,2)	Y		Relative humidity at 2 m, % (RH2M).	NASA POWER

#	Column	Data type	Null	Key	Business definition	Source / lineage
8	solar_mj_m2	numeric(6,2)	Y		All-sky surface shortwave irradiance, MJ/m2/day.	NASA POWER
9	is_heat_day	boolean	Y		True when t_max_c >= 35 deg C (crop heat-stress day).	derived
10	is_rain_day	boolean	Y		True when rainfall_mm >= 1.	derived
11	rainfall_imputed	boolean	N		True when rainfall was imputed by rule WE02.	ETL
12	temp_imputed	boolean	N		True when temperature was imputed by rule WE01.	ETL
13	dw_batch_id	integer	N		Audit: ETL batch.	ETL

dw.customer_segment

Model output: K-means customer segment per durable customer code (RFM + basket features, trailing 12 months).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	customer_code	character varying(10)	N	PK	Durable customer code.	clustering.py
2	segment_id	smallint	N		Cluster number.	clustering.py
3	segment_name	character varying(40)	N		Business label of the cluster.	clustering.py
4	recency_days	integer	N		Days since last invoice at 30 Sep 2026.	clustering.py
5	frequency_invoices	integer	N		Distinct invoices in the trailing 12 months.	clustering.py
6	monetary_nad	numeric(16,2)	N		Net revenue in the trailing 12 months, N$.	clustering.py
7	avg_order_kg	numeric(14,1)	N		Average kg per invoice line.	clustering.py
8	product_breadth	smallint	N		Distinct products bought.	clustering.py
9	silhouette_sample	numeric(6,4)	Y		Silhouette value of the customer within its cluster.	clustering.py
10	model_run_at	timestamp with time zone	N		Audit: model run timestamp.	clustering.py

dw.dim_commodity

International commodity price series (output prices for crops, input prices for fertiliser and fuel). SCD Type 1.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	commodity_key	smallint	N	PK	Surrogate key.	ETL
2	commodity_code	character varying(16)	N		Series code (natural key), e.g. MAIZE, UREA.	ref.commodity_series
3	commodity_name	character varying(40)	N		Descriptive series name.	World Bank Pink Sheet header
4	source_series_name	character varying(30)	N		Column header in the Pink Sheet workbook.	World Bank Pink Sheet
5	price_unit	character varying(10)	N		Unit of the published price (USD/mt, USD/bbl).	World Bank Pink Sheet
6	commodity_role	character varying(14)	N		Output price (crop sold) or Input price (cost driver).	derived
7	crop_key	smallint	N	FK → dim_crop.crop_key	Crop benchmarked by the series (sorghum proxies mahangu; -2 for input series).	ETL mapping
8	source_name	character varying(40)	N		Publisher of the series.	derived
9	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL

dw.dim_cost_category

Production cost category with cost group and behaviour (variable/fixed). SCD Type 1.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	cost_category_key	smallint	N	PK	Surrogate key (-1 Unknown).	ETL
2	cost_category_code	character(4)	N		Cost category code CC01..CC14 (natural key).	Master data workbook, sheet Cost_Categories
3	cost_category_name	character varying(40)	N		Cost category name.	Master data workbook
4	cost_group	character varying(30)	N		Cost group: Direct inputs, Labour, Energy & water, Mechanisation, Packaging & logistics, Overheads.	Master data workbook
5	cost_behaviour	character varying(15)	N		Variable, Semi-variable or Fixed.	Master data workbook
6	gl_account	character(4)	Y		General-ledger account number.	Master data workbook
7	quantity_uom	character varying(6)	Y		Unit of the physical input quantity (kg, l, hours, kWh, m3) where captured.	Master data workbook
8	is_crop_specific	boolean	N		False for farm overheads that are not allocated to a crop cycle.	derived (Fixed behaviour)
9	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL

dw.dim_crop

Crop (middle level of the product snowflake). Also a conformed shrunken dimension: harvest, cost, cycle, market-price and NSA facts are recorded at crop grain. SCD Type 1.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	crop_key	smallint	N	PK	Surrogate key (-1 Unknown, -2 Not crop-specific / farm overhead).	ETL
2	crop_code	character(3)	N		Three-letter crop code (natural key), e.g. MZW = white maize.	Master data workbook, sheet Crops
3	crop_name	character varying(40)	N		Crop name.	Master data workbook, sheet Crops
4	crop_category_key	smallint	N	FK → dim_crop_category.crop_category_key	FK to dim_crop_category (snowflake).	ETL lookup on category_code
5	scientific_name	character varying(60)	Y		Botanical name.	Master data workbook
6	is_nab_controlled	boolean	Y		True if the crop is a Namibian Agronomic Board controlled product (border closures / Market Share Promotion apply).	Master data workbook
7	storage_type	character varying(30)	Y		Post-harvest storage type (Silo, Cold room, ...).	Master data workbook
8	shelf_life_days	smallint	Y		Typical shelf life in days.	Master data workbook
9	typical_cull_pct	numeric(5,2)	Y		Typical % of harvested mass culled at grading.	Master data workbook
10	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL
11	dw_updated_at	timestamp with time zone	N		Audit: last Type 1 update timestamp.	ETL

dw.dim_crop_category

Top level of the snowflaked product hierarchy (Cereals & grains, Vegetables, Fruit). SCD Type 1.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	crop_category_key	smallint	N	PK	Surrogate key (-1 Unknown, -2 Not applicable).	ETL

#	Column	Data type	Null	Key	Business definition	Source / lineage
2	category_code	character(3)	N		Category code (natural key): CER, VEG, FRU.	Master data workbook, sheet Crop_Categories
3	category_name	character varying(30)	N		Category name.	Master data workbook, sheet Crop_Categories
4	sector	character varying(20)	N		Agricultural sector: Field crops or Horticulture.	Master data workbook
5	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL

dw.dim_customer

Customer (business, institution, pseudonymised informal trader or export buyer). SCD Type 2 on channel, credit terms and key-account flag; Type 1 elsewhere. Supports inferred (late-arriving) members.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	customer_key	integer	N	PK	Surrogate key per customer VERSION (-1 Unknown).	ETL identity
2	customer_code	character varying(10)	N		Durable natural key from ERP/MDM, e.g. C00042 (duplicates merged to survivor).	Master data workbook, sheet Customers
3	customer_name	character varying(80)	N		Trading name (Type 1). Informal traders carry a pseudonymous label only.	Master data workbook (rule MD05 formatting)
4	customer_type	character varying(24)	Y		Business, Government institution, Informal trader or Export (Type 1).	Master data workbook
5	town	character varying(40)	Y		Delivery town (Type 1).	Master data workbook
6	region_key	smallint	N	FK → dim_region.region_key	FK to dim_region for domestic customers; -2 for foreign customers, -1 when missing (rule MD02).	ETL via ref.region_alias
7	country_code	character(2)	Y		ISO 3166-1 alpha-2 country.	Master data workbook
8	country_name	character varying(30)	Y		Country name.	ref.country
9	market_group	character varying(20)	Y		Market grouping: Domestic, SACU, SADC (non-SACU), Europe, Middle East.	ref.country
10	currency_code	character(3)	Y		Invoicing currency (NAD, ZAR, USD, EUR, GBP).	Master data workbook
11	onboarding_date	date	Y		Date the account was opened.	Master data workbook
12	customer_status	character varying(10)	Y		Active or Inactive at extract date (Type 1).	Master data workbook
13	channel	character varying(20)	Y		Sales channel in this version: Miller/Processor, Wholesale, Retail, Informal market, Institutional, Export (Type 2).	Master data + Customer_History
14	credit_terms_days	smallint	Y		Payment terms in days in this version (0 = cash on delivery) (Type 2).	Master data + Customer_History
15	is_key_account	boolean	Y		Key-account flag in this version (Type 2).	Master data + Customer_History
16	is_inferred	boolean	N		True for an inferred member created from a fact before the master record arrived (rule SA04).	ETL
17	merged_codes	text	Y		Duplicate customer codes merged into this survivor (rule MD04).	ref.customer_xref

#	Column	Data type	Null	Key	Business definition	Source / lineage
18	scd_effective_from	date	N		First date this version is valid.	ETL (SCD2)
19	scd_effective_to	date	N		Last date this version is valid (9999-12-31 = open).	ETL (SCD2)
20	scd_is_current	boolean	N		True for the current version.	ETL (SCD2)
21	scd_version	smallint	N		Version sequence number per customer_code.	ETL (SCD2)
22	scd_row_hash	character(32)	N		MD5 of the Type 2 attributes.	ETL (SCD2)
23	dw_batch_id	integer	Y		Audit: ETL batch that created/last updated the row.	ETL
24	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL
25	dw_updated_at	timestamp with time zone	N		Audit: last update timestamp.	ETL

dw.dim_date

Conformed calendar dimension, one row per day 2020-01-01..2027-12-31 plus -1 Unknown; carries calendar, fiscal (Apr-Mar) and agricultural season (Oct-Sep) hierarchies.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	date_key	integer	N	PK	Smart surrogate key YYYYMMDD (-1 = unknown / not yet occurred).	generated by etl.sp_load_dim_date
2	full_date	date	Y		Calendar date.	generated
3	day_of_month	smallint	Y		Day number within the month (1-31).	derived
4	day_of_year	smallint	Y		Day number within the calendar year (1-366).	derived
5	day_of_week_iso	smallint	Y		ISO weekday number, Monday = 1 ... Sunday = 7 (sort key for day_name).	derived
6	day_name	character varying(9)	Y		Weekday name (Monday..Sunday).	derived
7	day_short	character(3)	Y		Three-letter weekday abbreviation.	derived
8	is_weekend	boolean	Y		True for Saturday and Sunday.	derived
9	is_public_holiday	boolean	Y		True on Namibian public holidays (Public Holidays Act 26 of 1990 incl. Sunday->Monday rule).	etl.fn_namibian_holiday
10	holiday_name	character varying(40)	Y		Name of the Namibian public holiday, NULL otherwise.	etl.fn_namibian_holiday
11	is_working_day	boolean	Y		Farm working day: Monday-Saturday and not a public holiday.	derived
12	iso_week	smallint	Y		ISO-8601 week number.	derived
13	week_start_date	date	Y		Monday of the ISO week.	derived
14	month_num	smallint	Y		Calendar month number 1-12 (sort key for month_name).	derived
15	month_name	character varying(9)	Y		Calendar month name.	derived
16	month_short	character(3)	Y		Three-letter month abbreviation.	derived
17	year_month	character(7)	Y		Year-month label YYYY-MM.	derived
18	year_month_key	integer	Y		Integer YYYYMM (sort key for year_month).	derived

#	Column	Data type	Null	Key	Business definition	Source / lineage
19	month_start_date	date	Y		First day of the month.	derived
20	month_end_date	date	Y		Last day of the month (key used by month-grain facts).	derived
21	is_month_end	boolean	Y		True on the last day of the month.	derived
22	days_in_month	smallint	Y		Number of days in the month.	derived
23	quarter_num	smallint	Y		Calendar quarter 1-4.	derived
24	year_quarter	character(7)	Y		Calendar year-quarter label YYYY-Qn.	derived
25	calendar_year	smallint	Y		Calendar year.	derived
26	fiscal_year	smallint	Y		Green Basket fiscal year (1 April - 31 March) named by its end year, e.g. FY2026 = Apr 2025-Mar 2026.	derived
27	fiscal_year_label	character(6)	Y		Fiscal year label FYyyyy.	derived
28	fiscal_quarter_num	smallint	Y		Fiscal quarter 1-4 (FQ1 = Apr-Jun).	derived
29	fiscal_quarter_label	character(10)	Y		Fiscal quarter label, e.g. FY2026-FQ1.	derived
30	fiscal_month_num	smallint	Y		Month number within the fiscal year (April = 1).	derived
31	fiscal_year_start	date	Y		First day of the fiscal year.	derived
32	fiscal_year_end	date	Y		Last day of the fiscal year.	derived
33	crop_season_start_year	smallint	Y		Start year of the agricultural season (1 October - 30 September).	derived
34	crop_season_label	character(7)	Y		Agricultural season label, e.g. 2023/24 (NSA/MAFWLR convention).	derived
35	season_month_num	smallint	Y		Month number within the agricultural season (October = 1).	derived
36	is_current_month	boolean	Y		True for days in the month of the data cut-off (30 Sep 2026).	derived
37	dw_created_at	timestamp with time zone	N		Audit: timestamp the row was created.	ETL

dw.dim_farm

Farm unit. SCD Type 2 on cultivated_ha, irrigation_type, water_source, farm_manager_code and GlobalG.A.P. status; Type 1 on name/location. Snowflaked to dim_region.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	farm_key	integer	N	PK	Surrogate key: one value per farm VERSION (-1 Unknown). Facts store the version valid on the event date.	ETL identity
2	farm_code	character varying(10)	N		Durable natural key, e.g. GB-KAW-01.	Master data workbook, sheet Farms
3	farm_name	character varying(60)	N		Farm unit name (Type 1 - corrections overwrite all versions).	Master data workbook, sheet Farms
4	region_key	smallint	N	FK → dim_region.region_key	FK to dim_region (snowflake outrigger).	ETL via ref.region_alias (rule MD01)
5	town	character varying(40)	Y		Nearest town (Type 1).	Master data workbook

#	Column	Data type	Null	Key	Business definition	Source / lineage
6	latitude	numeric(9,5)	Y		Latitude in decimal degrees (WGS84), used to request NASA POWER weather.	Master data workbook
7	longitude	numeric(9,5)	Y		Longitude in decimal degrees (WGS84).	Master data workbook
8	elevation_m	integer	Y		Elevation in metres.	Master data workbook
9	total_ha	numeric(10,1)	Y		Total farm area in hectares (Type 1).	Master data workbook
10	established_year	smallint	Y		Year the farm unit joined Green Basket (Type 1).	Master data workbook
11	harvest_log_template	character(1)	Y		Layout of the farm's harvest log export (A, B, C).	Master data workbook
12	cultivated_ha	numeric(10,1)	Y		Cultivated (arable) hectares in this version (Type 2).	Master data workbook + Farm_History
13	irrigation_type	character varying(24)	Y		Primary irrigation system in this version: Dryland, Centre pivot, Drip, Drip + shade-net, Flood (canal) (Type 2).	Master data workbook + Farm_History
14	is_irrigated	boolean	Y		True when irrigation_type <> Dryland (Type 2, derived).	derived
15	water_source	character varying(40)	Y		Main water source in this version (Type 2).	Master data workbook + Farm_History
16	farm_manager_code	character varying(10)	Y		Pseudonymised employee code of the farm manager in this version (Type 2; no names stored).	Master data workbook + Farm_History
17	is_globalgap_certified	boolean	Y		GlobalG.A.P. certification status in this version; gates export-grade sales (Type 2).	Master data workbook + Farm_History
18	scd_effective_from	date	N		First date this version is valid (1900-01-01 for the first version).	ETL (SCD2)
19	scd_effective_to	date	N		Last date this version is valid (9999-12-31 for the current version).	ETL (SCD2)
20	scd_is_current	boolean	N		True for the current version (exactly one per farm_code).	ETL (SCD2)
21	scd_version	smallint	N		Version sequence number per farm_code.	ETL (SCD2)
22	scd_row_hash	character(32)	N		MD5 of the Type 2 attributes, used for change detection and idempotent re-runs.	ETL (SCD2)
23	dw_batch_id	integer	Y		Audit: ETL batch that created/last updated the row.	ETL
24	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL
25	dw_updated_at	timestamp with time zone	N		Audit: last update timestamp.	ETL

dw.dim_product

Saleable SKU (crop x grade x pack). Lowest level of the snowflake product → crop → category. SCD Type 1 (attributes are corrections, not business changes).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	product_key	integer	N	PK	Surrogate key (identity; -1 Unknown).	ETL

#	Column	Data type	Null	Key	Business definition	Source / lineage
2	product_code	character varying(16)	N		ERP product code (natural key), e.g. TOM-C1-C10.	Master data workbook, sheet Products
3	product_name	character varying(60)	N		Product description.	Master data workbook, sheet Products
4	crop_key	smallint	N	FK → dim_crop.crop_key	FK to dim_crop (snowflake).	ETL lookup on crop_code
5	grade	character varying(12)	Y		Quality grade (WM1/WM2 maize grades, B1/B3 wheat grades, Class 1/2, Export/Local).	Master data workbook
6	pack_type	character varying(24)	Y		Packaging format (Bulk, 10kg crate, 4.5kg carton …).	Master data workbook
7	sales_uom	character varying(8)	Y		Unit of measure used on invoices (t, crate, bag, head …).	Master data workbook
8	net_weight_kg	numeric(10,3)	Y		Net kg per pack unit as recorded in master data (NULL for bulk).	Master data workbook
9	kg_per_sales_uom	numeric(10,3)	N		Conversion factor from invoice quantity to kg (1000 for tonnes).	derived (rule MD03 imputes missing weights)
10	is_export_grade	boolean	Y		True for export-grade SKUs (requires GlobalG.A.P.-certified farm).	Master data workbook
11	is_active	boolean	Y		Product currently sold.	Master data workbook
12	net_weight_imputed	boolean	N		True when kg_per_sales_uom was imputed by rule MD03.	ETL
13	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL
14	dw_updated_at	timestamp with time zone	N		Audit: last Type 1 update timestamp.	ETL

dw.dim_region

Conformed geography (Namibia's 14 regions) with management zone. Outrigger of dim_farm/dim_customer and the region grain of fact_crop_estimate. SCD Type 1.

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	region_key	smallint	N	PK	Surrogate key (-1 = Unknown).	ETL
2	region_code	character(5)	N		ISO 3166-2:NA region code (natural key), e.g. NA-KE.	Master data workbook, sheet Regions
3	region_name	character varying(30)	N		Official region name.	Master data workbook, sheet Regions
4	zone_code	character(2)	N		Green Basket management zone code (NC, NE, NW, CE, SO); drives row-level security.	Master data workbook, sheet Regions
5	zone_name	character varying(20)	N		Management zone name (North-Central, North-East, North-West, Central, South).	Master data workbook
6	capital_town	character varying(30)	Y		Regional capital.	Master data workbook
7	is_northern_crop_region	boolean	N		True for the northern communal crop-growing regions (NC, NE, NW zones) used in NSA crop reports.	derived from zone_code
8	dw_created_at	timestamp with time zone	N		Audit: row creation timestamp.	ETL
9	dw_updated_at	timestamp with time zone	N		Audit: last Type 1 update timestamp.	ETL

dw.harvest_anomaly

Model output: harvest records scored as anomalous by Isolation Forest / robust z-score (for steward review).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	source_file	character varying(40)	N	PK	Lineage: harvest log file.	anomaly_detection.py
2	source_row_num	integer	N	PK	Lineage: row in the file.	anomaly_detection.py
3	farm_code	character varying(10)	Y		Farm code.	anomaly_detection.py
4	crop_code	character(3)	Y		Crop code.	anomaly_detection.py
5	harvest_date	date	Y		Harvest date.	anomaly_detection.py
6	harvested_kg	numeric(14,1)	Y		Harvested kg as loaded.	anomaly_detection.py
7	anomaly_score	numeric(10,4)	Y		Anomaly score (higher = more anomalous).	anomaly_detection.py
8	method	character varying(30)	N	PK	Detection method.	anomaly_detection.py

dw.yield_driver_importance

Model output: yield-driver coefficients (OLS) and permutation importances (random forest).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	model_name	character varying(40)	N	PK	Model identifier.	yield_drivers.py
2	feature	character varying(40)	N	PK	Explanatory feature.	yield_drivers.py
3	importance	numeric(10,4)	Y		Permutation importance (drop in R2).	yield_drivers.py
4	coefficient	numeric(12,4)	Y		OLS coefficient on log-yield (approx. % effect).	yield_drivers.py
5	p_value	numeric(10,6)	Y		OLS p-value.	yield_drivers.py
6	rank_order	smallint	Y		Rank by importance.	yield_drivers.py
7	model_run_at	timestamp with time zone	N		Audit: model run timestamp.	yield_drivers.py

sec.user_zone_access

Maps Power BI user principal names to the management zones they may see (dynamic RLS).

#	Column	Data type	Null	Key	Business definition	Source / lineage
1	user_upn	character varying(80)	N	PK	User principal name (Entra ID sign-in) matched by USERPRINCIPALNAME() in the RLS rule.	Head of Agriculture access request
2	zone_code	character(2)	N	PK	Management zone the user may see (NC, NE, NW, CE, SO); one row per zone granted.	access request
3	granted_by	character varying(40)	N		Role that approved the access (accountability).	access request
4	valid_from	date	N		Date the access became effective (recertified quarterly).	access request

Appendix B – All DAX measures

All 79 measures of the semantic model, grouped by display folder. They are generated from `04_powerbi/model_spec.py`, the same source that builds the TMDL, and are also delivered as `04_powerbi/measures.dax`.

Folder 0 Labels

```
-- Freshness label for page headers.
Data As Of =
    "Data to " & FORMAT ( [Last Sales Date], "d mmm yyyy" )

-- Dynamic title.
Title Revenue Trend =
    "Net revenue by month - " & SELECTEDVALUE ( 'Date'[Fiscal Year], "all fiscal years" )

-- Dynamic title.
Title Yield =
    "Yield vs plan and NSA benchmark - " & SELECTEDVALUE ( 'Crop'[Crop], "all crops" )
```

Folder 1 Sales

```
-- Invoice + credit-note revenue after discount, N$ (FX-converted).
Net Revenue =
    SUM ( 'Sales'[Net Revenue] )

-- Revenue before discount, N$.
Gross Revenue =
    SUM ( 'Sales'[Gross Revenue] )

-- Discounts granted, N$.
Discount =
    SUM ( 'Sales'[Discount] )

-- Standard cost of goods sold, N$.
COGS =
    SUM ( 'Sales'[COGS] )

-- Outbound freight borne by Green Basket, N$.
Freight Cost =
    SUM ( 'Sales'[Freight Cost] )

-- Net revenue less COGS.
Gross Margin =
    [Net Revenue] - [COGS]

-- Gross margin as a share of net revenue.
Gross Margin % =
    DIVIDE ( [Gross Margin], [Net Revenue] )

-- Gross margin less outbound freight.
Contribution after Freight =
    [Gross Margin] - [Freight Cost]

-- Tonnes invoiced (net of credit notes).
Sales Volume (t) =
    DIVIDE ( SUM ( 'Sales'[Quantity (kg)] ), 1000 )

-- Net revenue per kg sold.
Avg Realised Price (N$/kg) =
    DIVIDE ( [Net Revenue], SUM ( 'Sales'[Quantity (kg)] ) )

-- Number of document lines.
Invoice Lines =
    COUNTROWS ( 'Sales' )

-- Distinct invoices and credit notes.
Invoices =
    DISTINCTCOUNT ( 'Sales'[Invoice ID] )

-- Distinct customers (durable code) with sales in the context.
Active Customers =
    CALCULATE ( DISTINCTCOUNT ( 'Customer'[Customer Code] ), 'Sales' )

-- Value credited for quality claims (positive number).
Credit Notes =
    - CALCULATE ( [Net Revenue], KEEPFILTERS ( 'Sales'[Document Type] = "CRN" ) )

-- Credit notes as a share of invoiced revenue.
Credit Note Rate % =
    DIVIDE ( [Credit Notes], CALCULATE ( [Net Revenue], KEEPFILTERS ( 'Sales'[Document Type] = "INV" ) ) )

-- Net revenue from customers in the Export channel (CALCULATE with filter modification).
Export Revenue =
    CALCULATE ( [Net Revenue], KEEPFILTERS ( 'Customer'[Channel] = "Export" ) )

-- Export revenue as a share of net revenue.
Export Share % =
    DIVIDE ( [Export Revenue], [Net Revenue] )

-- Share of the selection in total revenue across all crops (filters on the product hierarchy removed).
Revenue Share of All Crops % =
    DIVIDE (
        [Net Revenue],
        CALCULATE ( [Net Revenue], REMOVEFILTERS ( 'Product' ), REMOVEFILTERS ( 'Crop' ), REMOVEFILTERS ( 'Crop Category' ) )
    )
```

```
-- Revenue concentration: share of the ten largest customers in the context.
Top 10 Customer Share % =
    VAR _top10 =
        TOPN ( 10, VALUES ( 'Customer'[Customer Code] ), [Net Revenue] )
    RETURN
        DIVIDE ( CALCULATE ( [Net Revenue], _top10 ), [Net Revenue] )
```

Folder 2 Time Intelligence

```
-- Latest invoice date in the model (ignores date filters).
Last Sales Date =
    VAR _k =
        CALCULATE ( MAX ( 'Sales'[Invoice Date Key] ), REMOVEFILTERS ( 'Date' ) )
    RETURN
        IF ( NOT ISBLANK ( _k ), DATE ( INT ( _k / 10000 ), MOD ( INT ( _k / 100 ), 100 ), MOD ( _k, 100 ) ) )

-- Fiscal year-to-date (year ends 31 March), no future guard.
Net Revenue FYTD Raw =
    TOTALYTD ( [Net Revenue], 'Date'[Date], "3/31" )

-- Fiscal year-to-date net revenue; blank for periods after the last sale.
Net Revenue FYTD =
    IF ( MIN ( 'Date'[Date] ) <= [Last Sales Date], [Net Revenue FYTD Raw] )

-- Same fiscal-YTD window one year earlier, cut at the same day (like-for-like).
Net Revenue FYTD PY =
    VAR _cutoffPY =
        EDATE ( [Last Sales Date], -12 )
    RETURN
        CALCULATE (
            [Net Revenue FYTD Raw],
            SAMEPERIODLASTYEAR ( 'Date'[Date] ),
            KEEPFILTERS ( 'Date'[Date] <= _cutoffPY )
        )

-- Fiscal-YTD growth versus the like-for-like prior-year window.
Net Revenue FYTD YoY % =
    DIVIDE ( [Net Revenue FYTD] - [Net Revenue FYTD PY], [Net Revenue FYTD PY] )

-- Net revenue in the same period last year.
Net Revenue PY =
    CALCULATE ( [Net Revenue], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

-- Change versus the same period last year, N$.
Net Revenue YoY =
    VAR _cur = [Net Revenue]
    VAR _py = [Net Revenue PY]
    RETURN
        IF ( NOT ISBLANK ( _cur ) && NOT ISBLANK ( _py ), _cur - _py )

-- Growth versus the same period last year.
Net Revenue YoY % =
    VAR _cur = [Net Revenue]
    VAR _py = [Net Revenue PY]
    RETURN
        IF ( NOT ISBLANK ( _cur ) && NOT ISBLANK ( _py ), DIVIDE ( _cur - _py, _py ) )

-- Moving annual total: trailing 12 months ending at the context end (capped at the last sale).
Net Revenue MAT =
    VAR _end =
        MIN ( MAX ( 'Date'[Date] ), [Last Sales Date] )
    RETURN
        IF (
            MIN ( 'Date'[Date] ) <= [Last Sales Date],
            CALCULATE ( [Net Revenue], DATESINPERIOD ( 'Date'[Date], _end, -12, MONTH ) )
        )

-- Average monthly net revenue over the trailing three months.
Net Revenue 3M Rolling Avg =
    VAR _end =
        MIN ( MAX ( 'Date'[Date] ), [Last Sales Date] )
    RETURN
        IF (
            MIN ( 'Date'[Date] ) <= [Last Sales Date],
            DIVIDE ( CALCULATE ( [Net Revenue], DATESINPERIOD ( 'Date'[Date], _end, -3, MONTH ) ), 3 )
        )

-- Gross margin % in the same period last year.
Gross Margin % PY =
    CALCULATE ( [Gross Margin %], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

-- Tonnes by DISPATCH date - activates the inactive role-playing relationship.
Dispatched Volume (t) =
    CALCULATE (
        DIVIDE ( SUM ( 'Sales'[Quantity (kg)] ), 1000 ),
        USERELATIONSHIP ( 'Sales'[Dispatch Date Key], 'Date'[Date Key] ),
        KEEPFILTERS ( 'Sales'[Document Type] = "INV" )
    )
```

Folder 3 Production & Yield

```
-- Field harvest in tonnes by harvest date.
Harvest (t) =
    DIVIDE ( SUM ( 'Harvest'[Harvested (kg)] ), 1000 )

-- Area planted (by planting date).
Planted Area (ha) =
```

```
    SUM ( 'Crop Cycle'[Planted (ha)] )

-- Harvest of completed cycles, by agricultural season of harvest.
Completed Harvest (t) =
    CALCULATE (
        DIVIDE ( SUM ( 'Crop Cycle'[Harvested (kg)] ), 1000 ),
        KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Completed" ),
        USERELATIONSHIP ( 'Crop Cycle'[Harvest Season End Date Key], 'Date'[Date Key] )
    )

-- Harvested area of completed cycles, by season of harvest.
Harvested Area (ha) =
    CALCULATE (
        SUM ( 'Crop Cycle'[Harvested (ha)] ),
        KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Completed" ),
        USERELATIONSHIP ( 'Crop Cycle'[Harvest Season End Date Key], 'Date'[Date Key] )
    )

-- Tonnes per harvested hectare (completed cycles).
Yield (t/ha) =
    DIVIDE ( [Completed Harvest (t)], [Harvested Area (ha)] )

-- Budgeted yield of the same completed cycles.
Planned Yield (t/ha) =
    CALCULATE (
        DIVIDE ( SUM ( 'Crop Cycle'[Planned Production (kg)] ) / 1000, SUM ( 'Crop Cycle'[Planted (ha)] ) ),
        KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Completed" ),
        USERELATIONSHIP ( 'Crop Cycle'[Harvest Season End Date Key], 'Date'[Date Key] )
    )

-- Actual yield relative to plan.
Yield vs Plan % =
    VAR _plan = [Planned Yield (t/ha)]
    RETURN
        IF ( NOT ISBLANK ( _plan ), DIVIDE ( [Yield (t/ha)], _plan ) - 1 )

-- NSA commercial-sector yield for the same region/crop/season (drill-across).
Benchmark Yield (t/ha) =
    VAR _prod =
        CALCULATE ( SUM ( 'Crop Estimate'[Production (t)] ), KEEPFILTERS ( 'Crop Estimate'[Farming Sector] = "Commercial" ) )
    VAR _area =
        CALCULATE ( SUM ( 'Crop Estimate'[Area Planted (ha)] ), KEEPFILTERS ( 'Crop Estimate'[Farming Sector] = "Commercial" ) )
    RETURN
        DIVIDE ( _prod, _area )

-- Green Basket yield minus the NSA commercial benchmark.
Yield Gap vs Benchmark (t/ha) =
    VAR _y = [Yield (t/ha)]
    VAR _b = [Benchmark Yield (t/ha)]
    RETURN
        IF ( NOT ISBLANK ( _y ) && NOT ISBLANK ( _b ), _y - _b )

-- Crop-cycle direct cost per tonne harvested (completed cycles).
Direct Cost per Tonne =
    CALCULATE (
        DIVIDE ( SUM ( 'Crop Cycle'[Direct Cost] ), SUM ( 'Crop Cycle'[Harvested (kg)] ) / 1000 ),
        KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Completed" ),
        USERELATIONSHIP ( 'Crop Cycle'[Harvest Season End Date Key], 'Date'[Date Key] )
    )

-- Posted production cost (incl. overheads) by posting date.
Production Cost =
    SUM ( 'Production Cost'[Amount] )

-- Fertiliser applied per planted hectare.
Fertiliser Rate (kg/ha) =
    DIVIDE ( SUM ( 'Crop Cycle'[Fertiliser (kg)] ), SUM ( 'Crop Cycle'[Planted (ha)] ) )

-- Share of planted area abandoned (crop failure).
Failed Area % =
    DIVIDE ( CALCULATE ( [Planted Area (ha)], KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Failed" ) ), [Planted Area (ha)] )

-- Area still growing or being harvested.
Area in Progress (ha) =
    CALCULATE ( [Planted Area (ha)], KEEPFILTERS ( 'Crop Cycle'[Cycle Status] IN { "Growing", "Harvesting" } ) )

-- Distinct crop cycles completed, by last-harvest date.
Cycles Completed =
    CALCULATE (
        DISTINCTCOUNT ( 'Crop Cycle'[Cycle ID] ),
        KEEPFILTERS ( 'Crop Cycle'[Cycle Status] = "Completed" ),
        USERELATIONSHIP ( 'Crop Cycle'[Last Harvest Date Key], 'Date'[Date Key] )
    )
```

Folder 4 Inventory

```
-- Stock on hand at the LAST month-end snapshot in the period across all farms/products (semi-additive).
Closing Stock (t) =
    VAR _last =
        CALCULATE (
            MAX ( 'Inventory'[Snapshot Date Key] ),
            REMOVEFILTERS ( 'Farm' ), REMOVEFILTERS ( 'Region' ), REMOVEFILTERS ( 'Product' ), REMOVEFILTERS ( 'Crop' ), REMOVEFILTERS ( 'Crop
Category' )
        )
    RETURN
        DIVIDE ( CALCULATE ( SUM ( 'Inventory'[Closing Stock (kg)] ), 'Inventory'[Snapshot Date Key] = _last ), 1000 )

-- Closing stock at standard cost, last snapshot in the period.
Closing Stock Value =
```

```
    VAR _last =
        CALCULATE (
            MAX ( 'Inventory'[Snapshot Date Key] ),
            REMOVEFILTERS ( 'Farm' ), REMOVEFILTERS ( 'Region' ), REMOVEFILTERS ( 'Product' ), REMOVEFILTERS ( 'Crop' ), REMOVEFILTERS ( 'Crop
Category' )
        )
    RETURN
        CALCULATE ( SUM ( 'Inventory'[Closing Stock Value] ), 'Inventory'[Snapshot Date Key] = _last )

-- Stock on hand at the FIRST snapshot in the period.
Opening Stock (t) =
    VAR _first =
        CALCULATE (
            MIN ( 'Inventory'[Snapshot Date Key] ),
            REMOVEFILTERS ( 'Farm' ), REMOVEFILTERS ( 'Region' ), REMOVEFILTERS ( 'Product' ), REMOVEFILTERS ( 'Crop' ), REMOVEFILTERS ( 'Crop
Category' )
        )
    RETURN
        DIVIDE ( CALCULATE ( SUM ( 'Inventory'[Opening Stock (kg)] ), 'Inventory'[Snapshot Date Key] = _first ), 1000 )

-- Graded produce received into stock.
Stock Received (t) =
    DIVIDE ( SUM ( 'Inventory'[Received (kg)] ), 1000 )

-- Produce dispatched per the stock system.
Stock Dispatched (t) =
    DIVIDE ( SUM ( 'Inventory'[Dispatched (kg)] ), 1000 )

-- Post-harvest losses.
Shrinkage (t) =
    DIVIDE ( SUM ( 'Inventory'[Shrinkage (kg)] ), 1000 )

-- Losses relative to stock available (opening + received).
Shrinkage % =
    DIVIDE ( [Shrinkage (t)], [Opening Stock (t)] + [Stock Received (t)] )

-- Average tonnes invoiced per day over the last 90 days of the context.
Avg Daily Dispatch L90D (t) =
    VAR _end =
        MIN ( MAX ( 'Date'[Date] ), [Last Sales Date] )
    RETURN
        DIVIDE (
            CALCULATE (
                SUM ( 'Sales'[Quantity (kg)] ),
                DATESINPERIOD ( 'Date'[Date], _end, -90, DAY ),
                KEEPFILTERS ( 'Sales'[Document Type] = "INV" )
            ),
            90 * 1000
        )

-- Days of sales covered by closing stock.
Stock Cover (days) =
    DIVIDE ( [Closing Stock (t)], [Avg Daily Dispatch L90D (t)] )
```

Folder 5 Weather & Markets

```
-- Rainfall total per location, averaged across the selected farm locations.
Rainfall (mm) =
    AVERAGEX ( VALUES ( 'Farm'[Farm Code] ), CALCULATE ( SUM ( 'Weather'[Rainfall (mm)] ) ) )

-- Rainfall per agricultural season (Oct-Sep) per location: comparable whether one or several seasons are selected.
Rainfall per Season (mm) =
    VAR _seasons =
        CALCULATE ( DISTINCTCOUNT ( 'Date'[Crop Season] ), 'Weather' )
    RETURN
        DIVIDE ( [Rainfall (mm)], _seasons )

-- Days with Tmax >= 35 C per location, averaged across locations.
Heat Days =
    AVERAGEX ( VALUES ( 'Farm'[Farm Code] ), CALCULATE ( SUM ( 'Weather'[Heat Day] ) ) )

-- Mean daily maximum temperature.
Avg Max Temp (C) =
    AVERAGE ( 'Weather'[Max Temp (C)] )

-- Average international price converted to N$ (select one crop/commodity).
World Price (N$/t) =
    AVERAGE ( 'Market Price'[Price (N$)] )

-- Average international price in USD per published unit.
World Price (USD) =
    AVERAGE ( 'Market Price'[Price (USD)] )

-- Green Basket realised price per tonne.
Realised Price (N$/t) =
    [Avg Realised Price (N$/kg)] * 1000

-- Realised price relative to the international price (one crop selected).
Local Premium over World Price % =
    VAR _w = [World Price (N$/t)]
    RETURN
        IF ( HASONEVALUE ( 'Crop'[Crop] ) && NOT ISBLANK ( _w ), DIVIDE ( [Realised Price (N$/t)], _w ) - 1 )
```

Folder 6 Forecast

```
-- Forward forecast of net revenue for the selected model (default: seasonal naive).
Forecast Revenue =
    VAR _model =
```

```
        SELECTEDVALUE ( 'Forecast'[Model], "Seasonal naive" )
    RETURN
        CALCULATE (
            SUM ( 'Forecast'[Forecast] ),
            KEEPFILTERS ( 'Forecast'[Scenario] = "Forecast" ),
            'Forecast'[Model] = _model
        )

-- Lower bound of the 80 % prediction interval (sum of category bounds - approximate at total level).
Forecast Lower 80 =
    VAR _model =
        SELECTEDVALUE ( 'Forecast'[Model], "Seasonal naive" )
    RETURN
        CALCULATE ( SUM ( 'Forecast'[Lower 80] ), KEEPFILTERS ( 'Forecast'[Scenario] = "Forecast" ), 'Forecast'[Model] = _model )

-- Upper bound of the 80 % prediction interval.
Forecast Upper 80 =
    VAR _model =
        SELECTEDVALUE ( 'Forecast'[Model], "Seasonal naive" )
    RETURN
        CALCULATE ( SUM ( 'Forecast'[Upper 80] ), KEEPFILTERS ( 'Forecast'[Scenario] = "Forecast" ), 'Forecast'[Model] = _model )

-- Hold-out (Oct 2025 - Sep 2026) forecast of the selected model.
Backtest Forecast =
    VAR _model =
        SELECTEDVALUE ( 'Forecast'[Model], "Seasonal naive" )
    RETURN
        CALCULATE ( SUM ( 'Forecast'[Forecast] ), KEEPFILTERS ( 'Forecast'[Scenario] = "Backtest" ), 'Forecast'[Model] = _model )

-- Weighted absolute percentage error of the hold-out forecast (category grain).
Backtest WAPE % =
    VAR _model =
        SELECTEDVALUE ( 'Forecast'[Model], "Seasonal naive" )
    VAR _rows =
        FILTER ( 'Forecast', 'Forecast'[Scenario] = "Backtest" && 'Forecast'[Model] = _model )
    RETURN
        DIVIDE ( SUMX ( _rows, ABS ( 'Forecast'[Actual] - 'Forecast'[Forecast] ) ), SUMX ( _rows, ABS ( 'Forecast'[Actual] ) ) )

-- Actual net revenue up to the last sale, forecast afterwards (single continuous line).
Revenue (Actual + Forecast) =
    IF ( MAX ( 'Date'[Date] ) <= [Last Sales Date], [Net Revenue], [Forecast Revenue] )

-- Permutation importance of yield drivers (random forest).
Driver Importance =
    SUM ( 'Yield Drivers'[Importance] )
```

Folder 7 Data Quality & Security

```
-- Invoice lines flagged by rule SA07.
Price Anomaly Lines =
    CALCULATE ( COUNTROWS ( 'Sales' ), KEEPFILTERS ( 'Sales'[Price Anomaly] = TRUE () ) )

-- Revenue on lines awaiting steward review.
Flagged Price Anomaly Revenue =
    CALCULATE ( [Net Revenue], KEEPFILTERS ( 'Sales'[Price Anomaly] = TRUE () ) )

-- Revenue on lines without farm of origin (rule SA05).
Unknown Farm Revenue % =
    DIVIDE ( CALCULATE ( [Net Revenue], KEEPFILTERS ( 'Farm'[Farm Code] = "UNKNOWN" ) ), [Net Revenue] )

-- Harvest records corrected by rule HV06.
Corrected Harvest Records =
    CALCULATE ( COUNTROWS ( 'Harvest' ), KEEPFILTERS ( 'Harvest'[Is Corrected] = TRUE () ) )

-- Farms visible to the current user (RLS test).
Visible Farms =
    CALCULATE ( DISTINCTCOUNT ( 'Farm'[Farm Code] ), KEEPFILTERS ( 'Farm'[Farm Code] <> "UNKNOWN" ) )

-- UPN evaluated by row-level security.
Current User =
    USERPRINCIPALNAME ()
```

Appendix C – Warehouse DDL

`02_sql/03_ddl_warehouse.sql` is reproduced in full. The complete set of scripts (staging, transformations, loads, reconciliation, marts, security and optimisation) is in `02_sql/`, and a schema-only dump of the loaded database is in `05_backup/green_basket_dw_schema_ddl.sql`.

File: `02_sql/03_ddl_warehouse.sql` (lines 1–750)

```
-- ============================================================================
-- 03_ddl_warehouse.sql
-- Dimensional warehouse (Kimball bus architecture) for Green Basket cc.
--   Conformed dimensions : dim_date, dim_region, dim_crop_category, dim_crop,
--                          dim_product, dim_farm (SCD2), dim_customer (SCD2),
--                          dim_cost_category, dim_commodity
--   Snowflaked hierarchies: dim_product -> dim_crop -> dim_crop_category
--                           dim_farm    -> dim_region ; dim_customer -> dim_region
--   Facts: fact_sales (transaction), fact_harvest (transaction),
--          fact_production_cost (transaction), fact_crop_cycle (accumulating
--          snapshot), fact_inventory_monthly (periodic snapshot, semi-additive),
--          fact_weather_daily (periodic), fact_market_price_monthly (periodic,
--          non-additive), fact_crop_estimate (season benchmark, external)
-- Special members: -1 = Unknown, -2 = Not applicable (e.g. farm overhead cost).
-- Every column carries a COMMENT "definition | Source: ..." that feeds the
-- generated data dictionary (see 03_analytics/build_data_dictionary.py).
-- ============================================================================

-- ============================================================ DIMENSIONS
CREATE TABLE dw.dim_date (
    date_key              integer PRIMARY KEY,
    full_date             date UNIQUE,
    day_of_month          smallint,
    day_of_year           smallint,
    day_of_week_iso       smallint,
    day_name              varchar(9),
    day_short             char(3),
    is_weekend            boolean,
    is_public_holiday     boolean,
    holiday_name          varchar(40),
    is_working_day        boolean,
    iso_week              smallint,
    week_start_date       date,
    month_num             smallint,
    month_name            varchar(9),
    month_short           char(3),
    year_month            char(7),
    year_month_key        integer,
    month_start_date      date,
    month_end_date        date,
    is_month_end          boolean,
    days_in_month         smallint,
    quarter_num           smallint,
    year_quarter          char(7),
    calendar_year         smallint,
    fiscal_year           smallint,
    fiscal_year_label     char(6),
    fiscal_quarter_num    smallint,
    fiscal_quarter_label  char(10),
    fiscal_month_num      smallint,
    fiscal_year_start     date,
    fiscal_year_end       date,
    crop_season_start_year smallint,
    crop_season_label     char(7),
    season_month_num      smallint,
    is_current_month      boolean,
    dw_created_at         timestamptz NOT NULL DEFAULT now()
);
COMMENT ON TABLE dw.dim_date IS 'Conformed calendar dimension, one row per day 2020-01-01..2027-12-31 plus -1 Unknown; carries calendar, fiscal (Apr-Mar) and agricultural season (Oct-Sep) hierarchies.';
COMMENT ON COLUMN dw.dim_date.date_key IS 'Smart surrogate key YYYYMMDD (-1 = unknown / not yet occurred). | Source: generated by etl.sp_load_dim_date';
COMMENT ON COLUMN dw.dim_date.full_date IS 'Calendar date. | Source: generated';
COMMENT ON COLUMN dw.dim_date.day_of_month IS 'Day number within the month (1-31). | Source: derived';
COMMENT ON COLUMN dw.dim_date.day_of_year IS 'Day number within the calendar year (1-366). | Source: derived';
COMMENT ON COLUMN dw.dim_date.day_of_week_iso IS 'ISO weekday number, Monday = 1 ... Sunday = 7 (sort key for day_name). | Source: derived';
COMMENT ON COLUMN dw.dim_date.day_name IS 'Weekday name (Monday..Sunday). | Source: derived';
COMMENT ON COLUMN dw.dim_date.day_short IS 'Three-letter weekday abbreviation. | Source: derived';
COMMENT ON COLUMN dw.dim_date.is_weekend IS 'True for Saturday and Sunday. | Source: derived';
COMMENT ON COLUMN dw.dim_date.is_public_holiday IS 'True on Namibian public holidays (Public Holidays Act 26 of 1990 incl. Sunday->Monday rule). | Source: etl.fn_namibian_holiday';
COMMENT ON COLUMN dw.dim_date.holiday_name IS 'Name of the Namibian public holiday, NULL otherwise. | Source: etl.fn_namibian_holiday';
COMMENT ON COLUMN dw.dim_date.is_working_day IS 'Farm working day: Monday-Saturday and not a public holiday. | Source: derived';
COMMENT ON COLUMN dw.dim_date.iso_week IS 'ISO-8601 week number. | Source: derived';
COMMENT ON COLUMN dw.dim_date.week_start_date IS 'Monday of the ISO week. | Source: derived';
COMMENT ON COLUMN dw.dim_date.month_num IS 'Calendar month number 1-12 (sort key for month_name). | Source: derived';
COMMENT ON COLUMN dw.dim_date.month_name IS 'Calendar month name. | Source: derived';
COMMENT ON COLUMN dw.dim_date.month_short IS 'Three-letter month abbreviation. | Source: derived';
COMMENT ON COLUMN dw.dim_date.year_month IS 'Year-month label YYYY-MM. | Source: derived';
COMMENT ON COLUMN dw.dim_date.year_month_key IS 'Integer YYYYMM (sort key for year_month). | Source: derived';
COMMENT ON COLUMN dw.dim_date.month_start_date IS 'First day of the month. | Source: derived';
COMMENT ON COLUMN dw.dim_date.month_end_date IS 'Last day of the month (key used by month-grain facts). | Source: derived';
COMMENT ON COLUMN dw.dim_date.is_month_end IS 'True on the last day of the month. | Source: derived';
COMMENT ON COLUMN dw.dim_date.days_in_month IS 'Number of days in the month. | Source: derived';
COMMENT ON COLUMN dw.dim_date.quarter_num IS 'Calendar quarter 1-4. | Source: derived';
COMMENT ON COLUMN dw.dim_date.year_quarter IS 'Calendar year-quarter label YYYY-Qn. | Source: derived';
COMMENT ON COLUMN dw.dim_date.calendar_year IS 'Calendar year. | Source: derived';
COMMENT ON COLUMN dw.dim_date.fiscal_year IS 'Green Basket fiscal year (1 April - 31 March) named by its end year, e.g. FY2026 = Apr
```

```
2025-Mar 2026. | Source: derived';
        COMMENT ON COLUMN dw.dim_date.fiscal_year_label IS 'Fiscal year label FYyyyy. | Source: derived';
        COMMENT ON COLUMN dw.dim_date.fiscal_quarter_num IS 'Fiscal quarter 1-4 (FQ1 = Apr-Jun). | Source: derived';
        COMMENT ON COLUMN dw.dim_date.fiscal_quarter_label IS 'Fiscal quarter label, e.g. FY2026-FQ1. | Source: derived';
        COMMENT ON COLUMN dw.dim_date.fiscal_month_num IS 'Month number within the fiscal year (April = 1). | Source: derived';
        COMMENT ON COLUMN dw.dim_date.fiscal_year_start IS 'First day of the fiscal year. | Source: derived';
        COMMENT ON COLUMN dw.dim_date.fiscal_year_end IS 'Last day of the fiscal year. | Source: derived';
        COMMENT ON COLUMN dw.dim_date.crop_season_start_year IS 'Start year of the agricultural season (1 October - 30 September). | Source:
derived';
        COMMENT ON COLUMN dw.dim_date.crop_season_label IS 'Agricultural season label, e.g. 2023/24 (NSA/MAFWLR convention). | Source: derived';
        COMMENT ON COLUMN dw.dim_date.season_month_num IS 'Month number within the agricultural season (October = 1). | Source: derived';
        COMMENT ON COLUMN dw.dim_date.is_current_month IS 'True for days in the month of the data cut-off (30 Sep 2026). | Source: derived';
        COMMENT ON COLUMN dw.dim_date.dw_created_at IS 'Audit: timestamp the row was created. | Source: ETL';

        CREATE TABLE dw.dim_region (
            region_key      smallint PRIMARY KEY,
            region_code     char(5) NOT NULL UNIQUE,
            region_name     varchar(30) NOT NULL,
            zone_code       char(2) NOT NULL,
            zone_name       varchar(20) NOT NULL,
            capital_town    varchar(30),
            is_northern_crop_region boolean NOT NULL,
            dw_created_at   timestamptz NOT NULL DEFAULT now(),
            dw_updated_at   timestamptz NOT NULL DEFAULT now()
        );
        COMMENT ON TABLE dw.dim_region IS 'Conformed geography (Namibia''s 14 regions) with management zone. Outrigger of dim_farm/dim_customer
and the region grain of fact_crop_estimate. SCD Type 1.';
        COMMENT ON COLUMN dw.dim_region.region_key IS 'Surrogate key (-1 = Unknown). | Source: ETL';
        COMMENT ON COLUMN dw.dim_region.region_code IS 'ISO 3166-2:NA region code (natural key), e.g. NA-KE. | Source: Master data workbook,
sheet Regions';
        COMMENT ON COLUMN dw.dim_region.region_name IS 'Official region name. | Source: Master data workbook, sheet Regions';
        COMMENT ON COLUMN dw.dim_region.zone_code IS 'Green Basket management zone code (NC, NE, NW, CE, SO); drives row-level security. |
Source: Master data workbook, sheet Regions';
        COMMENT ON COLUMN dw.dim_region.zone_name IS 'Management zone name (North-Central, North-East, North-West, Central, South). | Source:
Master data workbook';
        COMMENT ON COLUMN dw.dim_region.capital_town IS 'Regional capital. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_region.is_northern_crop_region IS 'True for the northern communal crop-growing regions (NC, NE, NW zones) used
in NSA crop reports. | Source: derived from zone_code';
        COMMENT ON COLUMN dw.dim_region.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';
        COMMENT ON COLUMN dw.dim_region.dw_updated_at IS 'Audit: last Type 1 update timestamp. | Source: ETL';

        CREATE TABLE dw.dim_crop_category (
            crop_category_key smallint PRIMARY KEY,
            category_code     char(3) NOT NULL UNIQUE,
            category_name     varchar(30) NOT NULL,
            sector            varchar(20) NOT NULL,
            dw_created_at     timestamptz NOT NULL DEFAULT now()
        );
        COMMENT ON TABLE dw.dim_crop_category IS 'Top level of the snowflaked product hierarchy (Cereals & grains, Vegetables, Fruit). SCD Type
1.';
        COMMENT ON COLUMN dw.dim_crop_category.crop_category_key IS 'Surrogate key (-1 Unknown, -2 Not applicable). | Source: ETL';
        COMMENT ON COLUMN dw.dim_crop_category.category_code IS 'Category code (natural key): CER, VEG, FRU. | Source: Master data workbook,
sheet Crop_Categories';
        COMMENT ON COLUMN dw.dim_crop_category.category_name IS 'Category name. | Source: Master data workbook, sheet Crop_Categories';
        COMMENT ON COLUMN dw.dim_crop_category.sector IS 'Agricultural sector: Field crops or Horticulture. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_crop_category.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';

        CREATE TABLE dw.dim_crop (
            crop_key          smallint PRIMARY KEY,
            crop_code         char(3) NOT NULL UNIQUE,
            crop_name         varchar(40) NOT NULL,
            crop_category_key smallint NOT NULL REFERENCES dw.dim_crop_category (crop_category_key),
            scientific_name   varchar(60),
            is_nab_controlled boolean,
            storage_type      varchar(30),
            shelf_life_days   smallint,
            typical_cull_pct  numeric(5, 2),
            dw_created_at     timestamptz NOT NULL DEFAULT now(),
            dw_updated_at     timestamptz NOT NULL DEFAULT now()
        );
        COMMENT ON TABLE dw.dim_crop IS 'Crop (middle level of the product snowflake). Also a conformed shrunken dimension: harvest, cost,
cycle, market-price and NSA facts are recorded at crop grain. SCD Type 1.';
        COMMENT ON COLUMN dw.dim_crop.crop_key IS 'Surrogate key (-1 Unknown, -2 Not crop-specific / farm overhead). | Source: ETL';
        COMMENT ON COLUMN dw.dim_crop.crop_code IS 'Three-letter crop code (natural key), e.g. MZW = white maize. | Source: Master data
workbook, sheet Crops';
        COMMENT ON COLUMN dw.dim_crop.crop_name IS 'Crop name. | Source: Master data workbook, sheet Crops';
        COMMENT ON COLUMN dw.dim_crop.crop_category_key IS 'FK to dim_crop_category (snowflake). | Source: ETL lookup on category_code';
        COMMENT ON COLUMN dw.dim_crop.scientific_name IS 'Botanical name. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_crop.is_nab_controlled IS 'True if the crop is a Namibian Agronomic Board controlled product (border closures /
Market Share Promotion apply). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_crop.storage_type IS 'Post-harvest storage type (Silo, Cold room, ...). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_crop.shelf_life_days IS 'Typical shelf life in days. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_crop.typical_cull_pct IS 'Typical % of harvested mass culled at grading. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_crop.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';
        COMMENT ON COLUMN dw.dim_crop.dw_updated_at IS 'Audit: last Type 1 update timestamp. | Source: ETL';

        CREATE TABLE dw.dim_product (
            product_key       integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
            product_code      varchar(16) NOT NULL UNIQUE,
            product_name      varchar(60) NOT NULL,
            crop_key          smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            grade             varchar(12),
            pack_type         varchar(24),
            sales_uom         varchar(8),
            net_weight_kg     numeric(10, 3),
            kg_per_sales_uom  numeric(10, 3) NOT NULL,
            is_export_grade   boolean,
            is_active         boolean,
            net_weight_imputed boolean NOT NULL DEFAULT false,
            dw_created_at     timestamptz NOT NULL DEFAULT now(),
```

```
            dw_updated_at     timestamptz NOT NULL DEFAULT now()
        );
        COMMENT ON TABLE dw.dim_product IS 'Saleable SKU (crop x grade x pack). Lowest level of the snowflake product -> crop -> category. SCD
Type 1 (attributes are corrections, not business changes).';
        COMMENT ON COLUMN dw.dim_product.product_key IS 'Surrogate key (identity; -1 Unknown). | Source: ETL';
        COMMENT ON COLUMN dw.dim_product.product_code IS 'ERP product code (natural key), e.g. TOM-C1-C10. | Source: Master data workbook, sheet
Products';
        COMMENT ON COLUMN dw.dim_product.product_name IS 'Product description. | Source: Master data workbook, sheet Products';
        COMMENT ON COLUMN dw.dim_product.crop_key IS 'FK to dim_crop (snowflake). | Source: ETL lookup on crop_code';
        COMMENT ON COLUMN dw.dim_product.grade IS 'Quality grade (WM1/WM2 maize grades, B1/B3 wheat grades, Class 1/2, Export/Local). | Source:
Master data workbook';
        COMMENT ON COLUMN dw.dim_product.pack_type IS 'Packaging format (Bulk, 10kg crate, 4.5kg carton ...). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_product.sales_uom IS 'Unit of measure used on invoices (t, crate, bag, head ...). | Source: Master data
workbook';
        COMMENT ON COLUMN dw.dim_product.net_weight_kg IS 'Net kg per pack unit as recorded in master data (NULL for bulk). | Source: Master
data workbook';
        COMMENT ON COLUMN dw.dim_product.kg_per_sales_uom IS 'Conversion factor from invoice quantity to kg (1000 for tonnes). | Source: derived
(rule MD03 imputes missing weights)';
        COMMENT ON COLUMN dw.dim_product.is_export_grade IS 'True for export-grade SKUs (requires GlobalG.A.P.-certified farm). | Source: Master
data workbook';
        COMMENT ON COLUMN dw.dim_product.is_active IS 'Product currently sold. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_product.net_weight_imputed IS 'True when kg_per_sales_uom was imputed by rule MD03. | Source: ETL';
        COMMENT ON COLUMN dw.dim_product.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';
        COMMENT ON COLUMN dw.dim_product.dw_updated_at IS 'Audit: last Type 1 update timestamp. | Source: ETL';

        CREATE TABLE dw.dim_farm (
            farm_key              integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
            farm_code             varchar(10) NOT NULL,
            farm_name             varchar(60) NOT NULL,
            region_key            smallint NOT NULL REFERENCES dw.dim_region (region_key),
            town                  varchar(40),
            latitude              numeric(9, 5),
            longitude             numeric(9, 5),
            elevation_m           integer,
            total_ha              numeric(10, 1),
            established_year      smallint,
            harvest_log_template  char(1),
            cultivated_ha         numeric(10, 1),
            irrigation_type       varchar(24),
            is_irrigated          boolean,
            water_source          varchar(40),
            farm_manager_code     varchar(10),
            is_globalgap_certified boolean,
            scd_effective_from    date NOT NULL,
            scd_effective_to      date NOT NULL DEFAULT DATE '9999-12-31',
            scd_is_current        boolean NOT NULL,
            scd_version           smallint NOT NULL,
            scd_row_hash          char(32) NOT NULL,
            dw_batch_id           integer,
            dw_created_at         timestamptz NOT NULL DEFAULT now(),
            dw_updated_at         timestamptz NOT NULL DEFAULT now(),
            CONSTRAINT uq_dim_farm_version UNIQUE (farm_code, scd_effective_from),
            CONSTRAINT ck_dim_farm_dates CHECK (scd_effective_to >= scd_effective_from)
        );
        CREATE UNIQUE INDEX ux_dim_farm_current ON dw.dim_farm (farm_code) WHERE scd_is_current;
        COMMENT ON TABLE dw.dim_farm IS 'Farm unit. SCD Type 2 on cultivated_ha, irrigation_type, water_source, farm_manager_code and
GlobalG.A.P. status; Type 1 on name/location. Snowflaked to dim_region.';
        COMMENT ON COLUMN dw.dim_farm.farm_key IS 'Surrogate key: one value per farm VERSION (-1 Unknown). Facts store the version valid on the
event date. | Source: ETL identity';
        COMMENT ON COLUMN dw.dim_farm.farm_code IS 'Durable natural key, e.g. GB-KAW-01. | Source: Master data workbook, sheet Farms';
        COMMENT ON COLUMN dw.dim_farm.farm_name IS 'Farm unit name (Type 1 - corrections overwrite all versions). | Source: Master data
workbook, sheet Farms';
        COMMENT ON COLUMN dw.dim_farm.region_key IS 'FK to dim_region (snowflake outrigger). | Source: ETL via ref.region_alias (rule MD01)';
        COMMENT ON COLUMN dw.dim_farm.town IS 'Nearest town (Type 1). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_farm.latitude IS 'Latitude in decimal degrees (WGS84), used to request NASA POWER weather. | Source: Master
data workbook';
        COMMENT ON COLUMN dw.dim_farm.longitude IS 'Longitude in decimal degrees (WGS84). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_farm.elevation_m IS 'Elevation in metres. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_farm.total_ha IS 'Total farm area in hectares (Type 1). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_farm.established_year IS 'Year the farm unit joined Green Basket (Type 1). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_farm.harvest_log_template IS 'Layout of the farm''s harvest log export (A, B, C). | Source: Master data
workbook';
        COMMENT ON COLUMN dw.dim_farm.cultivated_ha IS 'Cultivated (arable) hectares in this version (Type 2). | Source: Master data workbook +
Farm_History';
        COMMENT ON COLUMN dw.dim_farm.irrigation_type IS 'Primary irrigation system in this version: Dryland, Centre pivot, Drip, Drip + shade-
net, Flood (canal) (Type 2). | Source: Master data workbook + Farm_History';
        COMMENT ON COLUMN dw.dim_farm.is_irrigated IS 'True when irrigation_type <> Dryland (Type 2, derived). | Source: derived';
        COMMENT ON COLUMN dw.dim_farm.water_source IS 'Main water source in this version (Type 2). | Source: Master data workbook +
Farm_History';
        COMMENT ON COLUMN dw.dim_farm.farm_manager_code IS 'Pseudonymised employee code of the farm manager in this version (Type 2; no names
stored). | Source: Master data workbook + Farm_History';
        COMMENT ON COLUMN dw.dim_farm.is_globalgap_certified IS 'GlobalG.A.P. certification status in this version; gates export-grade sales
(Type 2). | Source: Master data workbook + Farm_History';
        COMMENT ON COLUMN dw.dim_farm.scd_effective_from IS 'First date this version is valid (1900-01-01 for the first version). | Source: ETL
(SCD2)';
        COMMENT ON COLUMN dw.dim_farm.scd_effective_to IS 'Last date this version is valid (9999-12-31 for the current version). | Source: ETL
(SCD2)';
        COMMENT ON COLUMN dw.dim_farm.scd_is_current IS 'True for the current version (exactly one per farm_code). | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_farm.scd_version IS 'Version sequence number per farm_code. | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_farm.scd_row_hash IS 'MD5 of the Type 2 attributes, used for change detection and idempotent re-runs. | Source:
ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_farm.dw_batch_id IS 'Audit: ETL batch that created/last updated the row. | Source: ETL';
        COMMENT ON COLUMN dw.dim_farm.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';
        COMMENT ON COLUMN dw.dim_farm.dw_updated_at IS 'Audit: last update timestamp. | Source: ETL';

        CREATE TABLE dw.dim_customer (
            customer_key          integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
            customer_code         varchar(10) NOT NULL,
            customer_name         varchar(80) NOT NULL,
            customer_type         varchar(24),
```

```
            town                  varchar(40),
            region_key            smallint NOT NULL REFERENCES dw.dim_region (region_key),
            country_code          char(2),
            country_name          varchar(30),
            market_group          varchar(20),
            currency_code         char(3),
            onboarding_date       date,
            customer_status       varchar(10),
            channel               varchar(20),
            credit_terms_days     smallint,
            is_key_account        boolean,
            is_inferred           boolean NOT NULL DEFAULT false,
            merged_codes          text,
            scd_effective_from    date NOT NULL,
            scd_effective_to      date NOT NULL DEFAULT DATE '9999-12-31',
            scd_is_current        boolean NOT NULL,
            scd_version           smallint NOT NULL,
            scd_row_hash          char(32) NOT NULL,
            dw_batch_id           integer,
            dw_created_at         timestamptz NOT NULL DEFAULT now(),
            dw_updated_at         timestamptz NOT NULL DEFAULT now(),
            CONSTRAINT uq_dim_customer_version UNIQUE (customer_code, scd_effective_from),
            CONSTRAINT ck_dim_customer_dates CHECK (scd_effective_to >= scd_effective_from)
        );
        CREATE UNIQUE INDEX ux_dim_customer_current ON dw.dim_customer (customer_code) WHERE scd_is_current;
        COMMENT ON TABLE dw.dim_customer IS 'Customer (business, institution, pseudonymised informal trader or export buyer). SCD Type 2 on
channel, credit terms and key-account flag; Type 1 elsewhere. Supports inferred (late-arriving) members.';
        COMMENT ON COLUMN dw.dim_customer.customer_key IS 'Surrogate key per customer VERSION (-1 Unknown). | Source: ETL identity';
        COMMENT ON COLUMN dw.dim_customer.customer_code IS 'Durable natural key from ERP/MDM, e.g. C00042 (duplicates merged to survivor). |
Source: Master data workbook, sheet Customers';
        COMMENT ON COLUMN dw.dim_customer.customer_name IS 'Trading name (Type 1). Informal traders carry a pseudonymous label only. | Source:
Master data workbook (rule MD05 formatting)';
        COMMENT ON COLUMN dw.dim_customer.customer_type IS 'Business, Government institution, Informal trader or Export (Type 1). | Source:
Master data workbook';
        COMMENT ON COLUMN dw.dim_customer.town IS 'Delivery town (Type 1). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_customer.region_key IS 'FK to dim_region for domestic customers; -2 for foreign customers, -1 when missing
(rule MD02). | Source: ETL via ref.region_alias';
        COMMENT ON COLUMN dw.dim_customer.country_code IS 'ISO 3166-1 alpha-2 country. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_customer.country_name IS 'Country name. | Source: ref.country';
        COMMENT ON COLUMN dw.dim_customer.market_group IS 'Market grouping: Domestic, SACU, SADC (non-SACU), Europe, Middle East. | Source:
ref.country';
        COMMENT ON COLUMN dw.dim_customer.currency_code IS 'Invoicing currency (NAD, ZAR, USD, EUR, GBP). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_customer.onboarding_date IS 'Date the account was opened. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_customer.customer_status IS 'Active or Inactive at extract date (Type 1). | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_customer.channel IS 'Sales channel in this version: Miller/Processor, Wholesale, Retail, Informal market,
Institutional, Export (Type 2). | Source: Master data + Customer_History';
        COMMENT ON COLUMN dw.dim_customer.credit_terms_days IS 'Payment terms in days in this version (0 = cash on delivery) (Type 2). | Source:
Master data + Customer_History';
        COMMENT ON COLUMN dw.dim_customer.is_key_account IS 'Key-account flag in this version (Type 2). | Source: Master data +
Customer_History';
        COMMENT ON COLUMN dw.dim_customer.is_inferred IS 'True for an inferred member created from a fact before the master record arrived (rule
SA04). | Source: ETL';
        COMMENT ON COLUMN dw.dim_customer.merged_codes IS 'Duplicate customer codes merged into this survivor (rule MD04). | Source:
ref.customer_xref';
        COMMENT ON COLUMN dw.dim_customer.scd_effective_from IS 'First date this version is valid. | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_customer.scd_effective_to IS 'Last date this version is valid (9999-12-31 = open). | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_customer.scd_is_current IS 'True for the current version. | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_customer.scd_version IS 'Version sequence number per customer_code. | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_customer.scd_row_hash IS 'MD5 of the Type 2 attributes. | Source: ETL (SCD2)';
        COMMENT ON COLUMN dw.dim_customer.dw_batch_id IS 'Audit: ETL batch that created/last updated the row. | Source: ETL';
        COMMENT ON COLUMN dw.dim_customer.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';
        COMMENT ON COLUMN dw.dim_customer.dw_updated_at IS 'Audit: last update timestamp. | Source: ETL';

        CREATE TABLE dw.dim_cost_category (
            cost_category_key  smallint PRIMARY KEY,
            cost_category_code char(4) NOT NULL UNIQUE,
            cost_category_name varchar(40) NOT NULL,
            cost_group         varchar(30) NOT NULL,
            cost_behaviour     varchar(15) NOT NULL,
            gl_account         char(4),
            quantity_uom       varchar(6),
            is_crop_specific   boolean NOT NULL,
            dw_created_at      timestamptz NOT NULL DEFAULT now()
        );
        COMMENT ON TABLE dw.dim_cost_category IS 'Production cost category with cost group and behaviour (variable/fixed). SCD Type 1.';
        COMMENT ON COLUMN dw.dim_cost_category.cost_category_key IS 'Surrogate key (-1 Unknown). | Source: ETL';
        COMMENT ON COLUMN dw.dim_cost_category.cost_category_code IS 'Cost category code CC01..CC14 (natural key). | Source: Master data
workbook, sheet Cost_Categories';
        COMMENT ON COLUMN dw.dim_cost_category.cost_category_name IS 'Cost category name. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_cost_category.cost_group IS 'Cost group: Direct inputs, Labour, Energy & water, Mechanisation, Packaging &
logistics, Overheads. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_cost_category.cost_behaviour IS 'Variable, Semi-variable or Fixed. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_cost_category.gl_account IS 'General-ledger account number. | Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_cost_category.quantity_uom IS 'Unit of the physical input quantity (kg, l, hours, kWh, m3) where captured. |
Source: Master data workbook';
        COMMENT ON COLUMN dw.dim_cost_category.is_crop_specific IS 'False for farm overheads that are not allocated to a crop cycle. | Source:
derived (Fixed behaviour)';
        COMMENT ON COLUMN dw.dim_cost_category.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';

        CREATE TABLE dw.dim_commodity (
            commodity_key      smallint PRIMARY KEY,
            commodity_code     varchar(16) NOT NULL UNIQUE,
            commodity_name     varchar(40) NOT NULL,
            source_series_name varchar(30) NOT NULL,
            price_unit         varchar(10) NOT NULL,
            commodity_role     varchar(14) NOT NULL,
            crop_key           smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            source_name        varchar(40) NOT NULL,
            dw_created_at      timestamptz NOT NULL DEFAULT now()
        );
```

```
        COMMENT ON TABLE dw.dim_commodity IS 'International commodity price series (output prices for crops, input prices for fertiliser and
fuel). SCD Type 1.';
        COMMENT ON COLUMN dw.dim_commodity.commodity_key IS 'Surrogate key. | Source: ETL';
        COMMENT ON COLUMN dw.dim_commodity.commodity_code IS 'Series code (natural key), e.g. MAIZE, UREA. | Source: ref.commodity_series';
        COMMENT ON COLUMN dw.dim_commodity.commodity_name IS 'Descriptive series name. | Source: World Bank Pink Sheet header';
        COMMENT ON COLUMN dw.dim_commodity.source_series_name IS 'Column header in the Pink Sheet workbook. | Source: World Bank Pink Sheet';
        COMMENT ON COLUMN dw.dim_commodity.price_unit IS 'Unit of the published price (USD/mt, USD/bbl). | Source: World Bank Pink Sheet';
        COMMENT ON COLUMN dw.dim_commodity.commodity_role IS 'Output price (crop sold) or Input price (cost driver). | Source: derived';
        COMMENT ON COLUMN dw.dim_commodity.crop_key IS 'Crop benchmarked by the series (sorghum proxies mahangu; -2 for input series). | Source:
ETL mapping';
        COMMENT ON COLUMN dw.dim_commodity.source_name IS 'Publisher of the series. | Source: derived';
        COMMENT ON COLUMN dw.dim_commodity.dw_created_at IS 'Audit: row creation timestamp. | Source: ETL';

        -- ============================================================ FACTS
        CREATE TABLE dw.fact_sales (
            sales_line_key     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            invoice_date_key   integer  NOT NULL REFERENCES dw.dim_date (date_key),
            dispatch_date_key  integer  NOT NULL REFERENCES dw.dim_date (date_key),
            product_key        integer  NOT NULL REFERENCES dw.dim_product (product_key),
            customer_key       integer  NOT NULL REFERENCES dw.dim_customer (customer_key),
            farm_key           integer  NOT NULL REFERENCES dw.dim_farm (farm_key),
            invoice_no         varchar(16) NOT NULL,
            line_no            smallint NOT NULL,
            doc_type           char(3)  NOT NULL CHECK (doc_type IN ('INV', 'CRN')),
            ref_invoice_no     varchar(16),
            sales_rep_code     varchar(6),
            currency_code      char(3)  NOT NULL,
            quantity_sales_uom numeric(14, 3) NOT NULL,
            quantity_kg        numeric(14, 3) NOT NULL,
            unit_price_txn     numeric(14, 4) NOT NULL,
            fx_rate_nad        numeric(12, 6) NOT NULL,
            gross_revenue_nad  numeric(16, 2) NOT NULL,
            discount_nad       numeric(16, 2) NOT NULL,
            net_revenue_nad    numeric(16, 2) NOT NULL,
            cogs_nad           numeric(16, 2) NOT NULL,
            freight_cost_nad   numeric(16, 2) NOT NULL,
            gross_margin_nad   numeric(16, 2) GENERATED ALWAYS AS (net_revenue_nad - cogs_nad) STORED,
            is_price_anomaly   boolean NOT NULL DEFAULT false,
            dw_batch_id        integer NOT NULL,
            CONSTRAINT uq_fact_sales_line UNIQUE (invoice_no, line_no)
        );
        COMMENT ON TABLE dw.fact_sales IS 'Transaction fact. GRAIN: one row per ERP invoice or credit-note line. All monetary measures are
additive (N$).';
        COMMENT ON COLUMN dw.fact_sales.sales_line_key IS 'Fact surrogate key (technical; hidden in BI). | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_sales.invoice_date_key IS 'FK to dim_date: invoice (revenue recognition) date. | Source: ERP invoice_date';
        COMMENT ON COLUMN dw.fact_sales.dispatch_date_key IS 'FK to dim_date (role-playing): physical dispatch date used for stock
reconciliation. | Source: ERP dispatch_date';
        COMMENT ON COLUMN dw.fact_sales.product_key IS 'FK to dim_product. | Source: ERP product_code (rule SA02)';
        COMMENT ON COLUMN dw.fact_sales.customer_key IS 'FK to the dim_customer version valid on the invoice date. | Source: ERP customer_code
(rules SA03/SA04)';
        COMMENT ON COLUMN dw.fact_sales.farm_key IS 'FK to the dim_farm version of the farm of origin on the invoice date (-1 when missing, rule
SA05). | Source: ERP farm_code';
        COMMENT ON COLUMN dw.fact_sales.invoice_no IS 'Degenerate dimension: ERP document number. | Source: ERP';
        COMMENT ON COLUMN dw.fact_sales.line_no IS 'Degenerate dimension: line number within the document. | Source: ERP';
        COMMENT ON COLUMN dw.fact_sales.doc_type IS 'INV = invoice, CRN = credit note (quality claim). | Source: ERP';
        COMMENT ON COLUMN dw.fact_sales.ref_invoice_no IS 'For credit notes: the invoice being credited. | Source: ERP';
        COMMENT ON COLUMN dw.fact_sales.sales_rep_code IS 'Pseudonymised sales representative code. | Source: ERP';
        COMMENT ON COLUMN dw.fact_sales.currency_code IS 'Invoicing currency (rule SA06 defaults missing values). | Source: ERP / customer
master';
        COMMENT ON COLUMN dw.fact_sales.quantity_sales_uom IS 'Quantity in the invoice unit of measure (negative on credit notes). Additive only
within one product. | Source: ERP quantity';
        COMMENT ON COLUMN dw.fact_sales.quantity_kg IS 'Quantity converted to kilograms (additive). | Source: derived quantity x
dim_product.kg_per_sales_uom';
        COMMENT ON COLUMN dw.fact_sales.unit_price_txn IS 'Unit price in transaction currency per sales UoM (non-additive). | Source: ERP';
        COMMENT ON COLUMN dw.fact_sales.fx_rate_nad IS 'N$ per unit of transaction currency, monthly average of the invoice month (non-
additive). | Source: ref.fx_rate_monthly (FRED)';
        COMMENT ON COLUMN dw.fact_sales.gross_revenue_nad IS 'quantity x unit price x FX before discount, N$ (additive). | Source: derived';
        COMMENT ON COLUMN dw.fact_sales.discount_nad IS 'Discount granted, N$ (additive). | Source: derived from ERP discount_pct';
        COMMENT ON COLUMN dw.fact_sales.net_revenue_nad IS 'Gross revenue less discount, N$ (additive) - the headline revenue measure. | Source:
derived';
        COMMENT ON COLUMN dw.fact_sales.cogs_nad IS 'Standard cost of goods sold, N$ (0 on credit notes: returned produce is written off). |
Source: ERP unit_std_cost_nad x quantity';
        COMMENT ON COLUMN dw.fact_sales.freight_cost_nad IS 'Outbound freight cost borne by Green Basket, N$ (additive). | Source: ERP
freight_nad';
        COMMENT ON COLUMN dw.fact_sales.gross_margin_nad IS 'Net revenue less COGS, N$ (additive, computed column). | Source: derived';
        COMMENT ON COLUMN dw.fact_sales.is_price_anomaly IS 'True when the N$/kg price deviates > 4x from the product-month median (rule SA07).
| Source: ETL';
        COMMENT ON COLUMN dw.fact_sales.dw_batch_id IS 'Audit: ETL batch that loaded the row. | Source: ETL';

        CREATE TABLE dw.fact_harvest (
            harvest_event_key bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            harvest_date_key  integer NOT NULL REFERENCES dw.dim_date (date_key),
            farm_key          integer NOT NULL REFERENCES dw.dim_farm (farm_key),
            crop_key          smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            cycle_id          varchar(24),
            block_id          varchar(6) NOT NULL,
            recorder_code     varchar(10),
            harvested_kg      numeric(14, 1) NOT NULL CHECK (harvested_kg > 0),
            is_corrected      boolean NOT NULL DEFAULT false,
            source_file       varchar(40) NOT NULL,
            source_row_num    integer NOT NULL,
            dw_batch_id       integer NOT NULL,
            CONSTRAINT uq_fact_harvest_source UNIQUE (source_file, source_row_num)
        );
        COMMENT ON TABLE dw.fact_harvest IS 'Transaction fact. GRAIN: one row per harvest record = farm x crop x field block x harvest day
(weighbridge/packhouse slip).';
        COMMENT ON COLUMN dw.fact_harvest.harvest_event_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_harvest.harvest_date_key IS 'FK to dim_date: harvest day. | Source: harvest log date (template-specific
format)';
        COMMENT ON COLUMN dw.fact_harvest.farm_key IS 'FK to the dim_farm version valid on the harvest date. | Source: harvest log farm code
```

```
(rule HV08)';
        COMMENT ON COLUMN dw.fact_harvest.crop_key IS 'FK to dim_crop (crop grain - conformed rollup of dim_product). | Source: harvest log crop
text via ref.crop_alias';
        COMMENT ON COLUMN dw.fact_harvest.cycle_id IS 'Degenerate: crop-cycle identifier from the planting register (links to fact_crop_cycle).
| Source: ETL block matching';
        COMMENT ON COLUMN dw.fact_harvest.block_id IS 'Degenerate: field block / pivot / vineyard block code. | Source: harvest log';
        COMMENT ON COLUMN dw.fact_harvest.recorder_code IS 'Pseudonymised clerk code that captured the record. | Source: harvest log';
        COMMENT ON COLUMN dw.fact_harvest.harvested_kg IS 'Field mass harvested in kg after unit standardisation (additive). | Source: harvest
log quantity x ref.uom_conversion';
        COMMENT ON COLUMN dw.fact_harvest.is_corrected IS 'True when rule HV06 corrected a unit/decimal error. | Source: ETL';
        COMMENT ON COLUMN dw.fact_harvest.source_file IS 'Lineage: source file name. | Source: ETL';
        COMMENT ON COLUMN dw.fact_harvest.source_row_num IS 'Lineage: row number in the source file. | Source: ETL';
        COMMENT ON COLUMN dw.fact_harvest.dw_batch_id IS 'Audit: ETL batch. | Source: ETL';

        CREATE TABLE dw.fact_crop_cycle (
            crop_cycle_key            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            cycle_id                  varchar(24) NOT NULL,
            block_id                  varchar(6) NOT NULL,
            farm_key                  integer NOT NULL REFERENCES dw.dim_farm (farm_key),
            crop_key                  smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            planting_date_key         integer NOT NULL REFERENCES dw.dim_date (date_key),
            first_harvest_date_key    integer NOT NULL REFERENCES dw.dim_date (date_key),
            last_harvest_date_key     integer NOT NULL REFERENCES dw.dim_date (date_key),
            harvest_season_end_date_key integer NOT NULL REFERENCES dw.dim_date (date_key),
            cycle_status              varchar(10) NOT NULL CHECK (cycle_status IN ('Growing', 'Harvesting', 'Completed', 'Failed')),
            variety                   varchar(30),
            irrigation_method         varchar(20) NOT NULL,
            is_irrigated              boolean NOT NULL,
            planted_ha                numeric(10, 2) NOT NULL,
            harvested_ha              numeric(10, 2),
            planned_yield_t_ha        numeric(8, 2),
            planned_production_kg     numeric(14, 1),
            harvested_kg              numeric(14, 1) NOT NULL DEFAULT 0,
            harvest_event_count       integer NOT NULL DEFAULT 0,
            days_to_first_harvest     integer,
            harvest_duration_days     integer,
            direct_cost_nad           numeric(16, 2) NOT NULL DEFAULT 0,
            fertiliser_kg             numeric(14, 1) NOT NULL DEFAULT 0,
            planted_ha_imputed        boolean NOT NULL DEFAULT false,
            dw_batch_id               integer NOT NULL,
            dw_updated_at             timestamptz NOT NULL DEFAULT now(),
            CONSTRAINT uq_fact_crop_cycle UNIQUE (cycle_id, block_id)
        );
        COMMENT ON TABLE dw.fact_crop_cycle IS 'Accumulating snapshot. GRAIN: one row per planted field block per crop cycle; milestone dates
and measures are updated as the cycle moves from Growing to Harvesting to Completed.';
        COMMENT ON COLUMN dw.fact_crop_cycle.crop_cycle_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_crop_cycle.cycle_id IS 'Degenerate: crop-cycle id farm/crop/year+slot, e.g. GB-KAW-01/MZW/2023S. | Source:
planting register';
        COMMENT ON COLUMN dw.fact_crop_cycle.block_id IS 'Degenerate: field block code. | Source: planting register';
        COMMENT ON COLUMN dw.fact_crop_cycle.farm_key IS 'FK to the dim_farm version valid on the planting date (as-was irrigation regime). |
Source: planting register farm_code';
        COMMENT ON COLUMN dw.fact_crop_cycle.crop_key IS 'FK to dim_crop. | Source: planting register crop_code';
        COMMENT ON COLUMN dw.fact_crop_cycle.planting_date_key IS 'Milestone 1 (role-playing date, active in BI): planting / bud-break date. |
Source: planting register (rule PL01)';
        COMMENT ON COLUMN dw.fact_crop_cycle.first_harvest_date_key IS 'Milestone 2 (role-playing date): first harvest record (-1 = not yet). |
Source: derived from fact_harvest';
        COMMENT ON COLUMN dw.fact_crop_cycle.last_harvest_date_key IS 'Milestone 3 (role-playing date): latest harvest record to date (-1 = not
yet). | Source: derived from fact_harvest';
        COMMENT ON COLUMN dw.fact_crop_cycle.harvest_season_end_date_key IS 'Role-playing date: 30 September closing the agricultural season of
the (expected) first harvest; conforms with fact_crop_estimate. | Source: derived';
        COMMENT ON COLUMN dw.fact_crop_cycle.cycle_status IS 'Growing, Harvesting, Completed or Failed as at the extract date. | Source:
planting register validated against harvest log (rule PL03)';
        COMMENT ON COLUMN dw.fact_crop_cycle.variety IS 'Cultivar planted. | Source: planting register';
        COMMENT ON COLUMN dw.fact_crop_cycle.irrigation_method IS 'Degenerate: irrigation method of THIS block in this cycle (Dryland, Centre
pivot, Drip, ...); can differ from the farm''s main system, e.g. a borehole pivot block on a dryland farm. | Source: planting register
(fallback: dim_farm as-was)';
        COMMENT ON COLUMN dw.fact_crop_cycle.is_irrigated IS 'Degenerate: true when the block was irrigated in this cycle. | Source: derived
from irrigation_method';
        COMMENT ON COLUMN dw.fact_crop_cycle.planted_ha IS 'Planted area of the block in hectares (additive). | Source: planting register (rule
PL02 imputes)';
        COMMENT ON COLUMN dw.fact_crop_cycle.harvested_ha IS 'Harvested area in hectares once Completed (0 if Failed, NULL while open)
(additive). | Source: planting register';
        COMMENT ON COLUMN dw.fact_crop_cycle.planned_yield_t_ha IS 'Budgeted yield t/ha at planting (non-additive). | Source: planting
register';
        COMMENT ON COLUMN dw.fact_crop_cycle.planned_production_kg IS 'Budgeted production = planned yield x planted ha x 1000 (additive). |
Source: derived';
        COMMENT ON COLUMN dw.fact_crop_cycle.harvested_kg IS 'Cumulative harvested kg to date (additive). | Source: aggregated from
fact_harvest';
        COMMENT ON COLUMN dw.fact_crop_cycle.harvest_event_count IS 'Number of harvest records to date (additive). | Source: aggregated from
fact_harvest';
        COMMENT ON COLUMN dw.fact_crop_cycle.days_to_first_harvest IS 'Days from planting to first harvest (non-additive; average in BI). |
Source: derived';
        COMMENT ON COLUMN dw.fact_crop_cycle.harvest_duration_days IS 'Days between first and last harvest record (non-additive). | Source:
derived';
        COMMENT ON COLUMN dw.fact_crop_cycle.direct_cost_nad IS 'Crop-cycle direct cost allocated to the block by planted-area share, N$
(additive). | Source: cost ledger cost_centre = cycle_id';
        COMMENT ON COLUMN dw.fact_crop_cycle.fertiliser_kg IS 'Fertiliser applied, allocated by area share (additive). | Source: cost ledger
input_qty (CC02)';
        COMMENT ON COLUMN dw.fact_crop_cycle.planted_ha_imputed IS 'True when planted_ha was imputed by rule PL02. | Source: ETL';
        COMMENT ON COLUMN dw.fact_crop_cycle.dw_batch_id IS 'Audit: ETL batch that last updated the row. | Source: ETL';
        COMMENT ON COLUMN dw.fact_crop_cycle.dw_updated_at IS 'Audit: last update timestamp (accumulating snapshot rows are updated in place). |
Source: ETL';

        CREATE TABLE dw.fact_production_cost (
            cost_posting_key  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            posting_date_key  integer NOT NULL REFERENCES dw.dim_date (date_key),
            farm_key          integer NOT NULL REFERENCES dw.dim_farm (farm_key),
            crop_key          smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            cost_category_key smallint NOT NULL REFERENCES dw.dim_cost_category (cost_category_key),
            posting_id        varchar(24) NOT NULL UNIQUE,
```

```
            cost_centre       varchar(30) NOT NULL,
            supplier_code     varchar(8),
            amount_nad        numeric(16, 2) NOT NULL,
            input_qty         numeric(14, 2),
            input_uom         varchar(6),
            is_reversal       boolean NOT NULL DEFAULT false,
            dw_batch_id       integer NOT NULL
        );
        COMMENT ON TABLE dw.fact_production_cost IS 'Transaction fact. GRAIN: one row per cost-ledger posting (farm x crop cycle or overhead x
cost category x posting date).';
        COMMENT ON COLUMN dw.fact_production_cost.cost_posting_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_production_cost.posting_date_key IS 'FK to dim_date: posting date. | Source: cost ledger';
        COMMENT ON COLUMN dw.fact_production_cost.farm_key IS 'FK to the dim_farm version valid on the posting date. | Source: cost ledger
farm_code';
        COMMENT ON COLUMN dw.fact_production_cost.crop_key IS 'FK to dim_crop (-2 farm overhead, -1 crop missing - rule CL03). | Source: cost
ledger crop_code';
        COMMENT ON COLUMN dw.fact_production_cost.cost_category_key IS 'FK to dim_cost_category. | Source: cost ledger free text via
ref.cost_category_alias (rule CL02)';
        COMMENT ON COLUMN dw.fact_production_cost.posting_id IS 'Degenerate: journal posting id (suffix -R reversal, -C correction). | Source:
cost ledger';
        COMMENT ON COLUMN dw.fact_production_cost.cost_centre IS 'Degenerate: cost centre = crop cycle id, FARM/OVERHEAD or FARM/UNALLOCATED. |
Source: cost ledger';
        COMMENT ON COLUMN dw.fact_production_cost.supplier_code IS 'Pseudonymised supplier code. | Source: cost ledger';
        COMMENT ON COLUMN dw.fact_production_cost.amount_nad IS 'Posted cost amount in N$ (additive; reversals negative). | Source: cost ledger
(rule CL04)';
        COMMENT ON COLUMN dw.fact_production_cost.input_qty IS 'Physical input quantity where captured (additive within one category). | Source:
cost ledger';
        COMMENT ON COLUMN dw.fact_production_cost.input_uom IS 'Unit of input_qty. | Source: cost ledger';
        COMMENT ON COLUMN dw.fact_production_cost.is_reversal IS 'True for reversal postings. | Source: derived from posting_id';
        COMMENT ON COLUMN dw.fact_production_cost.dw_batch_id IS 'Audit: ETL batch. | Source: ETL';

        CREATE TABLE dw.fact_inventory_monthly (
            inventory_snapshot_key  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            snapshot_date_key       integer NOT NULL REFERENCES dw.dim_date (date_key),
            farm_key                integer NOT NULL REFERENCES dw.dim_farm (farm_key),
            product_key             integer NOT NULL REFERENCES dw.dim_product (product_key),
            opening_stock_kg        numeric(14, 1) NOT NULL,
            received_kg             numeric(14, 1) NOT NULL,
            dispatched_kg           numeric(14, 1) NOT NULL,
            shrinkage_kg            numeric(14, 1) NOT NULL,
            closing_stock_kg        numeric(14, 1) NOT NULL,
            closing_stock_value_nad numeric(16, 2) NOT NULL,
            is_imputed              boolean NOT NULL DEFAULT false,
            is_sign_corrected       boolean NOT NULL DEFAULT false,
            dw_batch_id             integer NOT NULL,
            CONSTRAINT uq_fact_inventory UNIQUE (snapshot_date_key, farm_key, product_key)
        );
        COMMENT ON TABLE dw.fact_inventory_monthly IS 'Periodic snapshot. GRAIN: one row per farm store x product x month-end. Stock levels are
SEMI-ADDITIVE (sum across farms/products, never across months); flows are additive.';
        COMMENT ON COLUMN dw.fact_inventory_monthly.inventory_snapshot_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_inventory_monthly.snapshot_date_key IS 'FK to dim_date: month-end count date. | Source: stock system period';
        COMMENT ON COLUMN dw.fact_inventory_monthly.farm_key IS 'FK to the dim_farm version valid at month-end. | Source: stock system
farm_code';
        COMMENT ON COLUMN dw.fact_inventory_monthly.product_key IS 'FK to dim_product. | Source: stock system product_code';
        COMMENT ON COLUMN dw.fact_inventory_monthly.opening_stock_kg IS 'Stock on hand at the start of the month, kg (semi-additive). | Source:
stock system';
        COMMENT ON COLUMN dw.fact_inventory_monthly.received_kg IS 'Graded produce received from the field in the month, kg (additive). |
Source: stock system';
        COMMENT ON COLUMN dw.fact_inventory_monthly.dispatched_kg IS 'Produce dispatched to customers in the month, kg (additive; reconciles to
ERP dispatches). | Source: stock system';
        COMMENT ON COLUMN dw.fact_inventory_monthly.shrinkage_kg IS 'Losses (spoilage, moisture, theft) in the month, kg (additive). | Source:
stock system';
        COMMENT ON COLUMN dw.fact_inventory_monthly.closing_stock_kg IS 'Stock on hand at month-end, kg (semi-additive: use closing-balance
logic over time). | Source: stock system (rule ST01)';
        COMMENT ON COLUMN dw.fact_inventory_monthly.closing_stock_value_nad IS 'Closing stock at standard cost, N$ (semi-additive). | Source:
stock system';
        COMMENT ON COLUMN dw.fact_inventory_monthly.is_imputed IS 'True when the snapshot row was imputed for a missing month (rule ST03). |
Source: ETL';
        COMMENT ON COLUMN dw.fact_inventory_monthly.is_sign_corrected IS 'True when rule ST01 corrected a sign error. | Source: ETL';
        COMMENT ON COLUMN dw.fact_inventory_monthly.dw_batch_id IS 'Audit: ETL batch. | Source: ETL';

        CREATE TABLE dw.fact_weather_daily (
            weather_key      bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            date_key         integer NOT NULL REFERENCES dw.dim_date (date_key),
            farm_key         integer NOT NULL REFERENCES dw.dim_farm (farm_key),
            rainfall_mm      numeric(7, 2),
            t_max_c          numeric(5, 2),
            t_min_c          numeric(5, 2),
            rh_pct           numeric(5, 2),
            solar_mj_m2      numeric(6, 2),
            is_heat_day      boolean,
            is_rain_day      boolean,
            rainfall_imputed boolean NOT NULL DEFAULT false,
            temp_imputed     boolean NOT NULL DEFAULT false,
            dw_batch_id      integer NOT NULL,
            CONSTRAINT uq_fact_weather UNIQUE (date_key, farm_key)
        );
        COMMENT ON TABLE dw.fact_weather_daily IS 'Periodic fact (external). GRAIN: one row per farm location per day. Rainfall additive over
time but averaged across locations; temperatures non-additive.';
        COMMENT ON COLUMN dw.fact_weather_daily.weather_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_weather_daily.date_key IS 'FK to dim_date. | Source: NASA POWER date key YYYYMMDD';
        COMMENT ON COLUMN dw.fact_weather_daily.farm_key IS 'FK to the dim_farm version valid on the day (weather point = farm coordinates). |
Source: NASA POWER request per farm';
        COMMENT ON COLUMN dw.fact_weather_daily.rainfall_mm IS 'Corrected precipitation, mm/day (PRECTOTCORR). | Source: NASA POWER (rule
WE02)';
        COMMENT ON COLUMN dw.fact_weather_daily.t_max_c IS 'Maximum 2 m air temperature, deg C (T2M_MAX). | Source: NASA POWER (rule WE01)';
        COMMENT ON COLUMN dw.fact_weather_daily.t_min_c IS 'Minimum 2 m air temperature, deg C (T2M_MIN). | Source: NASA POWER (rule WE01)';
        COMMENT ON COLUMN dw.fact_weather_daily.rh_pct IS 'Relative humidity at 2 m, % (RH2M). | Source: NASA POWER';
        COMMENT ON COLUMN dw.fact_weather_daily.solar_mj_m2 IS 'All-sky surface shortwave irradiance, MJ/m2/day. | Source: NASA POWER';
        COMMENT ON COLUMN dw.fact_weather_daily.is_heat_day IS 'True when t_max_c >= 35 deg C (crop heat-stress day). | Source: derived';
```

```
        COMMENT ON COLUMN dw.fact_weather_daily.is_rain_day IS 'True when rainfall_mm >= 1. | Source: derived';
        COMMENT ON COLUMN dw.fact_weather_daily.rainfall_imputed IS 'True when rainfall was imputed by rule WE02. | Source: ETL';
        COMMENT ON COLUMN dw.fact_weather_daily.temp_imputed IS 'True when temperature was imputed by rule WE01. | Source: ETL';
        COMMENT ON COLUMN dw.fact_weather_daily.dw_batch_id IS 'Audit: ETL batch. | Source: ETL';

        CREATE TABLE dw.fact_market_price_monthly (
            market_price_key  bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            month_date_key    integer NOT NULL REFERENCES dw.dim_date (date_key),
            commodity_key     smallint NOT NULL REFERENCES dw.dim_commodity (commodity_key),
            crop_key          smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            price_usd         numeric(12, 2) NOT NULL,
            fx_nad_per_usd    numeric(12, 6) NOT NULL,
            price_nad         numeric(14, 2) NOT NULL,
            is_imputed        boolean NOT NULL DEFAULT false,
            dw_batch_id       integer NOT NULL,
            CONSTRAINT uq_fact_market_price UNIQUE (month_date_key, commodity_key)
        );
        COMMENT ON TABLE dw.fact_market_price_monthly IS 'Periodic fact (external). GRAIN: one row per commodity series per month (keyed on the
month-end date). Prices are NON-ADDITIVE (average only).';
        COMMENT ON COLUMN dw.fact_market_price_monthly.market_price_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_market_price_monthly.month_date_key IS 'FK to dim_date: last day of the price month. | Source: Pink Sheet
period YYYYMmm';
        COMMENT ON COLUMN dw.fact_market_price_monthly.commodity_key IS 'FK to dim_commodity. | Source: Pink Sheet column header';
        COMMENT ON COLUMN dw.fact_market_price_monthly.crop_key IS 'FK to dim_crop of the benchmarked crop (-2 for input series). | Source:
dim_commodity mapping';
        COMMENT ON COLUMN dw.fact_market_price_monthly.price_usd IS 'Monthly average price in USD per published unit (non-additive). | Source:
World Bank Pink Sheet (rule MP01)';
        COMMENT ON COLUMN dw.fact_market_price_monthly.fx_nad_per_usd IS 'Monthly average N$ per USD (non-additive). | Source: FRED DEXSFUS (NAD
= ZAR)';
        COMMENT ON COLUMN dw.fact_market_price_monthly.price_nad IS 'Price converted to N$ per published unit (non-additive). | Source:
derived';
        COMMENT ON COLUMN dw.fact_market_price_monthly.is_imputed IS 'True when the price was interpolated (rule MP01). | Source: ETL';
        COMMENT ON COLUMN dw.fact_market_price_monthly.dw_batch_id IS 'Audit: ETL batch. | Source: ETL';

        CREATE TABLE dw.fact_crop_estimate (
            crop_estimate_key     bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            season_end_date_key   integer NOT NULL REFERENCES dw.dim_date (date_key),
            region_key            smallint NOT NULL REFERENCES dw.dim_region (region_key),
            crop_key              smallint NOT NULL REFERENCES dw.dim_crop (crop_key),
            farming_sector        varchar(10) NOT NULL CHECK (farming_sector IN ('Communal', 'Commercial')),
            area_planted_ha       numeric(12, 1),
            production_t          numeric(14, 1),
            yield_t_ha            numeric(8, 3),
            is_yield_derived      boolean NOT NULL DEFAULT false,
            release_date          date,
            source_table          varchar(80),
            dw_batch_id           integer NOT NULL,
            CONSTRAINT uq_fact_crop_estimate UNIQUE (season_end_date_key, region_key, crop_key, farming_sector)
        );
        COMMENT ON TABLE dw.fact_crop_estimate IS 'Benchmark fact (external). GRAIN: one row per agricultural season x region x crop x farming
sector. Area/production additive; yield non-additive (recompute as production/area).';
        COMMENT ON COLUMN dw.fact_crop_estimate.crop_estimate_key IS 'Fact surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_crop_estimate.season_end_date_key IS 'FK to dim_date: 30 September closing the agricultural season. | Source:
NSA season text e.g. 2023/2024';
        COMMENT ON COLUMN dw.fact_crop_estimate.region_key IS 'FK to dim_region (conformed geography). | Source: NSA region text via
ref.region_alias';
        COMMENT ON COLUMN dw.fact_crop_estimate.crop_key IS 'FK to dim_crop (rule CE01 excludes out-of-scope crops). | Source: NSA crop text via
ref.crop_alias';
        COMMENT ON COLUMN dw.fact_crop_estimate.farming_sector IS 'Communal or Commercial farming sector. | Source: NSA / MAFWLR table';
        COMMENT ON COLUMN dw.fact_crop_estimate.area_planted_ha IS 'Area planted, ha (additive). | Source: NSA / MAFWLR crop estimates';
        COMMENT ON COLUMN dw.fact_crop_estimate.production_t IS 'Production, tonnes (additive). | Source: NSA / MAFWLR crop estimates';
        COMMENT ON COLUMN dw.fact_crop_estimate.yield_t_ha IS 'Published (or derived) yield, t/ha (non-additive). | Source: NSA (rule CE02)';
        COMMENT ON COLUMN dw.fact_crop_estimate.is_yield_derived IS 'True when yield was derived as production/area. | Source: ETL';
        COMMENT ON COLUMN dw.fact_crop_estimate.release_date IS 'Publication date of the estimate. | Source: NSA';
        COMMENT ON COLUMN dw.fact_crop_estimate.source_table IS 'Lineage: report/table of origin. | Source: NSA';
        COMMENT ON COLUMN dw.fact_crop_estimate.dw_batch_id IS 'Audit: ETL batch. | Source: ETL';

        -- Analytics outputs written back by the Python scripts (03_analytics)
        CREATE TABLE dw.fact_forecast_monthly (
            forecast_key          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            month_date_key        integer NOT NULL REFERENCES dw.dim_date (date_key),
            crop_category_key     smallint NOT NULL REFERENCES dw.dim_crop_category (crop_category_key),
            model_name            varchar(30) NOT NULL,
            scenario              varchar(12) NOT NULL CHECK (scenario IN ('Backtest', 'Forecast')),
            actual_net_revenue_nad numeric(16, 2),
            forecast_net_revenue_nad numeric(16, 2) NOT NULL,
            lower_80_nad          numeric(16, 2),
            upper_80_nad          numeric(16, 2),
            lower_95_nad          numeric(16, 2),
            upper_95_nad          numeric(16, 2),
            model_run_at          timestamptz NOT NULL DEFAULT now(),
            CONSTRAINT uq_fact_forecast UNIQUE (month_date_key, crop_category_key, model_name, scenario)
        );
        COMMENT ON TABLE dw.fact_forecast_monthly IS 'Model output. GRAIN: one row per month x crop category x model x scenario (12-month back-
test on hold-out data, then 12-month forward forecast).';
        COMMENT ON COLUMN dw.fact_forecast_monthly.forecast_key IS 'Surrogate key. | Source: ETL identity';
        COMMENT ON COLUMN dw.fact_forecast_monthly.month_date_key IS 'FK to dim_date: month-end of the forecast month. | Source:
03_analytics/forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.crop_category_key IS 'FK to dim_crop_category. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.model_name IS 'Model: Seasonal naive, Holt-Winters ETS, SARIMA. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.scenario IS 'Backtest (hold-out evaluation) or Forecast (forward). | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.actual_net_revenue_nad IS 'Actual net revenue for back-test months, N$. | Source:
fact_sales';
        COMMENT ON COLUMN dw.fact_forecast_monthly.forecast_net_revenue_nad IS 'Point forecast of net revenue, N$. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.lower_80_nad IS 'Lower bound of the 80% prediction interval. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.upper_80_nad IS 'Upper bound of the 80% prediction interval. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.lower_95_nad IS 'Lower bound of the 95% prediction interval. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.upper_95_nad IS 'Upper bound of the 95% prediction interval. | Source: forecasting.py';
        COMMENT ON COLUMN dw.fact_forecast_monthly.model_run_at IS 'Audit: model run timestamp. | Source: forecasting.py';
```

```
        CREATE TABLE dw.customer_segment (
            customer_code       varchar(10) PRIMARY KEY,
            segment_id          smallint NOT NULL,
            segment_name        varchar(40) NOT NULL,
            recency_days        integer NOT NULL,
            frequency_invoices  integer NOT NULL,
            monetary_nad        numeric(16, 2) NOT NULL,
            avg_order_kg        numeric(14, 1) NOT NULL,
            product_breadth     smallint NOT NULL,
            silhouette_sample   numeric(6, 4),
            model_run_at        timestamptz NOT NULL DEFAULT now()
        );
        COMMENT ON TABLE dw.customer_segment IS 'Model output: K-means customer segment per durable customer code (RFM + basket features,
trailing 12 months).';
        COMMENT ON COLUMN dw.customer_segment.customer_code IS 'Durable customer code. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.segment_id IS 'Cluster number. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.segment_name IS 'Business label of the cluster. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.recency_days IS 'Days since last invoice at 30 Sep 2026. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.frequency_invoices IS 'Distinct invoices in the trailing 12 months. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.monetary_nad IS 'Net revenue in the trailing 12 months, N$. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.avg_order_kg IS 'Average kg per invoice line. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.product_breadth IS 'Distinct products bought. | Source: clustering.py';
        COMMENT ON COLUMN dw.customer_segment.silhouette_sample IS 'Silhouette value of the customer within its cluster. | Source:
clustering.py';
        COMMENT ON COLUMN dw.customer_segment.model_run_at IS 'Audit: model run timestamp. | Source: clustering.py';

        CREATE TABLE dw.yield_driver_importance (
            model_name     varchar(40),
            feature        varchar(40),
            importance     numeric(10, 4),
            coefficient    numeric(12, 4),
            p_value        numeric(10, 6),
            rank_order     smallint,
            model_run_at   timestamptz NOT NULL DEFAULT now(),
            PRIMARY KEY (model_name, feature)
        );
        COMMENT ON TABLE dw.yield_driver_importance IS 'Model output: yield-driver coefficients (OLS) and permutation importances (random
forest).';
        COMMENT ON COLUMN dw.yield_driver_importance.model_name IS 'Model identifier. | Source: yield_drivers.py';
        COMMENT ON COLUMN dw.yield_driver_importance.feature IS 'Explanatory feature. | Source: yield_drivers.py';
        COMMENT ON COLUMN dw.yield_driver_importance.importance IS 'Permutation importance (drop in R2). | Source: yield_drivers.py';
        COMMENT ON COLUMN dw.yield_driver_importance.coefficient IS 'OLS coefficient on log-yield (approx. % effect). | Source:
yield_drivers.py';
        COMMENT ON COLUMN dw.yield_driver_importance.p_value IS 'OLS p-value. | Source: yield_drivers.py';
        COMMENT ON COLUMN dw.yield_driver_importance.rank_order IS 'Rank by importance. | Source: yield_drivers.py';
        COMMENT ON COLUMN dw.yield_driver_importance.model_run_at IS 'Audit: model run timestamp. | Source: yield_drivers.py';

        CREATE TABLE dw.harvest_anomaly (
            source_file      varchar(40),
            source_row_num   integer,
            farm_code        varchar(10),
            crop_code        char(3),
            harvest_date     date,
            harvested_kg     numeric(14, 1),
            anomaly_score    numeric(10, 4),
            method           varchar(30),
            PRIMARY KEY (source_file, source_row_num, method)
        );
        COMMENT ON TABLE dw.harvest_anomaly IS 'Model output: harvest records scored as anomalous by Isolation Forest / robust z-score (for
steward review).';
        COMMENT ON COLUMN dw.harvest_anomaly.source_file IS 'Lineage: harvest log file. | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.source_row_num IS 'Lineage: row in the file. | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.farm_code IS 'Farm code. | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.crop_code IS 'Crop code. | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.harvest_date IS 'Harvest date. | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.harvested_kg IS 'Harvested kg as loaded. | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.anomaly_score IS 'Anomaly score (higher = more anomalous). | Source: anomaly_detection.py';
        COMMENT ON COLUMN dw.harvest_anomaly.method IS 'Detection method. | Source: anomaly_detection.py';

        -- ---------------------------------------------------------------------------
        -- Security mapping for dynamic RLS (pseudonymous test accounts on the reserved
        -- .example domain - replace with real Entra ID UPNs in the Power BI service).
        -- ---------------------------------------------------------------------------
        CREATE TABLE sec.user_zone_access (
            user_upn   varchar(80) NOT NULL,
            zone_code  char(2)     NOT NULL,
            granted_by varchar(40) NOT NULL,
            valid_from date        NOT NULL DEFAULT current_date,
            PRIMARY KEY (user_upn, zone_code)
        );
        COMMENT ON TABLE sec.user_zone_access IS 'Maps Power BI user principal names to the management zones they may see (dynamic RLS).';
        COMMENT ON COLUMN sec.user_zone_access.user_upn IS 'User principal name (Entra ID sign-in) matched by USERPRINCIPALNAME() in the RLS
rule. | Source: Head of Agriculture access request';
        COMMENT ON COLUMN sec.user_zone_access.zone_code IS 'Management zone the user may see (NC, NE, NW, CE, SO); one row per zone granted. |
Source: access request';
        COMMENT ON COLUMN sec.user_zone_access.granted_by IS 'Role that approved the access (accountability). | Source: access request';
        COMMENT ON COLUMN sec.user_zone_access.valid_from IS 'Date the access became effective (recertified quarterly). | Source: access
request';
        INSERT INTO sec.user_zone_access (user_upn, zone_code, granted_by) VALUES
         ('zm.northcentral@greenbasket.example', 'NC', 'Head of Agriculture'),
         ('zm.northeast@greenbasket.example',    'NE', 'Head of Agriculture'),
         ('zm.central@greenbasket.example',      'CE', 'Head of Agriculture'),
         ('zm.south@greenbasket.example',        'SO', 'Head of Agriculture'),
         ('ops.north@greenbasket.example',       'NC', 'Head of Agriculture'),
         ('ops.north@greenbasket.example',       'NE', 'Head of Agriculture');
```

Appendix D – Diagram sources and package inventory

Mermaid sources of the diagrams

File: `06_report/diagrams/bpmn.mmd` (lines 1–26)

```
flowchart TB
  subgraph FARM["Farm operations - farm manager, agronomist"]
    direction LR
    A([Season<br/>plan]) --> B[Planting<br/>register - S1] --> C[Field operations<br/>S5 cost ledger] --> D{Harvest<br/>ready?}
    D -- no --> C
    D -- yes --> E[Harvest log<br/>per block - S1]
  end
  subgraph PACK["Packhouse and stock - inventory controller"]
    direction LR
    F[Grade and pack<br/>cull removed] --> G[Stock receipt<br/>S4 stock system] --> H[Month-end<br/>stock count]
  end
  subgraph SALES["Sales and export desk"]
    direction LR
    I[Order<br/>S2 customer] --> J[Dispatch<br/>from stock] --> K[Invoice or credit<br/>note - S3 ERP] --> L([Payment<br/>received])
  end
  subgraph BI["BI platform - nightly"]
    direction LR
    X[(External<br/>S6-S9)] -.-> M[Extract<br/>to landing] --> N{DQ rules<br/>pass?}
    N -- no --> O[Quarantine,<br/>steward queue]
    N -- yes --> P[Load star<br/>schema] --> Q[Reconcile<br/>R01-R19] --> R([Refresh<br/>Power BI])
  end
  FARM -- produce --> PACK
  PACK -- stock --> SALES
  FARM -. logs, ledger .-> BI
  PACK -. stock extract .-> BI
  SALES -. ERP extract .-> BI
```

File: `06_report/diagrams/architecture.mmd` (lines 1–38)

```
flowchart LR
  subgraph SRC["Sources S1-S9"]
    direction TB
    S1["S1 Farm harvest logs, 14 CSV,<br/>3 layouts + planting register"]
    S2["S2 Master data workbook<br/>10 sheets + day-2 delta"]
    S3["S3 ERP invoice lines<br/>6 gz CSV, overlapping"]
    S4["S4 Stock system, monthly CSV"]
    S5["S5 Finance cost ledger, gz CSV"]
    S6["S6 NASA POWER daily JSON"]
    S7["S7 World Bank Pink Sheet xlsx"]
    S8["S8 FRED FX, 3 CSV"]
    S9["S9 NSA / MAFWLR estimates"]
    S1 ~~~ S2 ~~~ S3 ~~~ S4 ~~~ S5 ~~~ S6 ~~~ S7 ~~~ S8 ~~~ S9
  end
  subgraph PG["PostgreSQL 16 - green_basket_dw"]
    direction TB
    L["stg.*_raw - landing<br/>all text, lossless,<br/>source_file + row_num"]
    C["stg.*_clean - typed,<br/>conformed codes,<br/>dq_status / dq_note"]
    R["ref.* - alias + xref,<br/>UoM, FX, country"]
    W["dw.* - 9 facts,<br/>9 conformed dims, SCD2"]
    M["mart.* - BI views,<br/>aggregate table"]
    A["audit.* - batches, steps,<br/>DQ issues, reconciliation"]
    L --> C --> W --> M
    R --> C
    C -.-> A
    W -.-> A
  end
  subgraph AN["Python analytics"]
    direction TB
    PY["forecast, segments,<br/>yield drivers, anomalies"]
  end
  subgraph PBI["Power BI"]
    direction TB
    SM["Semantic model, Import<br/>79 measures, 3 RLS roles,<br/>incremental refresh"] --> RP["Report: 4 pages<br/>app + subscriptions"]
  end
  SRC --> PG
  PG <--> AN
  PG --> PBI
```

File: `06_report/diagrams/lineage.mmd` (lines 1–22)

```
flowchart LR
  subgraph E["1 Extract - Python"]
    direction TB
    E1["COPY raw files into stg.*_raw<br/>all columns as text<br/>+ source_file + row_num"]
    E2["harvest layout detection<br/>templates A / B / C"]
    E1 --- E2
  end
  subgraph T["2 Transform - 05_etl_transform.sql"]
    direction TB
    T1["master data MD01-05"] --> T2["market + FX MP01, FX01"] --> T3["planting PL01-02"]
    T3 --> T4["harvest HV01-08"] --> T5["sales SA01-08"] --> T6["costs CL01-04"]
    T6 --> T7["stock ST01-03"] --> T8["weather WE01-02"] --> T9["NSA CE01-02"]
  end
  subgraph LD["3 Load - 06 / 07"]
    direction TB
    D1["dimensions: date, region,<br/>crop hierarchy, cost category,<br/>commodity, farm SCD2,<br/>customer SCD2"] --> F1["facts: sales,
harvest,<br/>crop cycle, cost, inventory,<br/>weather, prices, estimates"]
  end
```

```
  subgraph Q["4 Control"]
    direction TB
    R1["sp_reconcile R01-R19"] --> O1["13_optimisation.sql<br/>indexes, BRIN, aggregate"] --> B2["batch 2: day-2 delta<br/>SCD2 master data"]
  end
  E --> T --> LD --> Q
```

File: `06_report/diagrams/erd_sales.mmd` (lines 1–82)

```
erDiagram
  DIM_DATE ||--o{ FACT_SALES : "invoice_date_key (active), dispatch_date_key"
  DIM_PRODUCT ||--o{ FACT_SALES : product_key
  DIM_CUSTOMER ||--o{ FACT_SALES : "customer_key (SCD2 as-was)"
  DIM_FARM ||--o{ FACT_SALES : "farm_key (origin, SCD2)"
  DIM_DATE ||--o{ FACT_INVENTORY_MONTHLY : snapshot_date_key
  DIM_FARM ||--o{ FACT_INVENTORY_MONTHLY : farm_key
  DIM_PRODUCT ||--o{ FACT_INVENTORY_MONTHLY : product_key
  DIM_CROP ||--o{ DIM_PRODUCT : "crop_key (snowflake)"
  DIM_CROP_CATEGORY ||--o{ DIM_CROP : "crop_category_key (snowflake)"
  DIM_REGION ||--o{ DIM_FARM : "region_key (snowflake)"
  FACT_SALES {
    bigint sales_line_key PK
    int invoice_date_key FK
    int dispatch_date_key FK
    int product_key FK
    int customer_key FK
    int farm_key FK
    varchar invoice_no "degenerate"
    smallint line_no "degenerate"
    numeric quantity_kg "additive"
    numeric net_revenue_nad "additive"
    numeric cogs_nad "additive"
    boolean is_price_anomaly "DQ flag SA07"
  }
  FACT_INVENTORY_MONTHLY {
    bigint inventory_snapshot_key PK
    int snapshot_date_key FK
    int farm_key FK
    int product_key FK
    numeric closing_stock_kg "semi-additive"
    numeric received_kg "additive"
    numeric dispatched_kg "additive"
  }
  DIM_DATE {
    int date_key PK "YYYYMMDD"
    date full_date
    smallint fiscal_year "Apr-Mar"
    char crop_season_label "Oct-Sep"
    boolean is_public_holiday "Namibia"
  }
  DIM_PRODUCT {
    smallint product_key PK
    varchar product_code "natural key"
    smallint crop_key FK
    varchar pack_type
    numeric net_weight_kg
  }
  DIM_CROP {
    smallint crop_key PK
    char crop_code "natural key"
    smallint crop_category_key FK
    boolean is_nab_controlled
    numeric typical_cull_pct
  }
  DIM_CROP_CATEGORY {
    smallint crop_category_key PK
    varchar category_name
  }
  DIM_CUSTOMER {
    int customer_key PK
    varchar customer_code "natural key"
    varchar channel "SCD2"
    smallint credit_terms_days "SCD2"
    date scd_effective_from
    date scd_effective_to
    boolean scd_is_current
  }
  DIM_FARM {
    int farm_key PK
    varchar farm_code "natural key"
    smallint region_key FK
    varchar irrigation_type "SCD2"
    numeric cultivated_ha "SCD2"
    date scd_effective_from
    date scd_effective_to
  }
  DIM_REGION {
    smallint region_key PK
    char region_code "ISO 3166-2"
    varchar zone_code "management zone"
  }
```

File: `06_report/diagrams/erd_production.mmd` (lines 1–54)

```
%%{init: {"er": {"layoutDirection": "LR"}}}%%
erDiagram
  DIM_DATE ||--o{ FACT_HARVEST : harvest_date_key
  DIM_FARM ||--o{ FACT_HARVEST : farm_key
  DIM_CROP ||--o{ FACT_HARVEST : crop_key
```

```
  DIM_DATE ||--o{ FACT_CROP_CYCLE : "planting (active), first/last harvest, season end"
  DIM_FARM ||--o{ FACT_CROP_CYCLE : "farm_key (as-was at planting)"
  DIM_CROP ||--o{ FACT_CROP_CYCLE : crop_key
  DIM_DATE ||--o{ FACT_PRODUCTION_COST : posting_date_key
  DIM_FARM ||--o{ FACT_PRODUCTION_COST : farm_key
  DIM_CROP ||--o{ FACT_PRODUCTION_COST : "crop_key (-2 = overhead)"
  DIM_COST_CATEGORY ||--o{ FACT_PRODUCTION_COST : cost_category_key
  FACT_HARVEST {
    bigint harvest_event_key PK
    int harvest_date_key FK
    int farm_key FK
    smallint crop_key FK
    varchar block_id "degenerate"
    numeric harvested_kg "additive"
    boolean is_corrected "rule HV06"
  }
  FACT_CROP_CYCLE {
    bigint crop_cycle_key PK
    int farm_key FK
    smallint crop_key FK
    int planting_date_key FK
    int harvest_season_end_date_key FK
    varchar cycle_id "degenerate"
    varchar irrigation_method "block level"
    numeric planted_ha "additive"
    numeric harvested_kg "additive"
    numeric direct_cost_nad "additive"
  }
  FACT_PRODUCTION_COST {
    bigint cost_posting_key PK
    int posting_date_key FK
    int farm_key FK
    smallint crop_key FK
    smallint cost_category_key FK
    numeric amount_nad "additive"
  }
  DIM_COST_CATEGORY {
    smallint cost_category_key PK
    varchar cost_group
  }
  DIM_DATE {
    int date_key PK
  }
  DIM_FARM {
    int farm_key PK
  }
  DIM_CROP {
    smallint crop_key PK
  }
```

Package inventory

Table 52. Folder structure of the submission package

Folder	Contents
`01_data/`	`generate_synthetic_data.py` (seeded generator), `fetch_public_data.py` (real public data), `raw/` (landed source files: farm logs, master data and delta, ERP, stock, ledger, NASA POWER JSON, Pink Sheet, FRED, NSA) with `generator_manifest.json`
`02_sql/`	`00`–`13` SQL scripts (database, DDL, functions, transforms, loads, reconciliation, marts, grants, OLAP, evidence, optimisation) and `run_pipeline.py`
`03_analytics/`	`common.py`, `data_profiling.py`, `forecasting.py`, `clustering.py`, `yield_drivers.py`, `anomaly_detection.py`, `build_data_dictionary.py`, `capture_sql_evidence.py`, `validate_dq_rules.py`, `performance_benchmark.py`
`04_powerbi/`	`GreenBasket_Agri.pbip` + `.SemanticModel/` (TMDL) + `.Report/` (PBIR), `model_spec.py`, `build_pbip.py`, `export_fallbacks.py`, `measures.dax`, `GreenBasket_model_script.tmdl`, `data_csv/`, `theme/`, `tools/TomTool` (TOM validator), `BUILD_GUIDE.md`
`05_backup/`	`green_basket_dw_pg16_warehouse.backup` (pg_dump -Fc), `green_basket_dw_schema_ddl.sql`
`06_report/`	This PDF; `src/` (report text), `evidence/` (generated tables), `figures/`, `diagrams/`, `data_dictionary.md/.xlsx`, `build_mockups.py`, `build_report.py`, `tools/` (render.js, template, CSS, title page)
`07_presentation/`	`GreenBasket_ABI820S_Presentation.pptx`, `deck_data.py` (numbers from the warehouse), `deck_data.json`, `build_presentation.js` (pptxgenjs)
root	`README.md`, `run_all.py`, `requirements.txt`